

HelixCode Constitution

HelixCode Project Constitution

Version: 1.0.0 **Effective Date:** 2026-04-30 **Scope:** This Constitution applies to HelixCode and ALL its submodules **Authority:** Cascaded from HelixAgent root governance with HelixCode-specific addenda

INHERITED FROM constitution/Constitution.md (HelixConstitution submodule)

All universal rules in constitution/Constitution.md apply unconditionally to HelixCode. This project Constitution **extends** those universal rules with HelixCode-specific addenda below — it does NOT and MAY NOT weaken any universal clause. When this Constitution disagrees with the constitution submodule, the constitution submodule wins.

The constitution submodule is at ./constitution/ (added 2026-05-14 per user mandate, pinned at the upstream HelixDevelopment/HelixConstitution HEAD). To locate it from nested submodules at arbitrary depth, source constitution/find_constitution.sh from the parent project tree.

Project remote-policy scope (revised 2026-06-03) — Per operator decision 2026-06-03, HelixCode aligns with the universal Constitution's multi-upstream Git-push allowance: GitHub (vasic-digital/*, HelixDevelopment/*), GitLab (same), GitFlic, and GitVerse are ALL permitted as Git remotes, and pushes target every configured upstream. This **SUPERSEDES** the earlier CLAUDE.md §6.W tightening that forbade GitFlic/GitVerse — that remote-provider restriction is **LIFTED**. Project tightenings unrelated to remote providers (host power management, anti-bluff, secret-leak, etc.) remain in force.

Preamble

HelixCode is an enterprise-grade distributed AI development platform. This Constitution establishes the non-negotiable rules that govern all development, testing, deployment, and maintenance activities within the project. Every contributor, agent, and automated process **MUST** adhere to these rules. No exceptions.

CONST-001: No CI/CD Pipelines (Permanent)

No `.github/workflows/`, `.gitlab-ci.yml`, Jenkinsfile, `.travis.yml`, `.circleci/`, or any automated pipeline. No Git hooks. All builds and tests run manually or via Makefile/script targets.

Rationale: Manual execution ensures human oversight and prevents automated propagation of bluffs.

CONST-002: No Mocks in Production (Permanent)

CONST-002a: Production Code

Mocks, stubs, fakes, placeholder classes, TODO implementations are STRICTLY FORBIDDEN in production code. All production code is fully functional with real integrations.

CONST-002b: Test Code

Mocks/stubs/fakes MAY be used ONLY in unit tests (files ending `_test.go` run under `go test -short`).

Rationale: Production bluffs have repeatedly been discovered where features appeared implemented but were non-functional.

CONST-003: No HTTPS for Git (Permanent)

SSH URLs only (`git@github.com:...`, `git@gitlab.com:...`, etc.) for clones, fetches, pushes, and submodule updates. SSH keys are configured on every service.

CONST-004: No Manual Container Commands (Permanent)

Container orchestration is owned by the project's binary/orchestrator (e.g., `make build` → `./bin/<app>`). Direct `docker/podman start|stop|rm` and `docker-compose up|down` are prohibited as workflows.

CONST-005: 100% Real Data for Non-Unit Tests

Beyond unit tests, all components MUST use actual API calls, real databases, live services. No simulated success. Fallback chains tested with actual failures.

Verification: Every integration/E2E test MUST connect to real services or skip (not fail) if unavailable.

CONST-006: Challenge Coverage (Permanent)

Every component MUST have Challenge scripts (`./challenges/scripts/`) validating real-life use cases. No false success — validate actual behavior, not return codes.

CONST-007: Health & Observability

Every service MUST expose health endpoints. Circuit breakers for all external dependencies. Prometheus / OpenTelemetry integration where applicable.

CONST-008: Documentation & Quality

Update `CLAUDE.md`, `AGENTS.md`, and relevant docs alongside code changes. Pass language-appropriate format/lint/security gates. Conventional Commits: `<type>(<scope>): <description>`.

CONST-009: Validation Before Release

Pass the project's full validation suite (make `ci-validate-all-equivalent`) plus all challenges (`./challenges/scripts/run_all_challenges.sh`).

CONST-010: Comprehensive Verification

Every fix MUST be verified from all angles: runtime testing (actual HTTP requests / real CLI invocations), compile verification, code structure checks, dependency existence checks, backward compatibility, and no false positives. Grep-only validation is NEVER sufficient.

CONST-011: Resource Limits for Tests & Challenges

ALL test and challenge execution MUST be strictly limited to 30-40% of host system resources. Use `GOMAXPROCS=2, nice -n 19, ionice -c 3, -p 1` for go test. Container limits required.

CONST-012: Bugfix Documentation

All bug fixes MUST be documented in docs/issues/fixed/BUGFIXES.md with root cause analysis, affected files, fix description, and a link to the verification test/challenge.

CONST-013: Real Infrastructure for All Non-Unit Tests

Mocks/fakes/stubs/placeholders MAY be used ONLY in unit tests. ALL other test types — integration, E2E, functional, security, stress, chaos, challenge, benchmark, runtime verification — MUST execute against REAL running systems with REAL containers, REAL databases, REAL services, and REAL HTTP calls.

CONST-014: Reproduction-Before-Fix (Mandatory)

Every reported error, defect, or unexpected behavior MUST be reproduced by a Challenge script BEFORE any fix is attempted. Sequence: 1. Write the Challenge first 2. Run it; confirm fail (it reproduces the bug) 3. Then write the fix 4. Re-run; confirm pass 5. Commit Challenge + fix together

The Challenge becomes the regression guard for that bug forever.

CONST-015: Concurrent-Safe Containers

Any struct field that is a mutable collection (map, slice) accessed concurrently MUST use thread-safe primitives. Bare `sync.Mutex` + `map/slice` combinations are prohibited for new code.

CONST-016: Definition of Done (Universal)

A change is NOT done because code compiles and tests pass. “Done” requires pasted terminal output from a real run.

- **No self-certification:** Words like *verified*, *tested*, *working*, *complete*, *fixed*, *passing* are forbidden in commits/PRs/replies unless accompanied by pasted output from a command that ran in that session.
 - **Demo before code:** Every task begins by writing the runnable acceptance demo
 - **Real system, every time:** Demos run against real artifacts
 - **Skips are loud:** `t.Skip` without a trailing `SKIP-OK: #<ticket>` comment breaks validation
-

CONST-035 — Anti-Bluff Tests & Challenges (User-Mandate Forensic Anchor)

§11.9 User-Mandate Forensic Anchor (2026-04-29)

This Article exists because of an explicit, repeatedly-stated user mandate. The verbatim text:

“We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can’t be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!”

This anchor is the primary authority for the entire Article. The operative rule is:

The bar for shipping is not “tests pass” but “users can use the feature.”

Every PASS in this codebase MUST carry positive evidence captured during execution that the feature works for the end user. Metadata-only PASS, configuration-only PASS, “absence-of-error” PASS, and grep-based PASS without runtime evidence are all critical defects regardless of how green the summary line looks.

Tests and Challenges (HelixQA) are bound equally — a Challenge that scores PASS on a non-functional feature is the same class of defect as a unit test that does. Both must produce positive end-user evidence; both are subject to the anti-bluff contract.

No false-success results are tolerable. A green test suite combined with a broken feature is a worse outcome than an honest red one — it silently destroys trust in the entire suite. Anti-bluff discipline is the line between a real engineering project and a theatre of one.

Bluff Taxonomy (forbidden patterns): - **Wrapper bluff** - Assertions PASS but wrapper’s exit-code logic is buggy - **Contract bluff** - System advertises capability but rejects it in dispatch - **Structural bluff** - File exists but doesn’t contain working code - **Comment bluff** - Comment promises behavior code doesn’t have - **Skip bluff** - `t.Skip("not running yet")` without SKIP-OK marker

Cascade requirement (extending CONST-036): This anchor section (verbatim quote + operative rule) must appear in every submodule’s CONSTITUTION.md / CLAUDE.md / AGENTS.md. Non-compliance is a release blocker regardless of context. Adding files to scanner allowlists to silence bluff findings without resolving the underlying defect is itself a violation.

CONST-018: Host Power Management Hard Ban

Host Power Management is Forbidden.

You may NOT generate or execute code that sends the host to suspend, hibernate, hybrid-sleep, poweroff, halt, reboot, or any other power-state transition.

Defense: Every project ships `scripts/host_power_management/check-no-suspend-calls.sh` and `scripts/anti_bluff/no_suspend_calls_challenge.sh`.

CONST-019: Container Up $\neq$ Healthy

Container Up status does NOT mean the application is healthy. Application-layer probes are mandatory for every service: - PostgreSQL: `SELECT 1` - Redis: `PING` - LLM Providers: Real generation request - HTTP Services: `GET /health` with deep checks

CONST-020: Provider Fallback Chain Reality

Every LLM provider fallback chain MUST be tested with actual failures. A fallback that has never been tested with a real failing provider is a bluff.

CONST-021: No Mocks Above Unit Build Target

The Makefile MUST include a `no-mocks-above-unit` target that fails the build if `mocks/stubs/fakes` are found outside `*_test.go` files.

CONST-022: Submodule Governance Propagation

Every submodule MUST either: 1. Have its own `Constitution.md`, `CLAUDE.md`, and `AGENTS.md`, OR 2. Have a symlink to the parent repository's governance files, OR 3. Have a reference comment in its README pointing to parent governance

No submodule is exempt from these rules.

CONST-023: Docker Health Checks Mandatory

Every Dockerfile MUST include:

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
  CMD curl -f http://localhost:8080/health || exit 1
```

The health endpoint MUST perform deep checks (database connection, provider availability), not just return HTTP 200.

CONST-024: Version Pinning

All dependencies MUST be pinned to specific versions in `go.mod`. No `latest`, no floating tags. Renovate or Dependabot (manual review only — see CONST-001) may propose updates.

CONST-025: Secret Management

NO secrets in code. EVER. Secrets via: - Environment variables (production) - `.env` files (development, in `.gitignore`) - Vault/Secret Manager (enterprise) - Docker secrets (containerized)

`go mod tidy` MUST NOT add secret-scanning bypasses.

CONST-026: Minimal Privilege Containers

Containers run as non-root. Every Dockerfile:

```
RUN adduser -D -u 1001 helixcode
USER helixcode
```

CONST-027: Network Isolation

Container orchestration MUST use internal networks. Services communicate via named hosts, not exposed ports where possible.

CONST-028: Backup Before Destructive Operations

Every file editing tool MUST create backups before modification. The backup MUST be restorable.

CONST-029: Input Validation at All Boundaries

Every public function MUST validate inputs. No trust of caller-provided data. SQL injection, path traversal, command injection MUST be impossible by design.

CONST-030: Graceful Degradation

When external services are unavailable, the system **MUST** degrade gracefully: - Return partial results where possible - Queue operations for retry - Inform user of degraded state - **NEVER** crash or hang indefinitely

CONST-031: Audit Trail

Every significant operation **MUST** be logged with: - Timestamp - User identity - Operation type - Success/failure status - Resource affected

Log retention: 90 days minimum.

CONST-032: Emergency Stop

Every long-running or distributed operation **MUST** support cancellation via context.Context. Users **MUST** be able to interrupt any operation.

CONST-033: Data Integrity

Database writes **MUST** be transactional. Partial writes **MUST** be rolled back. Consistency checks **MUST** run periodically.

CONST-034: API Stability

Public APIs maintain backward compatibility within major versions. Deprecation requires: - 6-month notice - Migration guide - Compatibility shim

CONST-035: End-User Usability Mandate (2026-04-29 Strengthening)

A test or Challenge that **PASSES** is a **CLAIM** that the tested behavior **works for the end user of the product**.

The HelixAgent project has repeatedly hit the failure mode where every test ran green **AND** every Challenge reported **PASS**, yet most product features did not actually work. This **MUST NOT** recur in HelixCode.

Every **PASS** result **MUST** guarantee: a. **Quality** - correct behavior under real inputs, edge cases, concurrency b. **Completion** - wired end-to-end with no stub/placeholder gaps c. **Full usability** - a user following documented request shapes **SUCCEEDS**

A passing test that doesn't certify all three is a **bluff** and MUST be tightened.

Bluff taxonomy (each pattern observed and now forbidden): - **Wrapper bluff** - assertions PASS but wrapper's exit-code logic is buggy - **Contract bluff** - system advertises capability but rejects it in dispatch - **Structural bluff** - `check_file_exists` passes but doesn't run the test - **Comment bluff** - comment promises behavior code doesn't actually have - **Skip bluff** - `t.Skip("not running yet")` without SKIP-OK: #<ticket> marker

Full background: docs/HOST_POWER_MANAGEMENT.md and this Constitution (CONST-035).

CONST-036: Propagation to Submodules

This Constitution, along with CLAUDE.md and AGENTS.md, MUST be propagated to ALL submodules. Each submodule's governance MUST reference this parent Constitution. Changes to this Constitution MUST trigger review of all submodule governance files.

CONST-037: LLMsVerifier Single Source of Truth Mandate

Rule: LLMsVerifier SHALL BE the sole authoritative source for: 1. All model metadata (names, IDs, context windows, capabilities) 2. All provider metadata (endpoints, auth types, supported models) 3. All verification status (verified, partial, failed, pending) 4. All scoring data (overall scores, capability scores, tier rankings) 5. All rate-limit and cooldown state

Prohibition: NO hardcoded model lists, NO hardcoded provider lists, NO simulated model discovery. Any code path that presents a model or provider listing to a user MUST fetch that listing from the LLMsVerifier subsystem or its cached replica.

Anti-Bluff Verification: - The challenge script `scripts/anti_bluff/verifier_hardcode_check.sh` MUST scan all Go source files for hardcoded model arrays. - Any `[]string{"gpt-4", "claude-3"}` or equivalent literal in production code is a constitutional violation. - The only permitted hardcoded data is the LLMsVerifier service endpoint URL and the list of verification test types.

Enforcement: `make test-complete` MUST include a test that asserts `ModelManager.GetAvailableModels()` returns at least as many models as the verifier's database contains for configured providers. A test that passes while the CLI shows a hardcoded list is a TEST BLUFF and violates CONST-035.

CONST-038: Model Provider Anti-Bluff Guarantee

Rule: Every model displayed to an end user MUST have been verified by LLMsVerifier within the last `verification_timeout` period (default: 24h). Models older than this MUST display a “stale” indicator and be deprioritized.

Prohibition Against Test Bluffing: - A unit test that mocks the verifier client and asserts `GetAvailableModels()` returns 3 models DOES NOT satisfy this rule. - An integration test that starts the verifier server, performs real provider discovery, and confirms the model count matches the actual provider API response DOES satisfy this rule. - The Makefile target `make test-verifier-integration` MUST exist and MUST run without mocks.

The “Tests Pass But Features Don’t Work” Guarantee:

NO TEST MAY PASS UNLESS THE FEATURE IT TESTS IS DEMONSTRABLY USABLE BY AN END USER IN THE SAME BUILD.

- If `TestModelList` passes but `helixcode --list-models` shows hardcoded data, the test is a BLUFF.
- If `TestProviderHealth` passes but the health endpoint returns `200 OK` for a provider that is actually down, the test is a BLUFF.
- If `TestLLMGeneration` passes but `--prompt "hello"` returns a simulated string, the test is a BLUFF.
- Bluff tests MUST be rewritten or deleted. There is no “grandfather” exception.

Evidence Standard: Every test that claims to verify model/provider functionality MUST: 1. Call a real API endpoint or a real verifier database 2. Assert on response content that could only come from that real source 3. Include a test that runs the CLI binary with `--list-models` and checks output against verifier data

CONST-039: Real-Time Model Status Accuracy

Rule: Model status (available, rate-limited, cooldown, offline, deprecated) displayed to users MUST reflect the actual state as known by LLMsVerifier within `max_staleness` seconds (default: 60s).

Polling vs. Push: - If WebSocket/SSE push is unavailable, the system MUST poll LLMsVerifier at most every `status_poll_interval` (default: 30s). - The TUI MUST display a “last updated” timestamp with every model listing. - Models in “cooldown” or “rate-limited” state MUST show the estimated recovery time if known.

Accuracy Verification: - Challenge script `challenges/scripts/model_status_accuracy_challenge.sh` MUST: 1. Artificially rate-limit a provider by exhausting its quota 2. Wait for the status to propagate to the verifier 3. Check that `helixcode --list-models` shows the rate-limited status within 60s 4. Check that `SelectOptimalModel()` no longer selects the rate-limited model

Prohibition: Status indicators that are “always green” or that lag >60s behind reality violate this rule.

CONST-040: All Providers and Models Integration Mandate

Rule: HelixCode MUST integrate with ALL providers and models that LLMsVerifier supports, subject only to: 1. The provider being explicitly disabled in configuration (enabled: false) 2. The API key being absent and the provider requiring one 3. The provider being marked deprecated in the verifier database

Minimum Provider Set (SHALL NOT be reduced without constitutional amendment):

Provider	Auth Type	Required Env Var
OpenAI	API Key	OPENAI_API_KEY
Anthropic	API Key / OAuth	ANTHROPIC_API_KEY
Gemini	API Key	GEMINI_API_KEY
DeepSeek	API Key	DEEPSEEK_API_KEY
Groq	API Key	GROQ_API_KEY
Mistral	API Key	MISTRAL_API_KEY
xAI	API Key	XAI_API_KEY
OpenRouter	API Key	OPENROUTER_API_KEY
Ollama	Local	None (auto-detect)
Llama.cpp	Local	None (auto-detect)

Integration Requirement: For every provider in the minimum set: - There MUST be a provider adapter file in internal/llm/ or internal/verifier/adapters/ - There MUST be a *_test.go file with real API tests (skipped only if HELIX_SKIP_LIVE_PROVIDER_TESTS is set) - There MUST be a challenge script in challenges/scripts/ - The model listing MUST include models from this provider when the provider is enabled

CONST-041: MCP / LSP / ACP / Embedding / RAG / Skills / Plugins Integration Mandate

Rule: LLMsVerifier integration SHALL extend beyond basic model listing to cover ALL capability dimensions:

- MCP (Model Context Protocol):** The verifier MUST report which models support MCP tool calling. HelixCode's MCP subsystem MUST consult verifier capability flags before selecting a model for tool-use tasks.
- LSP (Language Server Protocol):** The verifier MUST report code-analysis capabilities. Models without code_analysis capability MUST NOT be selected for refactoring or debugging tasks.
- ACP (Agent Capability Protocol):** The verifier MUST report multi-agent coordination support. Models with supports_parallel_tool_use MUST be preferred for ACP workflows.
- Embedding:** The verifier MUST report supports_embeddings for each model. The CogneConfig embedding model selection MUST be verifier-aware.
- RAG (Retrieval-Augmented Generation):** The verifier MUST report context-window sizes. RAG chunking strategies MUST adapt to the selected model's context_window_tokens as reported by the verifier.

6. Skills / Plugins: The verifier MUST track plugin compatibility. Models flagged `plugin_compatible` MUST be used when skill/plugin execution is required.

Capability Checklist (MUST be verified by challenge): - [] MCP tool calling verified for at least 3 providers - [] LSP code-analysis verified for at least 3 providers - [] ACP parallel tool use verified for at least 2 providers - [] Embedding generation verified for at least 2 providers - [] RAG context-window adaptation verified - [] Skills/plugin execution verified for at least 2 providers

Prohibition: Capability flags MUST NOT be hardcoded. The `Provider.GetCapabilities()` method MUST return data sourced from the verifier's `VerificationResult` fields.

CONST-046: No Hardcoded Content — All User-Facing Text Must Be Dynamic

Rule: NO user-facing text, prompt template, question text, error message, label, helper text, or explanatory content may be hardcoded as a static literal string in any source file, test file, configuration template, script, or governance document. All such content MUST be either: 1. Generated dynamically by an LLM at runtime based on context (user's language, prompt content, session state), OR 2. Loaded from a resource file (`.yaml`, `.json`, `.toml`) that supports `i18n` and can be overridden per locale, OR 3. Composed programmatically from verifier-provided metadata or configuration data

Rationale: Hardcoded English-only text creates a system that silently breaks for non-English users. A clarification question hardcoded as "Which file has the bug?" could be asked identically to a Japanese, Serbian, or Spanish user who specified their request in that language — producing an incoherent, unusable experience. Every piece of user-facing text MUST adapt to the user's language and context. A static string array is a constitutional violation.

Prohibition: Static literal arrays/slices of question text, fixed prompt templates with hardcoded English phrases, hardcoded error messages in anything other than Go standard library format (which uses English identifiers but is not user-facing), and hardcoded label/display text in TUI, CLI, desktop, or mobile interfaces.

Examples of violations: - `[]string{"Which file has the bug?", "What is the expected behavior?"}` — hardcoded question bank - `fmt.Sprintf("The build failed with: %s", err)` — user-visible English template - `textView.NewTextView().SetText("Press Enter to continue")` — hardcoded UI label - `return "feature not implemented"` — user-visible static string

Examples of compliant patterns: - Questions generated by `llm.Generate(ctx, prompt)` and parsed into structured output - Labels loaded from `locales/en.yaml` with fallback to verifier metadata - Error wrapping: `fmt.Errorf("build: %w", err)` — identifier-based, not user-facing prose - Dynamic formatting: `s.formatMessage(ctx, key, args...)` — dispatches through `i18n/LLM` layer

Enforcement: make lint MUST include a check that scans for hardcoded human-readable strings exceeding a length threshold. The anti-bluff sweep command (grep -rn "simulated\|placeholder\|stub\|TODO") MUST also flag obvious hardcoded-text patterns.

Cascade requirement: This rule (verbatim or by CONST-046 ID reference) MUST appear in every owned-by-us submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md.

CONST-045: No Hardcoded Distribution Hosts — All Distribution Via containers/.env

Rule: ALL container distribution targets SHALL be configured exclusively through the CONTAINERS_REMOTE_HOST_N_* environment variables in containers/.env (N=1..100; iteration stops at first absent _NAME). The containers module's pkg/envconfig/parser.go is the authoritative loader.

Prohibition: NO distribution host (hostname, IP address, SSH user, SSH key path, runtime, labels) may be hardcoded in ANY HelixCode-owned source file, test file, challenge, configuration template, script, documentation, governance document (CONSTITUTION.md, CLAUDE.md, AGENTS.md), or any other committed artefact.

The sole source of truth for host enrolment is containers/.env (gitignored, mode 0600). containers/.env.example documents the format but contains NO operative host entries. Adding, removing, or modifying a distribution host MUST be done by editing containers/.env ONLY; no code change is required or permitted.

Audit command: grep -rn 'CONTAINERS_REMOTE_HOST_' containers/.env. The configured set at any point is whatever .env declares. At rule introduction time (2026-05-07), the configured hosts are thinker.local, but the rule applies to whatever set is in .env at any future point.

Testing: Non-unit tests and Challenges that require remote distribution SHALL read containers/.env at runtime. They SHALL skip (with SKIP-OK: #P{X} no remote hosts configured) when CONTAINERS_REMOTE_ENABLED is false or unset, and SHALL use whatever hosts are configured when enabled. No test may hardcode a host name.

Cascade requirement: This rule (verbatim or by CONST-045 ID reference) MUST appear in every owned-by-us submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md.

Article XII — Repository Safety

§12.1 (CONST-042) — No-Secret-Leak

No API key, token, password, certificate, or other credential may be committed to any repository owned by HelixDevelopment or vasic-digital, transitively or otherwise. All secrets live in `.env` files (mode 0600) listed in `.gitignore`. Any leak — to git, logs, build artefacts, screenshots, or external services — is a release blocker until rotated and post-mortemed.

Operational requirements: - Every repo must have `.env`, `.env.local`, `.env.*` (with `!.env.example` exception), `*.pem`, `*.key`, `*.crt`, `id_rsa*` in `.gitignore`. - `scripts/scan-secrets.sh` (or equivalent) must run before every push; failing it blocks the push. - API keys for development are sourced from the canonical `../helix_agent/.env` (mode 0600, never under git) and copied — never symlinked, never committed — into per-repo `.env` files.

Cascade requirement: This article must appear verbatim in every owned-by-us repository's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Owned-by-us repos are listed in `scripts/owned-repos.txt` (or, until that file exists, the meta-repo `propagate-governance.sh` script's submodule walk excluding third-party trees).

§12.2 (CONST-043) — No-Force-Push

No force push, force-with-lease push, history rewrite, branch deletion of `main/` `master`, or upstream-overwriting operation may be performed without explicit, in-conversation user approval given for that specific operation. Authorization for one push does not extend to subsequent pushes. Bypassing hooks (`--no-verify`), signature verification (`--no-gpg-sign`), or protected-branch rules also requires explicit approval. This applies to every repository in the HelixDevelopment / vasic-digital stack.

Operational requirements: - Local pre-push hook at `scripts/git_hooks/pre-push` (installed by `scripts/install-git-hooks.sh`) must reject `--force` / `--force-with-lease` unless `HELIX_FORCE_PUSH_APPROVED=1` is set. - The hook is a courtesy gate; this constitutional clause is the actual contract. - Regular non-force pushes of new commits to existing branches on already-configured remotes are PERMITTED without per-push approval, scoped to a programme/conversation in which the user has authorised the cadence.

Cascade requirement: Same as §12.1 — verbatim, every owned-by-us repo's three governance files.

Article XIII — Programme Continuity

§13.1 (CONST-044) — Continuation Document Maintenance Mandate

The docs/CONTINUATION.md document MUST be maintained in sync with the actual state of the CLI-Agent Fusion programme at all times. It is the authoritative resumption record for any CLI agent or LLM picking up the work from any session, at any time.

Mandate: Every commit that advances programme state — feature task completion, feature close-out, push to remotes, known-issue discovery, deferred-item resolution, phase transition, addition or removal of submodules / remotes — MUST update docs/CONTINUATION.md to reflect the new state in the same commit (or in an immediately-following commit if the state-changing commit is small and topical).

Definition of “out-of-sync”: - Last updated SHA in CONTINUATION $\neq$ git rev-parse HEAD on main. - Active feature in flight in CONTINUATION $\neq$ feature in docs/improvements/PROGRESS.md “Current focus”. - Tasks marked done in CONTINUATION $\neq$ ticked tasks in PROGRESS.md. - Known-issue list missing a documented failure that exists in evidence files (docs/improvements/0[5-9]_phase_*_evidence.md). - Repository state table missing or stale-SHA for any submodule listed in .gitmodules.

Severity: Out-of-sync CONTINUATION.md is a **CRITICAL DEFECT**. Same severity as a false-success test result under CONST-035 / Article XI §11.9. A green build with an out-of-sync continuation document is unshippable.

Verification: scripts/verify_continuation_sync.sh (TBD; planned for Phase 3) compares CONTINUATION fields against PROGRESS.md, git rev-parse HEAD, evidence files, and .gitmodules. Non-zero exit = sync violation → blocking pre-push.

Cascade requirement: This article must appear verbatim — or be referenced by CONST-044 ID with a pointer back to this anchor — in every owned-by-us repository’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Submodules without their own docs/CONTINUATION.md reference the meta-repo’s CONTINUATION as the authoritative document; submodules that maintain their own CONTINUATION must pin its SHA and update on every state-changing commit identically.

CONST-047: Recursive Submodule Application Mandate

Verbatim user mandate (2026-05-14): *“Make sure all work we do is applied ALWAYS to all Submodules we control under our organizations (vasic-digital and HelixDevelopment) fully recursively everywhere with full bluff-proofing and comprehensive documentation, user manuals and guides and full tests and Challenges coverage!”*

Operative rule: Every engineering deliverable produced for HelixCode MUST be applied — fully and recursively — to every owned submodule under the vasic-digital and HelixDevelopment GitHub organizations. “Owned” means any submodule whose remote URL is `git@github.com:vasic-digital/*.git` or `git@github.com:HelixDevelopment/*.git` (including the `HelixDevelopment-s-Code/*` brand variant). Each such submodule MUST receive in lockstep:

1. **Anti-bluff posture (CONST-035 / Article XI §11.9):** every test and Challenge in the submodule must guarantee real, end-user-usable functionality. Metadata-only / absence-of-error / fake-PASS results are critical defects in submodules just as in the meta-repo.
2. **Comprehensive documentation:** README plus user manuals plus guides that match the submodule’s actual current capabilities. Documentation that advertises features the code doesn’t deliver is the same class of defect as a fake-PASS test.
3. **Full tests AND Challenges coverage:** unit tests where applicable, Challenges that exercise real workflows with captured runtime evidence (per CONST-035). 100% of user-reachable surfaces must be covered.
4. **Recursive propagation:** when an owned submodule itself depends on sub-submodules under vasic-digital or HelixDevelopment, the mandate cascades. A submodule that owns child submodules under our organizations is responsible for cascading the mandate to them.
5. **Synchronized commits:** when a meta-repo commit advances state that touches a submodule’s surface, the corresponding submodule commit MUST land in the same engineering session and be pushed to all configured remotes.

Owned submodule baseline (direct children of the meta-repo as of 2026-05-14): vasic-digital/Containers, vasic-digital/Security, vasic-digital/Challenges, vasic-digital/LLMsVerifier, vasic-digital/Models, HelixDevelopment/HelixQA, HelixDevelopment/DocProcessor, HelixDevelopment/LLMOrchestrator, HelixDevelopment/LLMProvider, HelixDevelopment/VisionEngine, HelixDevelopment/HelixAgent, HelixDevelopment-s-Code/Website. Plus any nested submodule under these orgs (full recursion).

Cascade requirement: This anchor (verbatim or by CONST-047 ID reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Non-compliance is a release blocker — same severity as a CONST-035 bluff or a CONST-044 out-of-sync CONTINUATION.

Verification: `scripts/verify-governance-cascade.sh` MUST be extended to confirm CONST-047 anchor presence in every owned submodule (in addition to its existing CONST-035 checks). A submodule missing the anchor is treated identically to one missing the anti-bluff anchor: blocked from merge.

CONST-048: Full-Automation-Coverage Mandate (cascaded from constitution submodule §11.4.25)

Verbatim user mandate (2026-05-15): “Make sure that every feature, every functionality, every flow, every use case, every edge case, every service or application, on every platform we support is covered with full automation tests which will confirm anti-bluff policy and provide the proof of fully working capabilities, working implementation as expected, no issues, no bugs, fully documented, tests covered! Nothing less than this does not give us a chance to deliver stable product! This is mandatory constraint which MUST BE respected without ignoring, skipping, slacking or forgetting it!”

No feature / functionality / flow / use case / edge case / service / application on any supported platform of HelixCode may be considered deliverable until covered by automation tests proving six invariants: (1) anti-bluff posture (CONST-035) with captured runtime evidence; (2) proof of working capability end-to-end on target topology (no mocks beyond unit tests — see CONST-050); (3) implementation matches documented promise; (4) no open issues/bugs surfaced — cross-checked against §11.4.15 / §11.4.16 trackers; (5) full documentation in sync per §11.4.12; (6) four-layer test floor per §1 (pre-build + post-build + runtime + paired mutation).

Consuming projects MUST publish a coverage ledger (feature × platform × invariant-1..6 × status) regenerated as part of the release-gate sweep. Gaps tracked per §11.4.15 (UNCONFIRMED: / PENDING_FORENSICS: / OPERATOR-BLOCKED: with §11.4.21 audit) — rows that quietly omit a platform are CONST-048 violations.

Cascade requirement: This anchor (verbatim or by CONST-048 ID reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the release-gate layer. No escape hatch. See constitution submodule Constitution.md §11.4.25 for the full mandate.

CONST-049: Constitution-Submodule Update Workflow Mandate (cascaded from constitution submodule §11.4.26)

Verbatim user mandate (2026-05-15): “Every time we add something into our root (constitution Submodule) Constitution, CLAUDE.MD and AGENTS.MD we MUST FIRST fetch and pull all new changes / work from constitution Submodule first! All changes we apply MUST BE committed and pushed to all constitution Submodule upstreams! In case of conflict, IT MUST BE carefully resolved! Nothing can be broken, made faulty, corrupted or unusable! After merging full validation and verification MUST BE done!”

Before ANY modification to constitution/Constitution.md, constitution/CLAUDE.md, or constitution/AGENTS.md, the agent or operator MUST execute the following 7-step pipeline in order:

1. **Fetch + pull first** inside the constitution submodule worktree — every configured remote fetched, then `git pull --ff-only` (or `--rebase` if non-

- FF; NEVER `--strategy=ours` / `--allow-unrelated-histories` without explicit authorization).
2. **Apply the change** with §11.4.17 classification + verbatim mandate quote.
 3. **Validate before commit** — `meta_test_inheritance.sh` (or equivalent), no merge-conflict markers, cross-file consistency.
 4. **Commit + push to ALL upstreams** — governance files only (NEVER `git add -A`); push to every configured remote. One-upstream commit = CONST-049 violation (also CONST-038/§6.W and §2.1).
 5. **Conflict resolution** preserving union of governance content. Force-push to bypass conflicts is FORBIDDEN (CONST-043 / §9.2).
 6. **Post-merge validation** — `git submodule update --remote --init + re-run cascade verifier` (CONST-047) confirming the new clause reaches every owned submodule.
 7. **Bump consuming project pointer** — `.gitmodules`-tracked submodule pointer advanced to the new constitution HEAD in the SAME commit as cascade work.

Cascade requirement: This anchor (verbatim or by CONST-049 ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a force-push without CONST-043 / §9.2 authorization. No escape hatch. See constitution submodule `Constitution.md` §11.4.26 for the full mandate.

CONST-050: No-Fakes-Beyond-Unit-Tests + 100%-Test-Type-Coverage Mandate (cascaded from constitution submodule §11.4.27)

Verbatim user mandate (2026-05-15): *“Mocks, stubs, placeholders, TODOs or FIXMEs are allowed to exist ONLY in Unit tests! All other test types MUST interact with real fully implemented System! No fakes, empty implementations or bluffing is allowed of any kind! All codebase of the project MUST BE 100% covered with every supported test type: unit tests, integration tests, e2e tests, full automation tests, security tests, ddos tests, scaling tests, chaos tests, stress tests, performance tests, benchmarking tests, ui tests, ux tests, Challenges (fully incorporating our Challenges Submodule — <https://github.com/vasic-digital/Challenges>). EVERYTHING MUST BE tested using helix_qa (fully incorporating helix_qa Submodule — <https://github.com/HelixDevelopment/HelixQA>). helix_qa MUST BE used with all possible written tests suites (test banks) for every applications, service, platform, etc and execution of the full helix_qa QA autonomous sessions! All required dependency Submodules MUST BE added into the project as well (fully recursive!!!).”*

Two cooperating invariants:

(A) No-fakes-beyond-unit-tests. Mocks, stubs, fakes, placeholders, TODO, FIXME, “for now”, “in production this would”, or empty-implementation patterns are PERMITTED only in unit-test sources (`*_test.go` files invoked without the integration build tag; `helix_code/tests/unit/`; etc.). Every other test type — integration, E2E, full automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges, `helix_qa` — MUST exercise the real, fully implemented HelixCode system against real infrastructure (real PostgreSQL, real Redis, real LLM endpoints, real containers, real captured devices).

Production code (anything under `helix_code/cmd/`, `helix_code/applications/`, `helix_code/internal/<pkg>/<file>.go` not ending `_test.go`) MUST NOT import from `helix_code/internal/mocks/`.

(B) 100% test-type coverage. HelixCode's codebase MUST be covered by every supported test type the domain warrants: - **Unit** — fast, isolated, mocks permitted per (A). - **Integration** — multi-component, no mocks, real backing services. - **End-to-end (E2E)** — full user-flow exercise on target topology. - **Full automation** — orchestrated suites exercising every feature × platform combination (CONST-048 coverage ledger). - **Security** — authn/authz boundaries, CONST-042 secret-leak scans, input-fuzzing, dependency-CVE scanning, threat-model verification. - **DDoS** — request-flood resilience at advertised throughput tier. - **Scaling** — horizontal + vertical scale behaviour under linear load growth. - **Chaos** — controlled failure injection (network partition, process kill, disk full, clock skew). - **Stress** — sustained load above advertised tier. - **Performance** — latency / throughput / tail-latency invariants vs SLO baselines. - **Benchmarking** — micro + macro suites with historical p95-drift detection. - **UI** — visual-regression + DOM-state + interaction-flow coverage on every target platform's UI surface. - **UX** — flow-correctness + accessibility + il8n + visual-cue ordering (§11.4.23 composition). - **Challenges** — vasic-digital/Challenges submodule (at `./challenges/`) fully incorporated; per-feature Challenge scripts with captured runtime evidence. - **HelixQA** — HelixDevelopment/HelixQA submodule (at `./helix_qa/`) fully incorporated; ALL written test banks executed; full autonomous QA sessions run as part of release gates with captured wire evidence per check.

Required dependency submodules (recursive per CONST-047): - Challenges — `git@github.com:vasic-digital/Challenges.git` — incorporated at `./challenges/`. - `helix_qa` — `git@github.com:HelixDevelopment/HelixQA.git` — incorporated at `./helix_qa/`. - Any additional functionality submodules under `vasic-digital/*` / `HelixDevelopment/*` orgs that HelixCode depends on — incorporate rather than duplicate work the orgs already maintain.

Submodule pointers MUST be bumped to upstream HEAD in the SAME commit as any dependent cascade work (CONST-049 step 7). Pointer drift = CONST-050 violation.

Cascade requirement: This anchor (verbatim or by CONST-050 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the release-gate layer. No escape hatch. See constitution submodule Constitution.md §11.4.27 for the full mandate.

CONST-051: Submodules-As-Equal-Codebase + Decoupling + Dependency-Layout Mandate (cascaded from constitution submodule §11.4.28)

Verbatim user mandate (2026-05-15): *“All existing Submodules in the project that we are controlling and belong to some our organizations (vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory - we can ALWAYS check dynamically using GitHub and GitLab CLIs) are equal parts of the project's codebase! We MUST work on that code as much as we do with main*

project's codebase! All on equal basis! Equally important! We MUST take it into the account, analyze it, extend it, create missing tests, do full testing of it, fill the gaps (if any), fix any issues that we discover or they pop-up, write and extend the documentation, user guides, manulas, diagrams, graphs, SQL definitions, Website(s) and all other relevant materials! We MUST NEVER modify Submodules to bring into them any project specific context since they all MUST BE ALWAYS fully decoupled, project not-aware, fully reusable and modular (by any other project(s)), completely testable! All Submodule dependencies that are used by Submodule MUST BE accessed from the root of the project! We MUST NOT have nested Submodule dependencies but accessing each from proper location from the root of the project - directly from project's root project_name/submodule_name or some more proper structure project_name/submodules/submodule_name!"

Three cooperating invariants apply to every HelixCode-owned submodule (those whose upstream origin lives under vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory, or any subsequently authorised org):

(A) Equal-codebase. Every owned-by-us submodule is an **equal part** of HelixCode's codebase. The same engineering practice — analysis, extension, test creation, gap-filling, bug-fix, documentation (user manuals, guides, diagrams, graphs, SQL definitions, website pages, all materials) — applies to each owned submodule on equal basis. A round of work that improves only HelixCode's main while leaving an owned-submodule deficiency unaddressed is a CONST-051 violation, severity-equivalent to a §11.4 PASS-bluff at the project-scope layer. The §11.4.25 / CONST-048 coverage ledger MUST list every owned submodule as an in-scope target.

(B) Decoupling / reusability. Owned submodules MUST remain fully decoupled from HelixCode (and any other consuming project). No HelixCode-specific context, hardcoded paths, hostnames, asset names, or runtime assumptions may be introduced into an owned submodule's source tree. When a submodule needs information from HelixCode, the honest path is configuration injection (env var, config file, constructor parameter) — never a hardcoded reach into the parent's tree. Every owned submodule MUST be project-not-aware, fully reusable, modular, and completely testable as a standalone repository.

(C) Dependency-layout. Every dependency that an owned submodule consumes MUST be accessible from HelixCode's root at one of two canonical paths: - <repo_root>/<submodule_name>/ (flat layout — current HelixCode layout for Challenges, HelixQA, Containers, Security, etc.) - <repo_root>/submodules/<submodule_name>/ (grouped layout — alternate)

Nested own-org submodule chains are FORBIDDEN. A submodule MUST NOT have its own .gitmodules entries pulling in further owned-by-us repos. Every dependency required by submodule X is added to HelixCode's root at the canonical path; X reaches it via documented import / SDK path / runtime resolver — never via its own nested submodule pointer. Third-party submodules (not under our orgs) are exempt — they MAY appear at any depth.

The owned-org list is dynamically discoverable at any time via `gh org list / glab CLIs` or the orgs' public APIs.

Cascade requirement: This anchor (verbatim or by CONST-051 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the codebase-completeness layer. No escape hatch. See constitution submodule Constitution.md §11.4.28 for the full mandate (audit gates, mutation pairs, workflow integration).

Amendment Process

Constitution amendments require: 1. Written proposal with rationale 2. Challenge demonstrating the need 3. 72-hour review period 4. Approval by project architect 5. Update to all submodule governance files

This Constitution is the supreme law of the HelixCode project. No code, test, or process may contradict it.

CONST-052: Lowercase-Snake_Case-Naming Mandate (cascaded from constitution submodule §11.4.29)

Verbatim user mandate (2026-05-15): **“naming convention for Submodules and directories (applied deep into hierarchy recursively) - all directories and Submodules MUST HAVE lowercase names with space separator between the words of ‘_’ character (snake-case)! All existing Submodules and directories which are not following this rule MUST BE renamed! However, since this will most likely break some of the functionalities renaming we do MUST BE applied to all references to particular Submodule or directory! ... There MUST BE reasonable exceptions for this rules - source code for programming languages or Submodules which apply different naming convention - Android, Java, Kotlin and others. ... Upstreams directory which all of our projects and Submodules have MUST BE renamed to the lowercase letters too, however root project containing the install_upstreams system command (it is exported in our paths in our .bashrc or .zshrc) MUST BE updated to fully work with both Upstreams and upstreams directory. ... NOTE: Rules lowercase / snake-case do apply to all project files as well and references to it and from them!”**

Every directory, submodule, and file in HelixCode MUST use lowercase snake_case names. Existing non-compliant names (helix_code/, challenges/, containers/, helix_agent/, helix_qa/, security/, github_pages_website/, upstreams/, dependencies/, etc.) MUST be renamed as part of the phased migration opened by this clause. Every reference (configs, docs, links, source-code imports, governance files) MUST be updated atomically with the rename — reference drift after a rename is a CONST-052 violation of equal severity to the rename itself.

Common-sense exceptions (technology-preserving): language-mandated case for Java/Kotlin/Android/Apple/C#/Swift INSIDE the language root (submodule root follows our convention; subtree follows language convention); vendor/upstream

third-party submodules keep upstream names; build artefacts (node_modules, __pycache__, .git, target, build, bin) keep tool-mandated names. The test “does renaming break the technology?” trumps the rule.

upstreams/ → upstreams/ transition: the constitution submodule’s install_upstreams.sh (exported via .bashrc/.zshrc) supports BOTH upstreams/ and upstreams/ directory layouts (commit 45d3678 of the constitution submodule); lowercase wins when both present.

Test coverage of renames (per CONST-050(B)): every rename batch ships with (i) regression test verifying every reference now resolves, (ii) full test-type matrix run post-rename, (iii) anti-bluff wire-evidence captured.

Phased execution per the operator’s explicit instruction: comprehensive brainstorming → phase-divided plan → fine-grained tasks/subtasks → every change covered by every applicable test type. §11.4.20 subagent delegation for cross-cutting rename sweeps.

Cascade requirement: This anchor (verbatim or by CONST-052 ID reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the reference-integrity layer. No escape hatch beyond the common-sense exceptions enumerated above. See constitution submodule Constitution.md §11.4.29 for the full mandate.

CONST-053: .gitignore + No-Versioned-Build-Artifacts Mandate (cascaded from constitution submodule §11.4.30)

Verbatim user mandate (2026-05-15): “every project module, every Submodule, every service and application MUST HAVE proper .gitignore file! We MUST NOT git version build artifacts, cache files, tmp files, main .env file(s) or any files containing sensitive data, API keys or token! Any build derivative which we can recreate by executing proper mechanism for generating MUST NOT be versioned! We MUST pay attention what is going to be committed every time we are preparing to execute commit! If any violation is detected it MUST be fixed before commit is executed!”

Every project module, owned-by-us submodule, service, and application MUST ship a proper .gitignore. Forbidden-from-version-control classes:

1. **Build artefacts:** /bin/, /build/, /dist/, /out/, target/, *.exe, *.dll, *.so, *.dylib, *.a, *.o, *.class, *.pyc, generator-produced files when the generator is committed.
2. **Cache files:** __pycache__/, .pytest_cache/, .mypy_cache/, .ruff_cache/, node_modules/, .next/, .cache/, .gradle/, .terraform/, language-server caches.
3. **Temp files:** *.tmp, *.swp, *~, .DS_Store, Thumbs.db, *.orig, *.rej.
4. **Sensitive-data files:** .env, .env.* (allow .env.example placeholder only — no real secrets even as examples), *.pem, *.key, *.crt, id_rsa*, id_ed25519*, .netrc, secrets/, api_keys.sh.

5. **Generated reports/logs:** *.log, coverage.out, htmlcov/, runtime captures unless reference assets.
6. **OS/IDE personal state:** .idea/, .history/, .vscode/ (except shared settings).

Anti-bluff invariant: .gitignore line alone is not sufficient — no file matching the forbidden patterns may be CURRENTLY TRACKED. A tracked *.log despite the ignore-line is a violation of equal severity to no ignore-line at all.

Pre-commit attention: every commit author (human OR agent) MUST inspect `git diff --staged + git status` BEFORE executing the commit. Forbidden-class hits abort the commit until fixed (un-stage, add to .gitignore, scrub if already-tracked). Gate CM-GITIGNORE-PRECOMMIT-AUDIT + paired mutation.

Secret-leak intersection (CONST-042 / §11.4.10): a .env leak is BOTH a CONST-053 and a CONST-042 violation; rotation + post-mortem required.

Recreatable-content test: if a documented mechanism regenerates the file from sources, it is a build derivative and MUST be ignored. The committed sources MUST include the generator.

Cascade requirement: This anchor (verbatim or by CONST-053 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the repository-hygiene layer. See constitution submodule Constitution.md §11.4.30 for the full mandate.

CONST-054: Submodule-Dependency-Manifest Mandate (cascaded from constitution submodule §11.4.31)

Verbatim user mandate (2026-05-15): “We MUST HAVE mechanism for each Submodule to determine / know what are its Submodule dependencies so new projects or palces we are incorporate them can add these Submodules to the project root and make them available! Suggested idea is configuration file with expected Submodules Git ssh urls perhaps? New project can read it, and recursively add each Submodule to the root of the project and install / expose it to everyone.”

Every owned-by-us submodule MUST ship `helix-deps.yaml` at its root declaring its own-org dependencies. Schema: `schema_version`, `deps: [{name, ssh_url, ref, why, layout: flat|grouped}], transitive_handling. {recursive, conflict_resolution}, language_specific_subtree`. Tooling: `incorporate-submodule <ssh-url>` adds the submodule at the parent project's canonical path (CONST-051(C)), reads `helix-deps.yaml`, recurses for each declared dep, aborts on conflicting refs, emits `<root>/ .helix-manifest.yaml` audit record.

Anti-bluff guarantee: every manifest paired with a Challenge that bootstraps a throwaway consuming project, runs `incorporate-submodule`, asserts produced layout matches the manifest, runs the submodule's own tests against the bootstrapped layout, captures wire evidence per §11.4.2. A manifest without this proof is a CONST-054 violation.

§11.4.31 / CONST-054 is the **operational complement** of CONST-051(C): nested own-org submodule chains are FORBIDDEN — manifests are the bridge that lets consumers reconstruct the dependency graph at the parent root.

Cascade requirement: This anchor (verbatim or by CONST-054 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to §11.4 PASS-bluff at the dependency-graph layer. See constitution submodule Constitution.md §11.4.31 for the full mandate.

CONST-055: Post-Constitution-Pull Validation Mandate (cascaded from constitution submodule §11.4.32)

Verbatim user mandate (2026-05-15): *“Every time we fetch and pull new changes on constitution Submodule we MUST process the whole project and all Submodule (deep recursively) for validation and verification taht every single rule or mandatory constraint is followed and respected! If it is not, IT MUST BE!”*

Whenever a project's constitution submodule is fetched + pulled with any content change, the project MUST run `scripts/verify-all-constitution-rules.sh` BEFORE the new constitution HEAD is treated as canonical for any other work. The sweep re-runs the governance-cascade verifier AND every implementable rule gate (CONST-053 `.gitignore` audit, CONST-051(C) nested-own-org-chain audit, CONST-052 case audit, CONST-050(A) mock-from-production audit, CONST-035 anti-bluff smoke, etc.) against the post-pull tree. Failures populate the project's Issues tracker per §11.4.15 (Status: Reopened, Type: Bug); closure requires positive-evidence per §11.4.

Pull-time invocation: `git submodule update --remote constitution` triggers the sweep automatically (post-update hook OR commit-wrapper invocation). Operator-explicit manual invocation also available.

Anti-bluff: the sweep's own meta-test (paired mutation per §1.1) plants a known violation of each enforced gate and asserts the sweep reports FAIL for the planted gate. A sweep that exits PASS without running every implementable gate is a CONST-055 violation.

CONST-055 is the **enforcement engine** for every other §11.4.x and CONST-NNN rule — without it, new rules cascade as anchors but never get enforced.

Cascade requirement: This anchor (verbatim or by CONST-055 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to §11.4 PASS-bluff at the constitutional-enforcement layer. See constitution submodule Constitution.md §11.4.32 for the full mandate.

CONST-056: Mandatory install_upstreams on clone/add Mandate (cascaded from constitution submodule §11.4.36)

Verbatim user mandate (2026-05-15): “Every Submodule or Git repository we add or clone MUST BE upstreams installed using Upstreamable utility which MUST BE available through exported paths of the host system (in .bashrc or .zshrc) using install_upstreams command executed from the root of the cloned (added) repository - only if in it is Upstreams or upstreams directory present with bash script files (recipes) for all repository’s upstreams!”

Every clone / add of a Git repository under HelixCode MUST be followed by install_upstreams invocation from the repository’s root IF its tree contains upstreams/ (or legacy upstreams/ per CONST-052 transition) populated with *.sh recipe files. The utility (installed on operator’s PATH via .bashrc/.zshrc; implementation in the constitution submodule’s install_upstreams.sh — already supports BOTH directory names since constitution commit 45d3678) reads the recipe files, configures every declared upstream as a named git remote, and fans out origin push URLs.

Skipping the invocation when upstreams/ is present silently breaks §2.1 (multi-upstream push is the norm) — the next push lands on only one upstream. Gate CM-INSTALL-UPSTREAMS-ON-CLONE + paired mutation. Automation: the future incorporate-submodule per CONST-054 auto-invokes; manual invocation supported. Pre-commit check: git remote -v | grep -c push reports expected count.

Cascade requirement: This anchor (verbatim or by CONST-056 ID reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.36 for the full mandate.

CONST-057: Type-aware Closure-Status Vocabulary (cascaded from constitution submodule §11.4.33)

Every project tracking work items by Type per §11.4.16 MUST close them with the Type-appropriate terminal **Status:** value, drawn from this 3-element closed map:

Item **Type:**	Closure **Status:** value
Bug	Fixed (→ Fixed.md)
Feature	Implemented (→ Fixed.md)
Task	Completed (→ Fixed.md)

The (→ Fixed.md) suffix is preserved across all three so the existing migration-discipline tooling (atomic Issues.md → Fixed.md move per §11.4.19) keeps working without per-Type branching. Generators (generate_issues_summary.sh,

generate_fixed_summary.sh, the §11.4.23 colorizer) MUST treat the three terminal values as semantically equivalent (all “closed, positive evidence captured”) while preserving the literal in the emitted document.

Closing a Feature with Fixed (→ Fixed.md) or a Task with Implemented (→ Fixed.md) is a CONST-057 violation. Gate CM-CLOSURE-VOCAB-TYPE-AWARE walks every Fixed.md heading + every Issues.md heading whose **Status:** is one of the three terminal values and asserts the Status-Type match. Composes with §11.4.15 / §11.4.16 / §11.4.19 / §11.4.23.

Cascade requirement: This anchor (verbatim or by CONST-057 ID reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.33 for the full mandate.

CONST-058: Reopened-Source Attribution Mandate (cascaded from constitution submodule §11.4.34)

Every Issues.md (or equivalent project tracker) heading whose **Status:** is Reopened MUST carry, within 8 non-blank lines of the heading, a **Reopened-Details:** line capturing four sub-facts:

- **By:** AI or User (source-of-truth observer who flipped the status). AI covers in-loop reopens (test failure, gate regression, captured-evidence retrospect). User covers operator-side observations (manual testing, end-user report, design reconsideration).
- **On:** ISO date (YYYY-MM-DD).
- **Reason:** one-line cause classification — chosen from the closed vocabulary { test-failed | manual-testing-detected | captured-evidence-contradicts | end-user-report | cycle-re-discovered | design-reconsidered }. Other values permitted with explicit Reason: <free text> annotation but the closed list MUST be tried first.
- **Evidence:** path to or short description of the captured artefact justifying the reopen — log file, recording, gate failure ID, operator quote, etc. Reopens without evidence are §11.4.6 / §11.4.7 violations (demotion from Fixed requires captured evidence under the conditions that re-exposed the defect).

The Issues_Summary.md Status column MUST distinguish the four Reopened sub-states by source so a sweep query for “reopens by AI in the last 30 days” is mechanically possible. Suggested column rendering: Reopened (AI: test-failed) vs Reopened (User: manual-testing). Gate CM-ITEM-REOPENED-DETAILS mirrors CM-ITEM-OPERATOR-BLOCKED-DETAILS (§11.4.21 walk pattern). Composes with §11.4.6 / §11.4.7 / §11.4.15 / §11.4.21.

Cascade requirement: This anchor (verbatim or by CONST-058 ID reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.34 for the full mandate.

CONST-059: Canonical-Root Inheritance Clarity (cascaded from constitution submodule §11.4.35)

The **constitution submodule's** three files (constitution/Constitution.md, constitution/CLAUDE.md, constitution/AGENTS.md) ARE the **canonical root** (also called the **parent** files). They contain only universal rules per §11.4.17.

The consuming project's **repository-root files** (<project-root>/CLAUDE.md, <project-root>/AGENTS.md, optionally <project-root>/Constitution.md) are **consumer extensions**. They MUST start with the inheritance pointer (either the Claude-Code native @constitution/CLAUDE.md import or the portable ## INHERITED FROM constitution/CLAUDE.md heading). They contain only project-specific rules per §11.4.17.

When in doubt about which file to edit: universal rule → constitution submodule's file; project-specific rule → consumer's file. Default consumer-side when uncertain (§11.4.17 — narrower scope is cheap to widen).

Terminology: “the parent CLAUDE.md” / “the root Constitution” → constitution-submodule file at constitution/<filename>; “the project CLAUDE.md” / “this project's AGENTS.md” → consumer-side file at <project-root>/<filename>.

No silent demotion or silent promotion. Moving a rule between layers MUST be a visible commit — git mv of a section if it's a clean clone, or explicit Lifted from <project> to constitution per §11.4.35 / Demoted from constitution to <project> per §11.4.35 commit-message annotation.

Gate CM-CANONICAL-ROOT-CLARITY verifies (a) consumer's CLAUDE.md opens with the inheritance pointer, (b) constitution submodule's three files are present at the expected path, (c) no ## INHERITED FROM block in the constitution submodule's own files (those ARE the source-of-truth, not consumers). Composes with §11.4.17.

Cascade requirement: This anchor (verbatim or by CONST-059 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.35 for the full mandate.

CONST-060: Fetch-before-edit Mandate (cascaded from constitution submodule §11.4.37)

Verbatim user mandate (2026-05-15): “Make sure that *feedback_fetch_before_edit* memory rule is part of our constitution Submodule - the root Consitution, AGENTS.MD and CLAUDE.MD. Validate and verify that Proejct-Toolkit and all Submodules do inherit all of them! Follow the constitution Submodule documentation for details.”

The FIRST git-touching action of every session, on every consuming project (owned or third-party), MUST be:

```
git fetch --all --prune
git log --oneline HEAD..@{u}
git submodule foreach --recursive 'git fetch --all --prune --
quiet'
```

If HEAD..`@{u}` is non-empty, integrate the upstream changes BEFORE any local edit. Acting on stale local state produces three failure modes documented in the originating §11.4.37 incident (multi-agent / parallel-session work): (1) **redundant work** — the agent re-does what a parallel session already finished, (2) **false confidence** — completion reports for already-done work, (3) **divergent history** — duplicate sibling commits that double the conflict surface on next push.

Anti-bluff invariant: the fetch+log check MUST produce captured evidence — the actual HEAD..`@{u}` output, even if empty. Skipping the check on the basis of “I just fetched” or “nothing could have changed in the last N minutes” is a §11.4.6 (no-guessing) violation: the remote state is not knowable without a fetch.

Cascade requirement: This anchor (verbatim or by CONST-060 ID reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to §11.4 PASS-bluff at the parallel-session-coordination layer. See constitution submodule Constitution.md §11.4.37 for the full mandate.

§11.4.68 — Positive Sink-Side / Downstream Evidence Mandate (cascaded from constitution submodule §11.4.68)

Verbatim user mandate (2026-05-20): “*We still do not hear any audio played from D3 device! Arvus Web Dashboard when we play music from D3 shows nothing for Codec In Use! This MUST BE investigated and fixed! How come we passed the tests with Arvus validation? What were values for the Codec In Use field? Empty means nothing! This is not working! It MUST BE FIXED, TESTED AND VERIFIED WITH FULL AUTOMATION TESTING ASAP!!!*”

A test that asserts audio or video routing PASS MUST capture and verify **positive sink-side or downstream evidence** — never config-only, never metadata-only, never PCM-open-state-only. At least one of the closed enumeration MUST be captured for every audio/video routing PASS: (1) sink-side codec-state with non-empty Codec-In-Use matching the expected codec regex; (2) strictly-positive PCM frames-written delta from /proc/asound/.../status hw_ptr; (3) ALSA ELD/EDID-Like-Data showing negotiated channel count + format; (4) ffprobe-on-captured-mp4 with non-zero frame count + expected codec/resolution/fps; (5) recording-analyzer event match per §11.4.2/§11.4.5; (6) tinycap RMS amplitude above the line-level floor. Empty / <unreachable> / <N.E.> / <None> placeholders are NOT positive evidence; a missing-but-required sink is OPERATOR-BLOCKED (release-blocker), never SKIP, never PASS. No escape hatch — no --skip-sink-evidence, --allow-empty-codec, --sink-unreachable-is-pass, --metadata-only-suffices flag exists.

Cascade requirement: This anchor (verbatim or by §11.4.68 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the sink-side-evidence layer. **Canonical authority:** constitution submodule Constitution.md §11.4.68 for the full mandate.

§11.4.70 — Subagent-Driven Execution Is The Default (cascaded from constitution submodule §11.4.70)

Verbatim user mandate (2026-05-20): *“Always do if possible Subagent-driven! Add this into our root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md. This should be the default choice ALWAYS!”*

When executing implementation plans (or any task-decomposed execution flow), the **default execution model is subagent-driven** per superpowers:subagent-driven-development. Inline execution is permitted ONLY when (a) the task is trivial AND fits a single sub-300-line edit, OR (b) the operator explicitly requests inline at brainstorm-handoff time. Subagent-driven is the default because it gives isolated context per task, naturally enforces two-stage review, is parallel-PWU compatible (§11.4.58), creates an anti-bluff seam (§11.4), and survives operator absence. No escape hatch — `--inline-execution-required`, `--no-subagents`, `--monolithic-execution` are NOT permitted flags. Skipping subagent-driven for non-trivial work without recorded operator authorisation is itself a §11.4 PASS-bluff.

Cascade requirement: This anchor (verbatim or by §11.4.70 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the execution-model layer. **Canonical authority:** constitution submodule Constitution.md §11.4.70 for the full mandate.

§11.4.71 — Pre-Push Fetch + Investigate + Integrate Mandate (cascaded from constitution submodule §11.4.71)

Verbatim user mandate (2026-05-20): *“before pushing changes to any upstream for any repository - main repo or Submodule, we MUST fetch and pull all changes. Once these are obtained WE MUST investigate what is different compared to head position we were on last time before fetching and pulling new changes! We MUST understand what is done and for what purpose, easpecially how that does affect our project and our System in general! Any mandatory changes or improvements required by fresh changes we just have brough in MUST BE incorporated, covered with all supported types of the tests which will produce as a result of its success execution REAL PROOFS of working for all componetns and functionalities covered and work fully in anti-bluff manner!”*

The everyday-push variant of §11.4.41. EVERY push (every repository — main + every submodule) MUST follow the 5-step cycle: (1) fetch all remotes (`git fetch --all --prune --tags`, capture stdout); (2) pull all upstream branches whose tip differs, resolving conflicts per consumer judgment (never auto-`---ours/--theirs`); (3) investigate the diff vs OUR previous HEAD — read EVERY foreign commit's body, understand what/why/how-it-affects-our-system; (4) integrate mandatory changes with §11.4.4(b) four-layer coverage + §11.4.43 TDD-fix discipline, every PASS carrying §11.4.5 captured-evidence (REAL PROOFS, not metadata-only); (5) only then push, verifying with `git ls-remote post-push`. No escape hatch — no `--skip-fetch`, `--no-investigate`, `--fast-push`, `--trust-upstream` flag.

Cascade requirement: This anchor (verbatim or by §11.4.71 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the push-discipline layer.
Canonical authority: constitution submodule Constitution.md §11.4.71 for the full mandate.

§11.4.72 — Audio Top-Priority Mandate (cascaded from constitution submodule §11.4.72)

Verbatim user mandate (2026-05-20): *“Make sure all fixes for audio are always top priority in main working stream!”*

The conductor (main working stream — Claude Code session, AI agent, or human operator) MUST treat audio fixes as the highest-priority class on the serial dispatch queue. Any time the conductor faces a choice between dispatching an audio task vs a non-audio task on the SAME serial resource, the audio task wins. Parallel BACKGROUND subagents (research, refactors, infrastructure documentation) MAY run concurrently with audio work but do NOT preempt audio on the main-stream serial dispatch queue. No escape hatch — there is no “but this non-audio task is faster” or “but this research is more interesting” override; audio-stack regressions are user-perceptible and high-impact while research and refactors can wait.

Cascade requirement: This anchor (verbatim or by §11.4.72 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a process violation at the dispatch-priority layer.
Canonical authority: constitution submodule Constitution.md §11.4.72 for the full mandate.

§11.4.73 — Main-Specification Document Versioning + Revision Discipline (cascaded from constitution submodule §11.4.73)

Verbatim user mandate (2026-05-20): *“Make sure everything we add now in previous and upcoming requests IS ALWAYS applied to the main specification — if we have one. Since all these are not major changes we could increase Specification version per change for secondary version instead of the primary. Primary version MUST BE increased for much bigger levels of changes! Add this into root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md as mandatory rule / constraint applicable*

ONLY IF we have something like the main specification document or we do recognize something like the main specification document. Document MUST BE updated ALWAYS to follow the versioning rules we are applying here + revision and other properties we have!”

Applies **only when a project recognises a main specification document**. When it does: (1) every additive operator requirement, refinement, or accepted recommendation MUST be applied to the spec before or as part of the implementing work; (2) spec versioning has two axes — *primary* (V1/V2/V3, bumped for major rewrites by explicit operator decision, old versions archived) and *secondary* (the §11.4.61 metadata-table Revision integer, bumped for every other change); (3) the metadata table MUST stay current (Revision, Last modified, Status summary, Fixed); (4) propagated copies of the rule MUST reference the active specification.V<primary>.md, not a stale archive; (5) on primary bump the old file moves to <spec-dir>/archive/ with Status: superseded. Classification: universal, applicable conditionally per the scope condition.

Cascade requirement: This anchor (verbatim or by §11.4.73 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a release blocker when a project has a main spec and lets it drift. **Canonical authority:** constitution submodule Constitution.md §11.4.73 for the full mandate.

§11.4.74 — Submodule-Catalogue-First Discovery + Extend-Don’t-Reimplement (cascaded from constitution submodule §11.4.74)

Verbatim user mandate (2026-05-20): “We MUST ALWAYS check which already developed features / functionalities do exist as a part of our comprehensive Submodules catalogue located in vasic-digital and HelixDevelopment organizations on GitHub and GitLab both! Project MUST BE aware of all its existence so we do not implement same things multiple times if they are already done as some of existing universal, reusable general development purpose Submodules! For any missing features that some Submodules we incorporate may be missing we MUST IMPLEMENT the properly and extend those Submodules further! We do control all of the and we CAN and MUST maintain and extend the regularly! All development cycle rules we have MUST BE applied to them and fully respected!”

Before scaffolding ANY new module, package, helper, or utility, the contributor (human or AI agent) MUST: (1) survey the canonical Submodule catalogue — vasic-digital and HelixDevelopment on both GitHub AND GitLab; (2) inventory existing Submodules; (3) reuse before reimplement — if a Submodule provides the functionality (or 80%+ of it), add it as a Git submodule rather than write fresh; (4) extend in-place when 80%+ matches but features are missing — add the missing features TO THAT SUBMODULE (PR upstream + bump pointer), never as a duplicating consuming-project helper; (5) apply all development-cycle rules to those Submodules; (6) document the survey result in the feature’s tracker entry with a Catalogue-Check: field (reuse <org/repo>@<sha> / extend <org/repo>@<sha> / no-match <date>). Classification: universal.

Cascade requirement: This anchor (verbatim or by §11.4.74 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a process violation; duplicate implementations landed without catalogue check are release blockers. **Canonical authority:** constitution submodule Constitution.md §11.4.74 for the full mandate.

§11.4.69 — Universal Sink-Side Positive-Evidence Taxonomy + Mechanical Enforcement (cascaded from constitution submodule §11.4.69)

Verbatim user mandate (2026-05-20): *“THIS MUST HAPPEN NEVER AGAIN!!! We MUST HAVE this all working! Not just for audio but for every single piece of the System!!! Proper full automation when executed with success MUST MEAN that manual testing will be as much positive at least regarding the success results! ... Solution MUST BE universal, generic that solves working flows for all System components and for all future and all existing projects! ... Everything we do MUST BE validated and verified with rock-solid proofs and anti-bluff policy enforcement and fulfillment!”*

Universal generalisation of §11.4.68 (audio-specific) across every user-visible feature class. Every user-visible feature MUST map to one entry in the closed-set §11.4.69 sink-side evidence taxonomy (audio_output, audio_input, video_display, network_throughput, network_connectivity, bluetooth_a2dp, bluetooth_pair, touch_input, sensor, gpu_render, storage_read, storage_write, mediacodec_decode, mediacodec_encode, miracast, cast, boot_service, package_install, permission_grant, wifi_link, wifi_throughput, ethernet_link, display_topology, drm_playback, subtitle_render — open to additions, never contraction). Every PASS for a feature in the taxonomy MUST cite a captured-evidence artefact path matching the required evidence shape. New helper contracts (additive during grace, mandatory after 2026-06-19): ab_pass_with_evidence <description> <evidence_path> (verifies path exists + non-empty), ab_skip_with_reason <description> <closed-set-reason> (reasons: geo_restricted, operator_attended, hardware_not_present, topology_unsupported, network_unreachable_external, feature_disabled_by_config; forbids network_unreachable_external for any taxonomy feature with a sink-side probe); bare ab_pass deprecated (WARN pre-grace, FAIL post-grace). Three pre-build gates + paired §1.1 mutations: CM-SINK-EVIDENCE-PER-FEATURE, CM-NO-FAIL-OPEN-SKIP, CM-AB-PASS-WITH-EVIDENCE-EVERYWHERE. No escape hatch — no --skip-evidence, --config-only-pass, --allow-fail-open-skip, --legacy-ab-pass-permitted flag.

Cascade requirement: This anchor (verbatim or by §11.4.69 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-69-PROPAGATION enforces the anchor literal across the consumer fleet; paired mutation strips the literal → gate FAILs. Severity-equivalent to a §11.4 PASS-bluff at the sink-side-evidence layer. **Canonical authority:** constitution submodule Constitution.md §11.4.69 for the full mandate.

§11.4.75 — Mechanical Enforcement Without Exception (cascaded from constitution submodule §11.4.75)

Verbatim user mandate (2026-05-20): *“Why do these violations still happen!? This is a serious problem! We cannot rely on stability nor consistency if we cannot respect our Constitution, mandatory rules and constraints! Is there a way to make this always respected, followed and applied without exception fully and unconditionally!? WE MUST HAVE THIS WORKING FLAWLESSLY!!! Do investigate the root causes of such problems! Once all problems are identified WE MUST apply proper mechanisms for this not to happen NEVER EVER AGAIN!”*

The §11.4 covenant historically relied on agent + operator vigilance; three 2026-05-19→20 forensic incidents proved that late-binding enforcement fires hours-to-days after the violator commit reaches every remote. §11.4.75 closes the gap with FIVE independent mechanical enforcement layers — bypassing any single layer does not bypass the discipline: (1) local pre-commit git hook (refuses staged .md lacking sibling .html+.pdf); (2) commit_all.sh integration (_constitution_sibling_check + auto-sync_all_markdown_exports.sh self-repair); (3) local pre-push git hook (re-runs siblings + propagation-gate subset); (4) post-commit auto-repair hook (auto-generates orphan-.md siblings, idempotent + recursion-guarded); (5) local-only final-gate ritual (remote CI DISABLED per User mandate — operator runs pre_build_verification.sh + meta-test before every tag per §11.4.40). Helper contracts: scripts/install_git_hooks.sh, scripts/git_hooks/{pre-commit,pre-push,post-commit,commit-msg}, _constitution_sibling_check. The commit-msg hook enforces a Bypass-rationale: <reason> footer when --no-verify is detected; docs/audit/bypass_events.md accumulates the audit trail. Five gates with paired §1.1 mutations: CM-COVENANT-114-75-PROPAGATION, CM-GIT-HOOKS-INSTALL-SCRIPT, CM-GIT-HOOKS-SOURCE-DIR, CM-COMMIT-ALL-SIBLING-CHECK, CM-CI-WORKFLOW-PRESENT. No escape hatch — no --skip-hooks, --bypass-enforcement, --allow-orphan-md, --ci-not-applicable, --mechanical-enforcement-not-needed flag.

Cascade requirement: This anchor (verbatim or by §11.4.75 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-75-PROPAGATION; paired mutation strips the literal → gate FAILs. Severity-equivalent to a §11.4 PASS-bluff at the enforcement layer. **Canonical authority:** constitution submodule Constitution.md §11.4.75 for the full mandate.

§11.4.76 — Containers-Submodule Mandate (cascaded from constitution submodule §11.4.76)

Verbatim user mandate (2026-05-20): *“For any work or requirements of running services or codebase inside the Containers (Docker / Podman / Qemu / Emulators, and so on) we MUST USE / INCORPORATE the Containers Submodule properly: <https://github.com/vasic-digital/containers> (git@github.com:vasic-digital/containers.git). Containers Submodule*

contains all means for us to Containerize our code and services! If any feature or Containing System is missing or not supported we MUST EXTEND IT properly like we do all of our projects! No bluff work is allowed of any kind!"

For ANY containerized workload (Docker / Podman / Qemu / Kubernetes / container-backed emulators), every consuming project MUST: (1) install vasic-digital/containers (digital.vasic.containers) as a Git submodule; (2) consume via replace directive during development + pinned commit SHAs in production; (3) boot infra on-demand via pkg/boot + pkg/compose + pkg/health so operators are never required to start podman machine / docker compose up manually — the boot is part of the test entry point (the on-demand-infra invariant); (4) extend the Submodule (PR upstream) for missing runtimes / lifecycle primitives — never reimplement in-project (per §11.4.74); (5) anti-bluff: integration tests claiming to exercise containerized components MUST actually boot them via the Submodule — short-circuit fakes that bypass boot are a §11.4 violation. Tracker rows touching containerization MUST record Catalogue-Check: extend vasic-digital/containers@<sha> (or reuse). Planned gate CM-CONTAINERS-USED scans container-touching PRs for digital.vasic.containers/... imports; paired mutation strips the import + asserts FAIL.

Cascade requirement: This anchor (verbatim or by §11.4.76 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-76-PROPAGATION; paired mutation strips the literal → gate FAILs. **Canonical authority:** constitution submodule Constitution.md §11.4.76 for the full mandate.

§11.4.77 — Regeneration-Mechanism-Required Mandate (cascaded from constitution submodule §11.4.77)

Verbatim user mandate (2026-05-20): "We must be sure that after excluding anything from Git versioning we still have the mechanism which will out of the box obtain or re-generate missing content!"

Every .gitignore entry excluding (a) >~100 MiB OR (b) any artefact essential to building / running / testing the project MUST carry a documented + automated mechanism to either re-obtain (download from authoritative source: vendor tarball, SDK installer, npm/pip/cargo/go-mod/container registry, dedicated git submodule, S3/GCS) OR re-generate (run from tracked source via build pipeline, code-gen, asset render, captured-evidence replay, container build). Required artefacts per qualifying entry: (1) .gitignore-meta/<entry-slug>.yaml declaring pattern + mechanism-type + script-path + expected-disk-usage + vendor-url-or-source + integrity hash + requires-network + requires-credentials; (2) a non-interactive entry in scripts/setup.sh post-clone bootstrap; (3) a pre-build gate verifying regenerated content present OR a recent .gitignore-meta/.regenerated/<slug>.ok stamp; (4) README + docs/guides/*.md describing the mechanism + manual fallback + time/disk budget + §11.4.10 credentials. Bare .gitignore additions without the mechanism are a §11.4 PASS-bluff variant — codebase appears complete but a fresh clone cannot build/run. No escape hatch — no --

skip-regen-mechanism, --gitignore-is-enough, --operator-already-has-content flag. Planned gate CM-GITIGNORE-REGEN-MECHANISM + paired §1.1 mutation (strip a required YAML key → gate FAILs).

Cascade requirement: This anchor (verbatim or by §11.4.77 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-77-PROPAGATION; paired mutation strips the literal → gate FAILs. Severity-equivalent to a §11.4 PASS-bluff at the repository-hygiene layer. **Canonical authority:** constitution submodule Constitution.md §11.4.77 for the full mandate.

§11.4.78 — CodeGraph Code-Intelligence Mandate (cascaded from constitution submodule §11.4.78)

Verbatim user mandate (2026-05-20): *“Make codegraph MANDATORY CHOICE for this purpose for all of our project ... All project which do not have configured and installed codegraph yet MUST DO IT and MUST USE IT!”*

Every consuming project worked on by AI coding agents MUST install, initialize, and use **CodeGraph** (<https://github.com/colbymchenry/codegraph>, npm @colbymchenry/codegraph) — a local SQLite semantic code-knowledge-graph exposed to agents over MCP (100% local, no cloud). (1) Install globally via npm with a user-writable npm prefix (no sudo). (2) codegraph init + codegraph index: .codegraph/config.json is tracked, .codegraph/codegraph.db is gitignored with codegraph index as its §11.4.77 regeneration mechanism; the config.json exclude list MUST exclude every credential/secret path per §11.4.10. (3) Wire codegraph serve --mcp into every CLI agent (Claude Code .mcp.json, OpenCode opencode.json, Qwen Code .qwen/settings.json, Crush .crush.json, host-local otherwise) referencing the bare codegraph command on PATH (no hardcoded host path). (4) Cover the integration with an anti-bluff suite whose per-agent end-to-end layer uses an unforgeable challenge (a fact obtainable only by calling a CodeGraph MCP tool, e.g. index node count via codegraph_status); a genuinely un-drivable agent is a documented SKIP per §11.4.3, never a faked PASS. (5) Document in docs/CODEGRAPH.md, kept in sync per §11.4.12 / §11.4.65. CodeGraph is consumed as the published npm package (§11.4.74) — not a git submodule, adds no Git remote. Planned gate CM-CODEGRAPH-WIRED + paired §1.1 mutation (strip a secret-exclusion → gate FAILs).

Cascade requirement: This anchor (verbatim or by §11.4.78 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-78-PROPAGATION; paired mutation strips the literal → gate FAILs. **Canonical authority:** constitution submodule Constitution.md §11.4.78 for the full mandate.

§11.4.79 — Own-Org Submodules MUST Be Included in the CodeGraph Index (cascaded from constitution submodule §11.4.79)

Verbatim user mandate (2026-05-21): *“All Submodules we use in the project and that are part of organizations to which we have the full access via GitHub, GitLab and other CLIs MUST BE included into the codegraph database and initialized / scanned / synced!”*

Refines §11.4.78’s exclude-list with a per-submodule-ownership split: (a) own-org submodules (full write access via the project’s CLIs — canonical orgs vasic-digital + HelixDevelopment) MUST be INCLUDED in the index; (b) third-party submodules (the §11.4.74 no-match → vendor path) MUST be EXCLUDED. Operational steps: (1) `git submodule update --remote --merge` to pull latest before re-indexing, respecting load-bearing pins on third-party submodules; (2) adjust `.codegraph/config.json` exclude list to keep own-org paths in scope; (3) re-index via `scripts/codegraph_setup.sh`; (4) verify via `scripts/codegraph_validate.sh` with ≥ 1 probe resolving a symbol living ONLY inside an own-org submodule; (5) paired §1.1 mutation — temporarily add the own-org submodule to exclude → validate MUST FAIL on the cross-submodule probe → restore. An index that lies about reachable symbols is a PASS-bluff against AI agents. Own-org submodules silently excluded without an audit trail in `.codegraph/config.json` comments is a release blocker.

Cascade requirement: This anchor (verbatim or by §11.4.79 reference) MUST appear in every owned submodule’s `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-79-PROPAGATION`; paired mutation strips the literal → gate FAILs. **Canonical authority:** constitution submodule `Constitution.md` §11.4.79 for the full mandate.

§11.4.80 — CodeGraph Regular-Update + Sync Automation Mandate (cascaded from constitution submodule §11.4.80)

Verbatim user mandate (2026-05-21): *“We MUST regularly check for the updates and execute codegraph npm updates so the latest version of it is always installed on the host machine! ... Make sure we have proper full automation bash scripts which will run regularly and that these are part of the constitution Submodule ... Make sure all updates, sync processes we do and important codegraph related events are all documented under docs/codegraph in Status and Status_Summary documents ... and regularly export them like all other Status docs into the PDF and HTML!”*

Three deliverables (all living in the constitution submodule, inherited by reference per §3 — consuming projects invoke at `${CONST_DIR}/scripts/codegraph_*.sh`, never copy): (1) `scripts/codegraph_update.sh` — `npm`-installs latest `@colbymchenry/codegraph` after a registry version check; appends old/new version to `docs/codegraph/Status.md`; anti-bluff verifies `codegraph` — version reflects the new version after install (`npm` exit 0 ≠ working binary). (2) `scripts/codegraph_sync.sh` — after a successful update runs `codegraph`

status → codegraph sync . → codegraph status → the project's scripts/codegraph_validate.sh; appends every step's output to BOTH the project's and the constitution's docs/codegraph/Status.md. (3) docs/codegraph/Status.md + Status_Summary.md append-only ledgers, exported to .html + .pdf per §11.4.65. Cadence: weekly floor (per §11.4.45). A consuming project that has not run codegraph_update.sh in >2 weeks AND has open AI-agent work is a release blocker. Paired §1.1 mutation: downgrade installed version → script detects drift → restore.

Cascade requirement: This anchor (verbatim or by §11.4.80 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-80-PROPAGATION; paired mutation strips the literal → gate FAILs. **Canonical authority:** constitution submodule Constitution.md §11.4.80 for the full mandate.

§11.4.81 — Cross-Platform-Parity Mandate (cascaded from constitution submodule §11.4.81)

Verbatim user mandate (2026-05-21): *“Any Linux-only blocker / issue we have MUST BE created macOS and other supported platforms equivalent! So, depending on platform proper implementation will be used for particular OS! EVERYTHING MUST BE PROPERLY EXTENDED AND UPDATED!”*

Every consuming project whose supported-platforms manifest lists more than one OS MUST, for every feature/test/gate/challenge/mutation depending on platform-specific primitives, ship a per-OS-equivalent implementation chosen at runtime via `uname -s` (or equivalent detection). Three sub-mandates: **(A) Per-OS implementation REQUIRED** — Linux `cgroup/systemd/proc` primitives MUST have documented per-OS equivalents (POSIX `setrlimit/ulimit`, macOS `launchd`, BSD `rctl`, Windows Job Object) chosen via runtime dispatch. **(B) Per-OS tests REQUIRED** — every platform-dependent gate test MUST have case `"$(uname -s)"` in branches with positive captured evidence per §11.4.2 + §11.4.5 in each branch; SKIP-with-reason acceptable ONLY when the platform genuinely cannot enforce the invariant. **(C) Honest kernel-gap citation + adjacent equivalent test REQUIRED** — where a Linux primitive has NO equivalent due to a documented kernel limitation (canonical: XNU does not enforce `RLIMIT_AS` for unprivileged processes), the test MUST detect the gap at runtime, SKIP with exact kernel reason + reproducer + honest-gap-doc link, AND provide an ADJACENT test exercising the closest invariant the platform CAN enforce (e.g. `RLIMIT_CPU+SIGXCPU` as the macOS proxy), itself anti-bluff with a paired §1.1 mutation. Gate CM-CROSS-PLATFORM-PARITY scans for case `"$(uname -s)"` blocks asserting a non-SKIP branch (or honest-gap citation) per platform in the manifest; paired mutation strips a Darwin branch → gate FAILs. No escape hatch.

Cascade requirement: This anchor (verbatim or by §11.4.81 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-81-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker on multi-platform projects. **Canonical authority:** constitution submodule Constitution.md §11.4.81 for the full mandate.

§11.4.82 — Iteration-Speedup Discipline Mandate (cascaded from constitution submodule §11.4.82)

Verbatim user mandate (2026-05-22): “How can we speed-up this whole development and fixing process? ... Do not forget to all speed optimizations critical rules and mandatory constraints **MUST BE** all added into our root (constitution Submodule) Constitution.md, CLAUDE.md, AGENTS.md and QWEN.md and all other relevant constitution Submodules files!”

Iteration cycle time is a first-order quality enabler. Every consuming project’s build / test / commit / debug pipeline **MUST** adopt these speedup disciplines AS **MANDATORY** (each independently enforceable): (A) Phase-1 forensic (superpowers:systematic-debugging) before any speculative source patch — speculative patches without FACT-grade root cause are §11.4.6 + §11.4.82 violations; (B) Live-ADB-First (or live-equivalent) before any rebuild — strengthens §11.4.51 to a release-blocker mandate; (C) 30-second pre-flight before launching rebuild orchestrators (device/sink reachability, host memory/disk, no stale locks, no orphan processes); (D) persistent build caches outside containers (ccache/sccache/Gradle daemon bind-mounted to host); (E) module-only rebuild for loadable-module-only changes; (F) parallel multi-device testing with separate qa-results/<TS>/<device-tag>/ outputs; (G) subagent scope discipline + worktree isolation (≤30 min budget, single-responsibility, isolation: "worktree" default); (H) lock-file + stale-process hygiene (clean .git/index.lock, disable auto git-gc in concurrent repos); (I) cycle telemetry per §11.4.24 (commit hash, per-phase wall-clock, speedup-flag set, outcome — aggregated weekly). Gate CM-ITERATION-SPEEDUP-DISCIPLINE audits recent cycles for telemetry citing which of (A)-(I) applied; paired §1.1 mutation strips the speedup-flag column → gate FAILs. No escape hatch — no --skip-phase1-forensic, --no-pre-flight, --rebuild-everything-always, --unlimited-subagent-scope, --ignore-locks, --no-telemetry flag.

Cascade requirement: This anchor (verbatim or by §11.4.82 reference) **MUST** appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-82-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.82 for the full mandate.

§11.4.83 — docs/qa/ End-User Evidence Mandate (cascaded from constitution submodule §11.4.83)

Verbatim user mandate (2026-05-22): “every feature that ships **MUST** carry a recorded e2e communication transcript + any attached materials under docs/qa/<run-id>/ (per-feature subdirectories). A feature with no QA transcript is itself a §107 PASS-bluff — it claims to work but has no auditable runtime evidence. Bot-driven automation **MUST** preserve full bidirectional communication threads as proof.”

Every feature that ships **MUST** carry a recorded end-to-end communication transcript plus any attached materials (screenshots, request/response payloads, audio, file uploads) committed under docs/qa/<run-id>/ — one directory per feature run. Operative rule: (1) every consuming project **MUST** maintain a docs/

qa/ tree, each new feature under docs/qa/<run-id>/ where <run-id> is monotonic + greppable (timestamp / ATM-NNN / other workable-item ID per §11.4.54); (2) transcripts MUST be full bidirectional — every prompt/command sent + every response received (one-sided is not a transcript); (3) attached materials MUST be committed in-repo (no external-only links — that is a §11.4.13 sink-side violation); (4) bot-driven / agent-driven QA automation MUST preserve the full conversation thread as the proof artefact; (5) release gates MUST refuse to tag a version that has any feature-shipping commit without its matching docs/qa/<run-id>/ directory. A feature with no QA transcript is a §11.4 / §107 PASS-bluff. Composes with §11.4.2 / §11.4.5 / §11.4.13 / §11.4.65 / §11.4.69 / §1.1.

Cascade requirement: This anchor (verbatim or by §11.4.83 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-83-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker — no --qa-evidence-optional escape hatch. **Canonical authority:** constitution submodule Constitution.md §11.4.83 for the full mandate.

§11.4.84 — Working-Tree Quiescence Rule for Subagent Commits (cascaded from constitution submodule §11.4.84)

Verbatim user mandate (2026-05-22): *“no subagent commit may proceed while any concurrent mutation gate is in flight in the same checkout. Before git add, the committing agent MUST grep its own working tree for mutation markers (MUTATED for paired, // always pass, return json.Marshal shortcut paths, etc.). Any unexplained file in the staging area triggers ABORT.”*

No subagent (or main-thread) commit may proceed while any concurrent mutation gate, paired-mutation experiment, or other in-flight mutation is live in the same checkout. Before git add, the committing agent MUST grep its own working tree for mutation markers (MUTATED for paired, // always pass, return json.Marshal shortcut paths, // MUTATION / # MUTATION annotations, _mutated_* filename suffixes, etc.) and explicitly account for every modified file in the staging area; any unexplained file → ABORT. (Forensic case: a logo-fix subagent's git add swept an // always pass JWT-verify mutation residue into an unrelated commit pushed to all four mirrors — a real security-defect window.) Operative rule: (1) pre-git add greps for mutation markers + cross-checks git status --porcelain against the subagent's declared scope; unaccounted entries → ABORT; (2) any active mutation gate MUST be serialised (mutate → assert FAIL → restore → assert PASS) and the working tree verifiably clean before any unrelated commit; (3) concurrent subagents in the SAME checkout MUST coordinate through a lockfile (.git/MUTATION_IN_PROGRESS) — cleaner solution is git worktree add per subagent (composes with §11.4.20/§11.4.70); (4) post-commit mutation-residue-scanner MUST run before push — any commit containing a mutation marker → push BLOCKED.

Cascade requirement: This anchor (verbatim or by §11.4.84 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-84-PROPAGATION; paired

mutation strips the literal → gate FAILs. A mutation marker that lands in a tagged commit is a critical defect regardless of how briefly it persisted. **Canonical authority:** constitution submodule Constitution.md §11.4.84 for the full mandate.

§11.4.85 — Stress + Chaos Test Mandate (cascaded from constitution submodule §11.4.85)

Verbatim user mandate (2026-05-24): *“Every fix or improvement you do MUST BE covered with full automation stress and chaos tests so we are sure nothing can break the functionality and all edge cases are monitored and polished and additionally fixed if that is needed! Everything must produce rock solid proofs and follow fully no-bluff policy!”*

Every fix or improvement landed MUST ship with full-automation **stress** AND **chaos** test suites exercising edge cases, sustained load, concurrent contention, and failure-injection. Happy-path coverage alone is a §11.4 / §107 PASS-bluff at the resilience layer. **Stress** (closed-set): sustained load ($N \geq 100$ iterations OR ≥ 30 s wall-clock, p50/p95/p99 latency recorded) + concurrent contention ($N \geq 10$ parallel invocations, no deadlock/leak) + boundary conditions (empty/max/off-by-one, each categorised). **Chaos** (closed-set, per fix-class appropriateness): process-death injection + network-fault injection (drop/delay/reorder) + input-corruption injection + resource-exhaustion injection (disk full, OOM, FD exhaustion — refuse cleanly OR degrade, NEVER crash) + state-corruption injection (mid-flight lock loss, partial-write). Every stress + chaos PASS MUST cite a captured-evidence artefact path per §11.4.5 + §11.4.69. Helper library `stress_chaos.sh` provides `ab_stress_run`, `ab_stress_concurrent`, `ab_chaos_kill_pid_during`, `ab_chaos_drop_network_during`, `ab_chaos_corrupt_file_during`, `ab_chaos_oom_pressure_during`, `ab_chaos_disk_full_during`, each composing with `ab_pass_with_evidence` / `ab_skip_with_reason`. Cleanup non-negotiable in `trap '... ' EXIT` (cleanup failure = §11.4.14 violation). Four-layer coverage per §11.4.4(b) + paired §1.1 mutation (strip chaos-injection or evidence-capture → gate FAILs). No escape hatch — no `--skip-stress`, `--no-chaos`, `--happy-path-suffices`, `--stress-test-later` flag.

Cascade requirement: This anchor (verbatim or by §11.4.85 reference) MUST appear in every owned submodule’s `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-85-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.85 for the full mandate.

§11.4.86 — Roster/Corpus-Backed Status-Doc Auto-Sync Mandate (cascaded from constitution submodule §11.4.86)

Verbatim user mandate (2026-05-25): *“Make sure that assets and players Status docs are ALWAYS regularly updated and in sync like all others Status docs — any time we add or modify the assets content(s) or we change or add new / remove existing pre-installed video and audio player apps! This MUST WORK OUT OF THE BOX!”*

Some Status docs (§11.4.45) are backed by a tracked roster (installed apps/components) or a tracked asset corpus (test/media asset directory) rather than narrative alone. Their freshness MUST NOT depend on operator vigilance — the moment a roster/corpus member changes (app added/removed/renamed; asset added/modified/removed) the Status doc + Status_Summary + HTML + PDF MUST resync out of the box, mechanically. Mechanism (all must hold): (1) drift-proof fingerprint — sha256 of the sorted member list (NOT mtime), persisted in a sidecar beside the Status doc; (2) a sync helper that regenerates the fingerprint + re-exports HTML+PDF via the §11.4.65 exporter, wired so sync is automatic; (3) a pre-build gate that FAILs when the live fingerprint differs from the persisted one (mirrors §11.4.12 CM-ISSUES-SUMMARY-SYNC + §11.4.45 sync_integration_status); (4) a paired §1.1 mutation corrupting the fingerprint and asserting the gate FAILs. Classification: universal — the consuming project supplies the specific docs, roster/corpus sources, helper, and gate name per §11.4.35.

Cascade requirement: This anchor (verbatim or by §11.4.86 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-86-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker — no --skip-roster-sync, --allow-status-drift, --roster-sync-not-applicable flag.
Canonical authority: constitution submodule Constitution.md §11.4.86 for the full mandate.

§11.4.87 — Endless-Loop Autonomous Work + Zero-Idle Agent Dispatch + Anti-Bluff Testing Mandate (cascaded from constitution submodule §11.4.87)

Verbatim user mandate (2026-05-26): *“continue in endless loop fully autonomously”* (and any semantically-equivalent phrasing).

When the operator instructs an AI agent to continue in an endless autonomous loop, the agent MUST treat it as a HARD-CONTRACT covenant: (A) continue working until docs/Issues.md Status-column has zero non-terminal entries AND docs/CONTINUATION.md §3 Active work is empty AND no background subagent is mid-execution AND no external dependency is in-flight; (B) dispatch background subagents for parallelisable work — main + every subagent operate concurrently, “waiting for results” is the ONLY acceptable idle reason; (C) every closure lands four-layer test coverage per §11.4.4(b) with captured-evidence (audio/video/network/UI/sysfs physical proofs); (D) the §11.4 anti-bluff covenant family (§11.4.1 / §11.4.2 / §11.4.6 / §11.4.7 / §11.4.27 / §11.4.50 / §11.4.52 / §11.4.68 /

§11.4.69 / §11.4.83) is the operative truth-discipline — tests AND HelixQA Challenges bound equally; (E) the loop terminates ONLY on all-conditions-met, explicit operator STOP, host-session-safety demand, or scheduled wake on a known-future-actionable signal. No escape hatch — no `--idle-OK`, `--skip-endless-loop`, `--bluff-permitted-for-this-task`, `--metadata-only-test-suffices`, `--no-physical-proof-required` flag.

Cascade requirement: This anchor (verbatim or by §11.4.87 reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-87-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.87 for the full mandate.

§11.4.88 — Background-Push Mandate: Commit-Lock Release Immediately After Commit, Push Runs Detached (cascaded from constitution submodule §11.4.88)

Forensic anchor (2026-05-26): a single `commit_all.sh` held its flock ~5 hours because `do_push` ran synchronously after the commit landed — every subsequent commit blocked on a slow mirror push irrelevant to the local commit's durability. Implementation seam for §11.4.87(B) zero-idle. The mandate: (A) `.git/.commit_all.lock` MUST be released IMMEDIATELY after `git commit` returns 0 — the commit is durable on local disk regardless of remote push outcome; (B) push runs detached via `nohup ./push_all.sh ... > <log> 2>&1 & + disown` — the orchestrator's exit code reports COMMIT success, NOT push success; (C) `push_all.sh` acquires per-remote flock `.git/.push.<remote>.lock` so concurrent invocations targeting the same remote serialize but different-remote invocations run in parallel; (D) backgrounded push failures land in `qa-results/push_failures/<ts>_<remote>.log` — the next autonomous-loop tick checks per §11.4.87(A) “no external dependency in-flight” gate; (E) synchronous-push escape: explicit `--sync-push` CLI flag preserves legacy behaviour for §11.4.41 force-push merge-first audit paths. Gates `CM-COVENANT-114-88-PROPAGATION` + `CM-BACKGROUND-PUSH-WIRED` + paired §1.1 mutations. Synchronous push (without `--sync-push`) = §11.4 PASS-bluff at the execution layer.

Cascade requirement: This anchor (verbatim or by §11.4.88 reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-88-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker — no escape hatch beyond `--sync-push` for force-push events. **Canonical authority:** constitution submodule `Constitution.md` §11.4.88 for the full mandate.

§11.4.89 — Background Test Execution Mandate (cascaded from constitution submodule §11.4.89)

Verbatim user mandate (2026-05-27): “Any tests we are executing, especially long test cycles, *MUST BE performed in background in parallel with main work stream! This MUST NOT block our capabilities to work on queued workable items. Main work stream can be blocked or sit iddle only if absolutely needed and if it depends hard on results of some background execution.*”

Symmetric anchor to §11.4.88 (background push) at the test-execution layer. Mandate: (A) long-running tests (>30 s expected: pre_build, meta_test, test_all_fixes, recent_work_validate, HelixQA banks, 4-phase cycles, full-suite retests, audio supervisors, dual-display recorders) MUST run via `nohup ... > <log> 2>&1 & + disown` with the log under a known dir (qa-results/<test_id>_<ts>.log); (B) the main stream proceeds to the §11.4.42 priority queue immediately; (C) hard-dependency gating — poll an exit-status file or `pgrep -af <test>` before steps that need the exit code, surfacing as §11.4.66 interactive options if the test is still running; (D) failures land in <log> files, the next loop tick checks; (E) foreground execution permitted ONLY for <30 s tests OR explicit operator authorisation; (F) per-script flock serialises same-script invocations, different-script invocations parallel. Gates CM-COVENANT-114-89-PROPAGATION + CM-BACKGROUND-TEST-EXECUTION-WIRED + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by §11.4.89 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-89-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker — no escape hatch beyond explicit per-invocation operator authorisation. **Canonical authority:** constitution submodule Constitution.md §11.4.89 for the full mandate.

§11.4.90 — Obsolete Status + Per-Item Obsolescence Audit (cascaded from constitution submodule §11.4.90)

Verbatim user mandate (2026-05-27): “Bug No 6 ... *seems obsolete after latest request for new behavior ... mark obsolete tickets with some light gray background ... text - the description to be strikethrough styled ... review all existing open or resolved workable items if they are obsolete - not valid any more ... There MUST NOT be any mistake! No bluff is allowed of any kind!*”

The §11.4.15 Status closed-set is extended with a terminal Obsolete (→ Fixed.md) value (orthogonal to Type per §11.4.16). Obsolescence reasons (closed vocabulary): superseded-by-design-change | superseded-by-later-mandate | feature-removed | duplicate-of | unsupported-topology. Every Obsolete heading MUST carry an ****Obsolete-Details:**** line (Since + Reason + Superseding-item + Triple-check evidence) within 8 non-blank lines. The §11.4.23 colorizer adds a cell-status-obsolete class — light-gray #E0E0E0 background + strikethrough description. Audit cadence: every release-gate sweep

per §11.4.40 + §11.4.42; triple-check is non-negotiable per the operator mandate. Composes with §11.4.15 / §11.4.16 / §11.4.19 / §11.4.21 / §11.4.23 / §11.4.33 / §11.4.34 / §11.4.40 / §11.4.42 / §11.4.66 / §11.4.71. Gates CM-COVENANT-114-90-PROPAGATION + CM-ITEM-OBSOLETE-DETAILS + CM-OBSOLETE-COLORIZER-WIRED + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by §11.4.90 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-90-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.90 for the full mandate.

§11.4.91 — Summary-Doc Clarity Mandate (cascaded from constitution submodule §11.4.91)

Verbatim user mandate (2026-05-27): *“Summary docs - Issues_Summary some not clear one line descriptions - like ‘Composes with’ ... For each workable item we MUST HAVE clearly understandable meaning ... every team member can clearly understand what that particular workable item is exactly about! There cannot be misunderstanding or unclarity of any kind and no bluff allowed!”*

Every summary entry (Issues_Summary, Fixed_Summary, README doc-link, Status_Summary pages 1+2, all one-liners) MUST contain a self-contained meaningful description ≥ 6 words OR ≥ 40 chars naming SUBJECT + PROBLEM/GOAL. Forbidden one-liner anti-patterns: section labels (Composes with, Closure criteria, Fix direction, etc.); bare metadata fragments (Critical, Bug, In progress, etc.); section-marker echoes; a §-letter alone. Generators (generate_issues_summary.sh / generate_fixed_summary.sh / update_readme_doc_links.sh / generate_status_summary.sh) MUST extract from the H1/H2 heading line per the §11.4.54 ATM-NNN convention, NEVER from arbitrary downstream text, and MUST refuse anti-pattern rows — emitting a (MISSING DESCRIPTION – fix source heading) placeholder with visual highlight. Gate CM-SUMMARY-CLARITY-DESCRIPTIONS scans every summary; an anti-pattern match = FAIL. Audit cadence: every §11.4.40 + §11.4.42 sweep.

Cascade requirement: This anchor (verbatim or by §11.4.91 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-91-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.91 for the full mandate.

§11.4.92 — Multi-Pass Change-Evaluation Discipline (cascaded from constitution submodule §11.4.92)

Verbatim user mandate (2026-05-27): *“Every change to the project or codebase we do MUST BE evaluated in several passes and in in-depth analysis for potential new issues or problems it can introduce! ... no bluff of any kind! After we do change or set of changes this mandatory steps MUST BE taken!”*

Every non-trivial change MUST pass a 5-pass evaluation BEFORE it is commit-ready: **(Pass 1)** main-task verification — change achieves the stated goal, captured-evidence per §11.4.5/§11.4.69; **(Pass 2)** regression-blast-radius analysis — enumerate every direct dependency, demonstrate no contract break; **(Pass 3)** cross-feature interaction analysis — audit parallel features sharing state/timing/hardware/shell environment; **(Pass 4)** deep-research validation per §11.4.8 — external precedent OR “NO external solution found — original work” + CodeGraph queries per §11.4.78/§11.4.79; **(Pass 5)** anti-bluff confirmation per §11.4 / §11.4.1 / §11.4.6 / §11.4.27 / §11.4.50 / §11.4.52 / §11.4.69 / §11.4.83 — no new bluff surface introduced. Each pass is documented (commit footers OR docs/ entries OR qa-results/ evidence). Only after all 5 passes complete may commit/push/test/release proceed. Trivial exemption: typo / revision-bump / MD-export-regen IF zero source touched AND the commit message cites the exemption explicitly. Gates CM-COVENANT-114-92-PROPAGATION + CM-MULTI-PASS-EVALUATION-EVIDENCE + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by §11.4.92 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-92-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.92 for the full mandate.

§11.4.93 — SQLite-Backed Single-Source-of-Truth for Workable Items (cascaded from constitution submodule §11.4.93)

Verbatim user mandate (2026-05-27): *“There MUST be single source of truth for all of our workable items - SQLite database ... proper scripts (we recommend Go programs) ... reduce a chance for sync to be broken ... generate always all docs from DB or to re-generate Db from all docs we have in opposite direction”*

The text-based Issues/Fixed/Summary/CONTINUATION constellation is converted to a SQLite-DB-backed single source of truth. Schema mandatory tables: items (atm_id PK + Type + Status incl. Obsolete + Severity + title + description ≥40 chars + created/modified + composes_with JSON + current_location); item_history (append-only audit per §11.4.34 By/Reason/Evidence); obsolete_details (§11.4.90); operator_block_details (§11.4.21); firebase_metadata (§11.4.47); meta (schema version + last sync + integrity hash). A Go binary at cmd/workable-items/ provides sync md-to-db / db-to-md / diff / validate / add / close; bidirectional regen is byte-identical round-trip (closed-set whitespace/

section-order tolerance). `commit_all.sh` refuses on non-empty diff; `sync_issues_docs.sh` invokes the Go binary; pre-build runs `workable-items` validate. Anti-bluff: unit + integration + stress (1000-row insert + 10 concurrent writers) + chaos (mid-write SIGKILL + corrupt-DB recovery + disk-full) + paired §1.1 mutation + HelixQA Challenge CME-WORKABLE-ITEMS-001. The Go binary lives in the constitution submodule (`constitution/scripts/workable-items/`) per §11.4.74. Gates CM-COVENANT-114-93-PROPAGATION + CM-WORKABLE-ITEMS-DB-PRESENT + CM-WORKABLE-ITEMS-MD-DB-IN-SYNC + paired §1.1 mutations. (NOTE: the DB tracking rule is AMENDED by §11.4.95 — DB is TRACKED, not gitignored.)

Cascade requirement: This anchor (verbatim or by §11.4.93 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-93-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker — text-based-only trackers are a §11.4 PASS-bluff at the data-architecture layer. **Canonical authority:** constitution submodule Constitution.md §11.4.93 for the full mandate.

§11.4.94 — Zero-Idle Priority-First Parallel-By-Default Operating Mode (cascaded from constitution submodule §11.4.94)

Verbatim user mandate (2026-05-27): *“We MUST NEVER sit iddle / wait or sleep if there is possibility for us to work on something ... Always check if there is a possibility to work on something while we are not working actively on something! Pick always by priority - most critical workable items and other tasks MUST BE done first! ... Stay still / iddle if nothing is left to be done at all or waiting for something that is blocking us / you!!!”*

§11.4.94 binds §11.4.20 + §11.4.42 + §11.4.58 + §11.4.70 + §11.4.72 + §11.4.82 + §11.4.87 + §11.4.88 + §11.4.89 into a single always-on enforcement: (A) idle ONLY when every queued item is genuinely blocked on an external dependency (hardware / network upstream / build/test completion the conductor cannot accelerate) OR operator STOP OR §12 host-safety — “don’t see what to do” is NEVER valid; (B) before ANY wake/sleep the conductor MUST survey parallel-work feasibility per §11.4.42 + §11.4.72 + §11.4.87, identify non-contending items, and dispatch in parallel per §11.4.20/§11.4.70 (subagent) + §11.4.58 (PWU disjoint scope) + §11.4.89 (background long tests); (C) priority order MANDATORY — pick highest-severity + §11.4.72 audio-first the conductor can autonomously progress; (D) subagent-driven default for non-trivial; (E) background default for >30 s wall-clock work via `nohup+disown`; (F) stability-preserving (composes with §11.4.92 multi-pass + §11.4.84 quiescence + §12.6–§12.9 host safety); (G) progress updates surfaced at milestone boundaries. Gates CM-COVENANT-114-94-PROPAGATION + CM-PARALLEL-WORK-AUDIT + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by §11.4.94 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-94-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.94 for the full mandate.

§11.4.95 — Workable-Items SQLite DB Is TRACKED in Git, NEVER Gitignored (cascaded from constitution submodule §11.4.95)

Verbatim user mandate (2026-05-27): “*We shall not Git ignore our workable items SQLite DB since it is our single source of truth ... workable items SQLite DB regularly committed and pushed to all upstreams!*”

§11.4.93’s earlier “gitignored per §11.4.30” clause is AMENDED — the DB at docs/workable_items.db is TRACKED in git, NEVER gitignored. It IS authoritative source data, NOT a build artefact. Every workable-items sync md-to-db that mutates state MUST stage + commit + push the DB alongside the MD regen per §11.4.19 atomic-move + §2.1 multi-upstream push. A WAL-checkpoint (PRAGMA wal_checkpoint(TRUNCATE)) is required before commit-stage so the transient .db-wal + .db-shm sidecars (gitignored per §11.4.30) are safely discardable. The §11.4.77 regeneration mechanism does NOT apply — the DB IS the source. Destructive DB ops require §9.2 hardlinked-backup + operator authorization; §11.4.41 force-push merge-first applies if DB history ever needs rewrite. Gates CM-COVENANT-114-95-PROPAGATION + CM-WORKABLE-ITEMS-DB-TRACKED + paired §1.1 mutation.

Cascade requirement: This anchor (verbatim or by §11.4.95 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-95-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.95 for the full mandate.

§11.4.96 — Safe-Parallel-Work-With-Long-Build Catalogue + Mandate (cascaded from constitution submodule §11.4.96)

Verbatim user mandate (2026-05-27): “*Are there except AOSP build process any other active jobs being done at the moment? Can we work on something in parallel while build is in progress so we slowly cleanup our slate? ... do as much as possible work in background in parallel with main work stream and oreferably using subagents-driven approach!*”

An operational catalogue for the canonical long-running workload (multi-hour containerised build per §12.9). **SAFE during build:** (A) MD/docs work; (B) generator/helper script work under scripts/; (C) pre-build + meta-test gate authoring + paired §1.1 mutations; (D) on-device test scripts; (E) constitution submodule edits + push; (F) any submodule commit + push per §11.4.88; (G) read-only live-ADB probes (dumpsys/getprop/cat /proc/.../screencap/logcat); (H) subagent dispatch per §11.4.20/§11.4.70 + §11.4.84 quiescence; (I) web research + external API queries with §11.4.10 credentials; (J) workable-items DB ops per §11.4.93+§11.4.95; (K) backgrounded pre-build + meta-test execution per §11.4.89. **UNSAFE during build:** (α) git checkout/reset --hard/clean -df on the source tree (use git worktree); (β) mass file deletes/renames under built source trees; (γ) submodule pointer updates affecting built artefacts; (δ) out/ mutations; (ε) make clean/m clobber/rm -rf out/; (ζ) container destruction; (η) disk-filling

breaching §12.9 free-space minimum; (θ) §12 host-session-safety breaches. Conductor responsibility: before EVERY pause point during a long build, consult the catalogue, identify (A)-(K) queue items per §11.4.42+§11.4.72, and dispatch ≥ 1 per §11.4.20/§11.4.70 subagent default + §11.4.89 background. “Build running, nothing else to do” is NEVER true per §11.4.94+§11.4.96. Gates CM-COVENANT-114-96-PROPAGATION + CM-PARALLEL-WORK-DURING-BUILD-AUDIT + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by §11.4.96 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-96-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.96 for the full mandate.

§11.4.97 — Maximum-Use-of-Idle-Time + Progress-Update Cadence (cascaded from constitution submodule §11.4.97)

Verbatim user mandate (2026-05-27): *“keep it working, we should do as much as possible, if not it all but as much as we can as long as there is iddle time! it MUST be used! ... keep us updated about all progress and all phisycal proofs and gathered data as you progress through all open workable items!”*

Operating-mode capstone strengthening §11.4.87 + §11.4.94 + §11.4.96: (A) every minute of conductor idle time during which work could autonomously progress AND is not genuinely blocked = a §11.4.97 violation; “as much as possible, if not it all but as much as we can” is operative — dispatch CONTINUOUSLY through the entire idle window, not just at scheduled wakes; (B) progress-update cadence — emit an operator-facing 1-line update at every commit landed / subagent return / constitutional anchor / captured evidence / milestone closure, no operator prompt required; (C) continuous physical-proof gathering per §11.4.5 + §11.4.6 + §11.4.69 — every autonomous closure cites captured-evidence (evidence path goes into the §11.4.93 item_history.evidence_path when the DB lands); (D) composes with §11.4.5/6/13/20/27/42/50/52/69/70/72/83/85/87/88/89/94/96; (E) the idle-only-when-blocked closed-set is unchanged from §11.4.94(A). Gates CM-COVENANT-114-97-PROPAGATION + CM-IDLE-TIME-AUDIT + paired §1.1 mutations.

Cascade requirement: This anchor (verbatim or by §11.4.97 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-97-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.97 for the full mandate.

§11.4.98 — Full-Automation Anti-Bluff Mandate (cascaded from constitution submodule §11.4.98)

Verbatim user mandate (2026-05-28): *“Make sure we have full automation testing of all scenarios with real bot, main group and users without any manual intervention or contribution of real user! Everything MUST BE fully automatic and autonomous! These tests MUST BE able to rerun endless times when needed! ... Make sure there is no false positives in testing! Every test and its results MUST obtain real proofs of everything working! No bluff is allowed!”*

Closes the manual-intervention gap (§11.4 / §11.4.2 / §11.4.5 / §11.4.50 / §11.4.85 / §11.4.87 / §11.4.89 / §11.4.94 did not explicitly forbid it). A live/integration/e2e/Challenge test that requires a human action during execution (typing a message, clicking UI, hand-triggering a webhook, attaching a file — anything beyond startup) is by definition a §11.4 PASS-bluff at the automation layer. (A) Every governed test — unit/integration/e2e/Challenge/stress/chaos/live — MUST be fully self-driving end-to-end, reporting PASS/FAIL/SKIP-with-reason without any further human action after startup. (B) Single permissible exception: one-time credential bootstrap performed OUTSIDE test execution (.env from vault, shell exports, OAuth at first install, MTPROTO session activation) — configuration, not test driving. (C) Live messenger/channel/agent tests: no “operator must type” prompts (drive programmatically via second account / webhook fixture / loopback); no hard-coded session UUIDs that collide with the active dev session (Herald 2026-05-28 c laude --resume silent exit -1 lesson); no 60 s human-response windows (§11.4.50 determinism violation); re-runability proof — PASS at -count=3 consecutive automated invocations with self-cleaning state; §11.4.98 obsolescence audit classifies every existing test COMPLIANT vs NON-COMPLIANT; no silent-skip-reported-as-PASS or stale-evidence-as-fresh. (D) With §11.4.85 + §11.4.89 + §11.4.87 + §11.4.94 forms a continuously-validated, non-flake, anti-bluff regime. (F) Manual-dependency tests not rewritten within 30 days graduate to §11.4.90 Obsolete citing §11.4.98.

Cascade requirement: This anchor (verbatim or by §11.4.98 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-98-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.98 for the full mandate.

§11.4.99 — Latest-Source Documentation Cross-Reference Mandate (cascaded from constitution submodule §11.4.99)

Verbatim user mandate (2026-05-28): *“Make sure we ALWAYS check against latest versions of services we use web / online docs before creating instructions! This situation is illustration of how we can misguide ourselves or get banned! ... These are mandatory rules / constraints and the result is consistency and safety of created instructions, guides and manuals!”*

Misguidance-by-stale-docs is the same severity class as a §11.4 PASS-bluff at the documentation layer (Herald 2026-05-28 case: a first-draft MTProto guide recommended VoIP fallback numbers and omitted the `recover@telegram.org` pre-login email — both contradicted Telegram’s official docs + the `gotd/td` maintainer guide and could have caused a permanent account ban). Closes the gap §11.4.92 Pass 4 alludes to but does not mandate. (A) Before committing any operator-facing instruction/guide/manual/troubleshooting/setup doc, the author **MUST**: (1) fetch the LATEST official online documentation of the documented service/library via WebFetch / MCP / direct browsing — NEVER training data, memory, or prior committed docs; (2) cross-reference every instruction step against that source; (3) seek secondary authoritative sources (maintainer SUPPORT.md, official changelogs, vetted community FAQs) when the official source is sparse/silent; (4) cite source URLs + date in a `## Sources verified` footer in the doc; (5) cite a `Sources verified <date>: <urls>` footer in the commit message. (B) Negative findings (gaps/silences/contradictions) **MUST** be documented explicitly. (C) Docs older than 6 months are STALE — re-verify before citing as operator authority, at every vN.0.0 release boundary, on service breaking-change announcements, or on operator error reports. (D) Risk-classified services (messaging, cloud APIs, payment systems, AI/LLM providers, code-hosting, package managers) carry a 90-day max staleness + explicit safety warnings. (E) Composes with but is **INDEPENDENT** of §11.4.92 Pass 4. (G) Commit missing either footer is **BLOCKED** at release-gate; stale-beyond-grace docs graduate to §11.4.90 Obsolete (Reason=stale-documentation).

Cascade requirement: This anchor (verbatim or by §11.4.99 reference) **MUST** appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-99-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.99 for the full mandate.

§11.4.101 — Autonomous-Decision-Over-Blocking Mandate (cascaded from constitution submodule §11.4.101)

Verbatim user mandate (2026-05-28): *“when working in endless working loop fully autonomously try to decide most properly about points which would block execution and wait for us. If we haven’t answered now work would be blocked whole night! If possible and if that will not cause any issues make proper and most reliable and safe decision so we achieve maximal efficiency and work gets fully done!”*

In autonomous / endless-loop mode (per §11.4.87), the agent **MUST** minimize operator-blocking and make the safe, reliable, reversible decision itself so work is not stalled (e.g. overnight) waiting for input — §11.4.87 says keep working, §11.4.101 says HOW to clear the decision points. **Proceed-autonomously (closed-set, ALL must hold):** (a) the action is reversible OR has a captured pre-op backup per §9.2; (b) the safe choice is determinable from captured evidence per §11.4.6 (no guessing — LIKELY/probably/seems is NOT a determination); (c) a wrong choice’s blast radius is bounded AND recoverable; (d) it composes with anti-bluff §11.4, host-safety §12, data-safety §9. **Block-only-when (BLOCK via the §11.4.66 interactive mechanism ONLY when ALL hold):** the action is irreversible AND

high-blast-radius AND the safe choice cannot be determined from evidence — e.g. external-account state the agent cannot inspect, hardware it cannot access, destructive ops without backup, force-push (also §9.2 + §11.4.41), spending money or sending data to third parties. Operator-blocked per §11.4.21 is reached only after this rule fires AND the self-resolution-exhaustion audit completes. An unavoidable block parks one work unit — it does NOT pause the loop; the agent keeps progressing every non-blocked item in parallel per §11.4.87 + §11.4.94 (posing the question then going idle is a §11.4.94 + §11.4.97 violation). Classification: universal (§11.4.17).

Cascade requirement: This anchor (verbatim or by §11.4.101 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Propagation gate CM-COVENANT-114-101-PROPAGATION; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.101 for the full mandate.

§11.4.102 — Mandatory systematic-debugging activation + always-loaded skill-discovery + plugin-dependency availability (cascaded from constitution submodule §11.4.102)

Verbatim user mandate (2026-05-29): **“Make sure that we ALWAYS trigger / start the”/superpowers:systematic-debugging” skills when any issues happen! If this is possible to activate and use in this situations out of the box when we spot problems / issues / bugs / misalignments / inconsistencies we MUST activate the skill(s) and make strongest efforts in full in depth analysis / debugging and determine root causes of all problem or obtain relevant data and information we need! ... we MUST make sure that “/using-superpowers” skill is ALWAYS loaded, applied and used! All dependencies (plugins) that Claude Code or other market places are offering MUST BE installed if these are not already available for loading and use!”*

Three cooperating invariants — the difference between guess-and-retry and investigate-to-root-cause-first. **(A) Mandatory systematic-debugging activation.** On ANY spotted issue / bug / test failure / gate failure / regression / misalignment / inconsistency / unexpected behaviour, the agent MUST activate `superpowers:systematic-debugging` (or the platform-equivalent structured-debugging discipline) **BEFORE proposing, writing, or applying any fix** — the **Iron Law: NO FIXES WITHOUT ROOT CAUSE INVESTIGATION FIRST.** Full four-phase arc: root-cause → pattern → hypothesis → implementation. Guess-and-retry, symptom-patching, and re-running a failed test hoping it passes (“probably transient / flaky”) WITHOUT a completed investigation are §11.4.102 violations; calling a failure transient/flaky/intermittent/probably-timing without captured forensic evidence is simultaneously a §11.4.6 and §11.4.7 violation. **(B) Mandatory always-loaded using-superpowers.** `superpowers:using-superpowers` (or platform-equivalent skill-discovery discipline) MUST be loaded and applied at session start and consulted before any task; if ANY skill could apply — even at 1% relevance — it MUST be invoked rather than improvised from memory. **(C) Mandatory plugin / dependency availability.** Every skill plugin / marketplace package / capability dependency the project relies on MUST be installed + loadable BEFORE the dependent work

proceeds; a missing plugin that blocks a mandated skill is a release-blocker until installed + confirmed loadable (install exit 0 ≠ skill loadable — confirm by observing the skill in the live capability list). Composes with §11.4.4 / §11.4.6 / §11.4.7 / §11.4.8 / §11.4.43 / §11.4.70 / §11.4.82(A) / §11.4.92. Classification: universal (§11.4.17). No escape hatch — no `--skip-systematic-debugging`, `--guess-and-retry-OK`, `--symptom-patch-permitted`, `--skip-skill-discovery`, `--plugin-optional`, `--missing-plugin-is-warning` flag.

Cascade requirement: This anchor (verbatim or by §11.4.102 reference) **MUST** appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Propagation gate `CM-COVENANT-114-102-PROPAGATION`; paired mutation strips the literal → gate FAILs. Release blocker. **Canonical authority:** constitution submodule `Constitution.md` §11.4.102 for the full mandate.

§11.4.103 — Continuous parallel-stream working routine (cascaded from constitution submodule §11.4.103)

Cascade reference — see constitution submodule `Constitution.md` §11.4.103 for the full mandate. The main work stream **MUST** always stay FREE. ALL commit AND push operations run detached. At least three parallel background subagent-driven streams **MUST** run alongside the main stream whenever three or more non-contending actionable items exist; the moment any one stream finishes a new stream **MUST** immediately start. Most-critical and most-visible first; audio always top per §11.4.72. Propagation gate `CM-COVENANT-114-103-PROPAGATION` (literal 11.4.103). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.103.

§11.4.104 — Participant identity, attribution and notification-tagging (cascaded from constitution submodule §11.4.104)

Cascade reference — see constitution submodule `Constitution.md` §11.4.104 for the full mandate. Every messenger **MUST** relate messages to a Participant (logical Subscriber/User); the same logical person may have a different username per messenger. Workable items **MUST** carry `created_by` and `assigned_to` (canonical handles). Notifications **MUST** @-tag the right participant — never the operator and never the system agent. Propagation gate `CM-COVENANT-114-104-PROPAGATION` (literal 11.4.104). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.104.

§11.4.105 — Natural-language intent recognition and clarification (cascaded from constitution submodule §11.4.105)

Cascade reference — see constitution submodule `Constitution.md` §11.4.105 for the full mandate. Users **MUST NOT** need to know command syntax. The System determines intent via three-tier resolution: recognise existing commands, infer exact intent, or reply-tag-and-ask with a precise clarifying question. Never guess, never drop a message silently. Propagation gate `CM-COVENANT-114-105-PROPAGATION` (literal `11.4.105`). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.105.

§11.4.106 — Docs Chain — mechanical documentation/DB sync engine (cascaded from constitution submodule §11.4.106)

Cascade reference — see constitution submodule `Constitution.md` §11.4.106 for the full mandate. Docs Chain (`vasic-digital/docs_chain`) is the canonical mechanical enforcer of documentation-sync mandates. Consumers **MUST** use the engine instead of ad-hoc sync scripts, register chains via per-context YAML, and never accept a faked transform. Propagation gate `CM-COVENANT-114-106-PROPAGATION` (literal `11.4.106`). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.106.

§11.4.107 — Anti-bluff AV/test-validation techniques mandate (cascaded from constitution submodule §11.4.107)

Cascade reference — see constitution submodule `Constitution.md` §11.4.107 for the full mandate. A single captured frame is **NOT** proof of playback. Every test asserting audio/video output **MUST** prove liveness via freeze-detection oracle **AND** an independent frame-advance counter from platform telemetry. No-stale-from-previous cross-check, measured FPS, no-flash-on-wrong-output, drive through realistic UI path, metamorphic relations, full-reference quality metrics, mutation-test every analyzer, per-channel audio RMS/loudness, calibrated thresholds. Propagation gate `CM-COVENANT-114-107-PROPAGATION` (literal `11.4.107`). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.107.

§11.4.108 — Four-layer fix-verification + runtime-signature-as-definition-of-done mandate (cascaded from constitution submodule §11.4.108)

Cascade reference — see constitution submodule `Constitution.md` §11.4.108 for the full mandate. A fix crosses four distinct layers: SOURCE, ARTIFACT, RUNTIME-ON-CLEAN-TARGET, USER-VISIBLE. Green at layer 1 says nothing about layers 2–4. Every fix declares one machine-checkable runtime signature verified on a clean deployment. Gates span all four layers. Deployment MUST yield a clean state. Three or more “fixed-but-not-working” discoveries signal an architectural verification flaw — stop patching symptoms, fix the verification pipeline. Propagation gate CM-COVENANT-114-108-PROPAGATION (literal 11.4.108). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.108.

§11.4.109 — Mandatory Anti-Forgetting Enforcement: PreToolUse Guard Hook + Subagent Constitutional Preamble + Orchestrator Pre-Action Checklist (cascaded from constitution submodule §11.4.109)

Cascade reference — see constitution submodule `Constitution.md` §11.4.109 for the full mandate. Every consumer MUST wire `constitution/scripts/hooks/guard-forbidden-commands.sh` as a `PreToolUse` hook, maintain `docs/AGENT_GUARDRAILS.md` with the subagent constitutional preamble and orchestrator pre-action checklist, and provide a hermetic hook test suite. The hook is inherited by reference, never copied. Propagation gate CM-COVENANT-114-109-PROPAGATION (literal 11.4.109). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.109.

§11.4.110 — Pre-build build-readiness verdict + change-impact clash detection mandate (cascaded from constitution submodule §11.4.110)

Cascade reference — see constitution submodule `Constitution.md` §11.4.110 for the full mandate. A single deterministic READY-FOR-BUILD verdict gates every rebuild. A diff-driven change-impact and clash detector cross-checks every newly-introduced second-artifact dependency. Coverage-completeness is a gate. Two-speed honesty separates grep-speed always-on gates from heavy `REQUIRES_BUILD` gates. Every gate is anti-bluff by paired §1.1 mutation. Propagation gate CM-COVENANT-114-110-PROPAGATION (literal 11.4.110). Non-compliance is a release blocker.

§11.4.111 — Resolve-by-stable-name-not-by-enumeration-index mandate (cascaded from constitution submodule §11.4.111)

Cascade reference — see constitution submodule Constitution.md §11.4.111 for the full mandate. Any binding to a hardware device / resource handle / enumerated entity MUST resolve by a stable identifier (name / UUID / serial / label) and MUST NOT bind by enumeration index / ordinal / slot unless the platform documents that ordinal as deterministically pinned and the pin is captured + asserted. Propagation gate CM-COVENANT-114-111-PROPAGATION (literal 11.4.111). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.111.

§11.4.112 — Structural-impossibility won't-fix classification mandate (cascaded from constitution submodule §11.4.112)

Cascade reference — see constitution submodule Constitution.md §11.4.112 for the full mandate. When deep research proves a goal is structurally impossible on the target platform (forbidden by platform design / hardware-protocol constraint / documented kernel-or-API limitation), the goal MUST be classified Won't-fix, documented with impossibility evidence, and NOT re-attempted without new evidence the platform constraint changed. Propagation gate CM-COVENANT-114-112-PROPAGATION (literal 11.4.112). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.112.

§11.4.113 — Absolute no-force-push + merge-onto-latest-main mandate (cascaded from constitution submodule §11.4.113)

Cascade reference — see constitution submodule Constitution.md §11.4.113 for the full mandate. Force-push is STRICTLY FORBIDDEN with NO exception. Every push MUST follow the 6-step integration procedure: fetch all remotes, set base to latest main/master, carefully merge all changes on top, resolve every conflict, commit, push to ALL upstreams as a fast-forward. The force-push escape hatch from §11.4.41/§11.4.71 is REMOVED. Propagation gate CM-COVENANT-114-113-PROPAGATION (literal 11.4.113). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.113.

§11.4.114 — Last-known-good-tag regression isolation mandate (cascaded from constitution submodule §11.4.114)

Cascade reference — see constitution submodule Constitution.md §11.4.114 for the full mandate. When a previously-working feature is observed broken, the FIRST diagnostic action **MUST** be to identify the last release tag at which it was known-good and diff/bisect against it before any open-ended root-cause hunt or speculative fix. Default to surgical forward-fix over wholesale revert unless the operator prefers otherwise. Propagation gate CM-COVENANT-114-114-PROPAGATION (literal 11.4.114). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.114.

§11.4.115 — RED-baseline-on-the-broken-artifact + polarity-switch mandate (cascaded from constitution submodule §11.4.115)

Cascade reference — see constitution submodule Constitution.md §11.4.115 for the full mandate. Every RED test **MUST** be authored to reproduce the defect on the CURRENT pre-fix artifact. The same test source **MUST** carry a polarity switch (RED_MODE, default 1 = reproduce-and-assert-defect-present; 0 = standing GREEN regression-guard). RED-on-broken-artifact then GREEN-on-fixed-artifact on a clean target **MUST** both be captured. Propagation gate CM-COVENANT-114-115-PROPAGATION (literal 11.4.115). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.115.

§11.4.116 — Real-time conductor[?] autonomous-test-framework sync channel mandate (cascaded from constitution submodule §11.4.116)

Cascade reference — see constitution submodule Constitution.md §11.4.116 for the full mandate. Any autonomous long-running test/QA/validation framework **MUST** expose a structured append-only event stream and an atomically-rewritten status snapshot. Every verdict event **MUST** carry the evidence path that backs it. A PASS event with no evidence path is a channel-layer PASS-bluff. Propagation gate CM-COVENANT-114-116-PROPAGATION (literal 11.4.116). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.116.

§11.4.117 — Computer-vision / OCR pixel-oracle fallback for non-introspectable UIs mandate (cascaded from constitution submodule §11.4.117)

Cascade reference — see constitution submodule `Constitution.md` §11.4.117 for the full mandate. Any test that needs to drive a UI control OR assert on-screen content MUST NOT assume the accessibility/semantic/DOM hierarchy is the source of truth. When the hierarchy is blank/partial/known-unreliable, the test MUST fall back to a pixel oracle: drive input by computer-vision template-match; assert content by ROI OCR with a per-word confidence floor. The CV/OCR analyzer MUST be self-validated with golden-good/golden-bad fixtures. Propagation gate CM-COVENANT-114-117-PROPAGATION (literal 11.4.117). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.117.

§11.4.118 — Discovery-pressure to confirm known-issue-set completeness mandate (cascaded from constitution submodule §11.4.118)

Cascade reference — see constitution submodule `Constitution.md` §11.4.118 for the full mandate. A remediation/release cycle MUST NOT treat “every reported defect is fixed” as “the build is good.” After/alongside fixing the reported set, the cycle MUST run a discovery + stress pass across ALL target devices/environments, producing an enumerated list of subsystems/user-journeys/stress scenarios actually exercised with outcomes. “We found no other issues” without a coverage enumeration is a §11.4 bluff. Propagation gate CM-COVENANT-114-118-PROPAGATION (literal 11.4.118). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.118.

§11.4.119 — Single-resource-owner partitioning for parallel hardware testing mandate (cascaded from constitution submodule §11.4.119)

Cascade reference — see constitution submodule `Constitution.md` §11.4.119 for the full mandate. When multiple parallel streams exercise shared hardware or any exclusive-access resource, exactly ONE stream MUST own each such resource at a time. Every other concurrent stream targeting the same resource MUST be READ-ONLY. Parallelism is partitioned by resource; distinct devices/sinks/handles run fully concurrent. Ownership MUST be enforced by an advisory lock/token. Propagation gate CM-COVENANT-114-119-PROPAGATION (literal 11.4.119). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.119.

§11.4.120 — Fix-breaks-its-own-gate reconciliation mandate (cascaded from constitution submodule §11.4.120)

Cascade reference — see constitution submodule `Constitution.md` §11.4.120 for the full mandate. When a correct fix causes a pre-existing gate/test to FAIL because it asserted old behaviour, the gate MUST be reconciled (rewritten to assert the new mechanism + paired §1.1 mutation updated) — never fake-passed, weakened to a tautology, or deleted; and the correct fix MUST NOT be reverted to satisfy the stale gate. Propagation gate CM-COVENANT-114-120-PROPAGATION (literal 11.4.120). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.120.

§11.4.121 — No-commit-while-build-writes-tracked-artifacts mandate (cascaded from constitution submodule §11.4.121)

Cascade reference — see constitution submodule `Constitution.md` §11.4.121 for the full mandate. A commit MUST NOT run while a build/packaging/generation step is actively writing artifacts into tracked (version-controlled) directories. The commit MUST be deferred until the writing step completes so the tree is quiescent and committed artifacts are the fresh, whole outputs. Propagation gate CM-COVENANT-114-121-PROPAGATION (literal 11.4.121). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.121.

§11.4.122 — No-silent-removal-of-existing-components-without-operator-confirmation mandate (cascaded from constitution submodule §11.4.122)

Cascade reference — see constitution submodule `Constitution.md` §11.4.122 for the full mandate. No application, system component, service, package, feature, driver, module, library, or prebuilt asset — any already-existing end-user capability — may be removed WITHOUT FIRST interactively asking the operator and receiving an EXPLICIT keep-or-remove decision. A silent removal is a release blocker regardless of rationale. Propagation gate CM-COVENANT-114-122-PROPAGATION (literal 11.4.122). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.122.

§11.4.123 — Rock-solid-proof-or-deep-research mandate (cascaded from constitution submodule §11.4.123)

Cascade reference — see constitution submodule `Constitution.md` §11.4.123 for the full mandate. Every reported issue, fix, and claimed completion **MUST** be fully validated with rock-solid captured proof before closure. When the validation method is unclear, deep web research **MUST** be performed to discover or build an evidence-producing method. Declaring something untestable without exhausting the research path is a §11.4.123 violation. Propagation gate `CM-COVENANT-114-123-PROPAGATION` (literal `11.4.123`). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.123.

§11.4.124 — Dead/unwired-code investigate-before-remove mandate (cascaded from constitution submodule §11.4.124)

Cascade reference — see constitution submodule `Constitution.md` §11.4.124 for the full mandate. Before removing any seemingly-dead (zero-importer / unwired) codebase, investigate via git history where/how it was originally used and how it became dead. Removal is permitted **ONLY** with captured proof it is genuinely no longer needed — in its own separate commit. If no such proof exists, investigate where/how to wire it in properly and add missing tests. Propagation gate `CM-COVENANT-114-124-PROPAGATION` (literal `11.4.124`). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.124.

§11.4.125 — Code-review-agent gate before pre-build + main build (cascaded from constitution submodule §11.4.125)

Cascade reference — see constitution submodule `Constitution.md` §11.4.125 for the full mandate. After all fixes/changes/implementations in a batch are done, and **BEFORE** running pre-build tests and the main build, the agent **MUST** dispatch dedicated code-review agent(s) performing a multi-layer review. Any finding **MUST** be fixed, polished, improved, and covered with additional tests before the build proceeds. The review iterates until no blocking findings remain. Propagation gate `CM-COVENANT-114-125-PROPAGATION` (literal `11.4.125`). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.125.

§11.4.126 — Default autonomous-loop working mode from first prompt (cascaded from constitution submodule §11.4.126)

Cascade reference — see constitution submodule `Constitution.md` §11.4.126 for the full mandate. The endless fully-autonomous loop is the **DEFAULT** working mode, engaged automatically the moment the operator sends the **FIRST** request/prompt of a session. The loop continues until a new fully-validated-and-verified version tag is created and published across all owned submodules and the main repo,

OR the main-stream goal is fully completed with the queue empty. Stops only on explicit operator STOP, empty queue, or §12 host-safety. Mimicking this behavior or producing false results is absolutely forbidden. Propagation gate CM-COVENANT-114-126-PROPAGATION (literal 11.4.126). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.126.

§11.4.127 — Session-handoff resumption-prompt mandate (cascaded from constitution submodule §11.4.127)

Cascade reference — see constitution submodule Constitution.md §11.4.127 for the full mandate. When a fresh session is needed, the agent MUST ALWAYS prepare and proactively provide a ready-to-paste resumption prompt valid for that exact moment and project phase — self-contained enough that pasting it into a fresh session resumes work with zero loss. Both SHORT and FULL variants. A missing, stale, or generic prompt is a §11.4.127 violation. Propagation gate CM-COVENANT-114-127-PROPAGATION (literal 11.4.127). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.127.

§11.4.128 — Always-on device-recording mandate (cascaded from constitution submodule §11.4.128)

Cascade reference — see constitution submodule Constitution.md §11.4.128 for the full mandate. For every test/debug device and every device under known manual testing, across every reachable transport, the project MUST ALWAYS live-record all analysable data. Extra-careful, side-effect-free, non-invasive read-only probes only. Background + parallel + subagent-driven. Raw data is git-ignored and code-intelligence-excluded; only curated evidence is committed at release prep. Propagation gate CM-COVENANT-114-128-PROPAGATION (literal 11.4.128). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.128.

§11.4.129 — Huge-blocker release protocol (cascaded from constitution submodule §11.4.129)

Cascade reference — see constitution submodule Constitution.md §11.4.129 for the full mandate. On discovery of a huge blocker during release validation: STOP all testing; fix ALL discovered issues; process all recorded data; land rock-solid fixes; author NEW validation+verification tests of ALL supported test types; rebuild (full) + reflash to a CLEAN target; RESTART the full validation+verification from the last release tag to now on all devices in parallel, recorded, with real physical captured proofs. Propagation gate CM-COVENANT-114-129-PROPAGATION (literal 11.4.129). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.129.

§11.4.130 — Post-remediation validate-the-fix-FIRST-after-redeploy (cascaded from constitution submodule §11.4.130)

Cascade reference — see constitution submodule `Constitution.md` §11.4.130 for the full mandate. When a blocker is fixed and a new artifact produced + target reflashed, FIRST re-test the specific last-failing features and validate the just-incorporated fixes with real captured evidence — BEFORE broader/full-suite validation. Only after the targeted fix is CONFIRMED working proceed to the full retest. Propagation gate CM-COVENANT-114-130-PROPAGATION (literal 11.4.130). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.130.

§11.4.131 — Standing session-resumption file mandate (cascaded from constitution submodule §11.4.131)

Cascade reference — see constitution submodule `Constitution.md` §11.4.131 for the full mandate. Every project MUST maintain a SINGLE canonical, always-current session-resumption file at a fixed project-declared standard path. Creating a new session requires ONLY pointing the new agent at this one file. The file MUST be rewritten whenever a fresh session is needed or live state materially changes. A stale resumption file is a §11.4.131 violation. Propagation gate CM-COVENANT-114-131-PROPAGATION (literal 11.4.131). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.131.

§11.4.132 — Risk-ordered validation priority mandate (cascaded from constitution submodule §11.4.132)

Cascade reference — see constitution submodule `Constitution.md` §11.4.132 for the full mandate. Tests/validations/verifications MUST run in RISK-DESCENDING order: most-recently-worked first, then historically most-problematic, then highest crash/break/regress likelihood, then most-reopened. The highest-risk set MUST pass with real captured evidence before the remainder of the suite runs. Running in arbitrary order is a §11.4.132 violation. Propagation gate CM-COVENANT-114-132-PROPAGATION (literal 11.4.132). Non-compliance is a release blocker.

Canonical authority: constitution submodule `Constitution.md` §11.4.132.

§11.4.133 — Target-System + hardware safety mandate (cascaded from constitution submodule §11.4.133)

Cascade reference — see constitution submodule `Constitution.md` §11.4.133 for the full mandate. Every change to the target system MUST ALWAYS be safe for both the target System itself (MUST NOT brick, boot-loop, corrupt data, or render the device unrecoverable) AND the hardware it runs on (MUST NOT exceed safe

electrical/thermal/voltage/clock limits). Reversible-first; no unverified hardware-control writes; thermal/perf changes respect cooling design; flashing uses the sanctioned tool + integrity-verified image. Propagation gate CM-COVENANT-114-133-PROPAGATION (literal 11.4.133). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.133.

§11.4.134 — Code-review iterate-until-GO + rock-solid-evidence mandate (cascaded from constitution submodule §11.4.134)

Cascade reference — see constitution submodule Constitution.md §11.4.134 for the full mandate. When a code review returns ANY finding, and the author fixes/improves the batch, the code review MUST BE RE-RUN and MUST KEEP being re-run after each remediation round, until it returns a clean GO with ZERO new issues AND ZERO warnings of any kind. Every round's verdict AND every fix's validation MUST carry rock-solid physical captured evidence. A reported GO unbacked by captured physical evidence is a §11.4 PASS-bluff. Propagation gate CM-COVENANT-114-134-PROPAGATION (literal 11.4.134). Non-compliance is a release blocker.

Canonical authority: constitution submodule Constitution.md §11.4.134.

§11.4.135 — Standing regression-guard suite + every-fixed-defect-gets-a-permanent-regression-test (cascaded from constitution submodule §11.4.135).

Every project MUST maintain a standing regression-guard suite that runs on every build+deploy and blocks the release tag on any failure. Every closed defect MUST, in the SAME commit as its fix, register a permanent §11.4.115 RED-on-broken-artifact regression test into the suite. A closure without a registered guard is a §11.4.123 violation. Propagation gate CM-COVENANT-114-135-PROPAGATION (literal 11.4.135). Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.135.

§11.4.136 — Real-content end-to-end playback-test mandate (cascaded from constitution submodule §11.4.136). Any test asserting media playback works MUST drive REAL content through the user's path and assert it genuinely plays via the §11.4.107 liveness battery PLUS a decoder-health census. Metadata-only/launch-only/registration-only/single-frame/config-only PASS is forbidden. Propagation gate CM-COVENANT-114-136-PROPAGATION (literal 11.4.136). Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.136.

§11.4.137 — Subtitle/caption content-correctness oracle + secure-display-proxy-honesty mandate (cascaded from constitution submodule §11.4.137). A subtitle-correctness test MUST classify the cue's content class — a present cue is NOT a correct cue. CHROME (FAIL) if a known control/menu label, time/numeric chrome, not prose, outside the lower safe-title band, or static across the window. DIALOGUE (PASS) only when prose + not-denied + not-chrome + position-ok + cadence ≥ 2 or fuzzy-matches the source-extracted cue. The oracle MUST be self-validated with golden-good/golden-bad fixtures. Propagation gate CM-

COVENANT-114-137-PROPAGATION (literal 11.4.137). Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.137.

§11.4.138 — Operator-escape => mandatory bluff-audit + permanent guard (cascaded from constitution submodule §11.4.138). When the operator finds a defect that the GREEN test suite passed, this MUST trigger before the fix is closed: a §11.4.102 systematic-debugging pass; a bluff-audit identifying the exact assertion that should have caught it but didn't; a permanent §11.4.135 regression guard registered in the SAME commit as the fix; the bluff-audit committed under docs/research/<scope>/<defect>_bluff_audit/. Propagation gate CM-COVENANT-114-138-PROPAGATION (literal 11.4.138). Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.138.

§11.4.139 — Fresh-process clean-artifact runtime-signature mandate (cascaded from constitution submodule §11.4.139). Before any post-deploy validation, the harness MUST assert running-artifact == built-artifact: the deploy yielded a clean target OR a pre-validation check proves no stale overlay shadows the deployed code. A stale shadow makes the proxy report on code that was never deployed — any PASS is a §11.4 PASS-bluff. Propagation gate CM-COVENANT-114-139-PROPAGATION (literal 11.4.139). Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.139.

§11.4.140 & §11.4.141 — action-prefix system + token-efficiency (cascaded from constitution submodule §11.4.140/§11.4.141)

§11.4.140 — Universal action-prefix system (ACTION_NAME ::): When a user prompt's FIRST non-blank line starts with a recognised action prefix, look it up in the action registry (constitution/actions/registry.yaml), REPLACE the prefix with that action's expansion text and apply its rules. Four equivalent forms: ACTION_NAME :: <rest>, PREFIX::ACTION_NAME :: <rest>, /ACTION_NAME <rest>, /PREFIX::ACTION_NAME <rest>. An unknown token that matches the grammar shape but is NOT registered is NEVER silently expanded — ask which registered action was meant or treat as a literal prompt. The registered action BACKGROUND expands to executing the prompt in background in parallel with all main work streams using subagents-driven development with rock-solid physical evidence. Propagation gate CM-COVENANT-114-140-PROPAGATION (literal 11.4.140). Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.140.

§11.4.141 — Token-efficiency mandate (cascaded from constitution submodule §11.4.141). Every project worked on by AI coding agents MUST cut token spend toward 30–40% of current (a 60–70% reduction) WITHOUT degrading quality/performance/safety or breaking any existing mechanism, via: prompt-cache the static governance prefix; subagent model-tiering + output-to-file; thin always-loaded INDEX + on-demand detail; CodeGraph/retrieval-first; output-token reduction; tool-call batching + no re-reads; compaction/context-editing for long sessions. A mandatory measured proof harness verifies the reduction from the authoritative

usage object. Propagation gate CM-COVENANT-114-141-PROPAGATION (literal 11.4.141). Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.141.

§11.4.142 — Universal code-review mandate (cascaded from constitution submodule §11.4.142)

§11.4.142 — EVERY change made to ANY governed repository — without exception (source, fixes, tests, gates, docs, build scripts, governance files, conductor edits, sub-agent output, single-line edits) — MUST pass through an INDEPENDENT code-review step BEFORE it is accepted, committed, or built. The ABSOLUTE form of §11.4.125: no “just a doc edit” / “just a one-liner” / “self-review suffices” / “trivial change” carve-out. The reviewer MUST be structurally separated from the author (dedicated code-review agent §11.4.70/§11.4.20 or distinct human), the review is itself anti-bluff (§11.4/§11.4.1, no rubber-stamp), and iterates to a clean GO per §11.4.134. Propagation gate CM-COVENANT-114-142-PROPAGATION (literal 11.4.142) + gate CM-EVERY-CHANGE-REVIEWED. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.142.

§11.4.143 — Real-user-journey mandate for video-streaming-app tests (cascaded from constitution submodule §11.4.143)

§11.4.143 — Any full-automation test asserting a video player / streaming app plays content MUST drive the REAL end-user journey through the app’s OWN UI (launch → browse catalog → choose a SPECIFIC title → press real Play/Resume → confirm THAT content plays with correct subtitles on the intended target). A sample/demo/loop-clip, deep-link/am start/intent shortcut, or any path bypassing browse-select-play is a §11.4 PASS-bluff (validates routing, leaves “the show I picked actually plays” unproven). Chosen-content confirmed via §11.4.107 liveness on the §11.4.136 real-content path + §11.4.137 subtitle oracle; non-introspectable UIs use the §11.4.117 CV/OCR pixel oracle; genuinely infeasible ⇒ honest §11.4.52/§11.4.3 SKIP, never a faked PASS. Propagation gate CM-COVENANT-114-143-PROPAGATION (literal 11.4.143) + gate CM-VIDEO-REAL-JOURNEY-TEST. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.143.

§11.4.144 — Tracked/recorded-device availability-following mandate (cascaded from constitution submodule §11.4.144)

§11.4.144 — Every device the project tracks/records (any reachable transport — USB/wireless-ADB/SSH/serial/network) MUST be availability-FOLLOWED: connection state continuously monitored, any drop handled, never silently abandoned nor presented as continuous recording. On a drop the system MUST automatically (a) DETECT + log an honest offline event into the corpus (a silent gap

presented as continuous capture is a fabricated-continuity §11.4 bluff), (b) WAIT using the project's already-defined reconnection timings (never invented, §11.4.6), (c) RE-ATTACH + log resume on return, (d) ESCALATE to the sanctioned authorization-gated recovery path (§11.4.21/§11.4.101 — no autonomous destructive recovery) on permanent absence. The agent-side analogue is §11.4.147. Propagation gate CM-COVENANT-114-144-PROPAGATION (literal 11.4.144) + gate CM-DEVICE-AVAILABILITY-FOLLOWED. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.144.

§11.4.145 — Independent multi-angle impact-research per change (cascaded from constitution submodule §11.4.145)

§11.4.145 — For EVERY fix/change/new feature, INDEPENDENT impact-research agents (subagent-driven §11.4.70/§11.4.20, adversarial “refute-the-change” stance, separate from author) MUST research the change + its connected features across a CLOSED SET OF EIGHT ANGLES: (1) correctness/logic, (2) regression (call-graph impact, every direct+transitive caller mapped to a test), (3) latent/dangerous-code (runtime-only/interface-ABI/concurrency/resource-lifecycle), (4) security, (5) performance, (6) host/data/target-hardware safety (§12/§9/§11.4.133), (7) cross-feature interaction, (8) business-logic conformance. Each angle names tools + obtains required depth + emits captured-evidence (“looks fine” forbidden); N/A angles recorded NOT_APPLICABLE: <reason>. Output = a per-change impact-research report + single GO/NO-GO blocking acceptance on any unmitigated risk, iterating to a clean GO before the §11.4.125 review gate. Propagation gate CM-COVENANT-114-145-PROPAGATION (literal 11.4.145) + gate CM-IMPACT-RESEARCH-PER-CHANGE. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.145.

§11.4.146 — Reproduce-first + same-test-confirms-fix + extend-to-all-cases (cascaded from constitution submodule §11.4.146)

§11.4.146 — Every reported problem handled by a named three-step workflow: STEP 1 REPRODUCE-FIRST + INVESTIGATE — author the §11.4.43 RED test as a §11.4.115 RED-baseline-on-the-broken-artifact capturing defect-present evidence AND used as a §11.4.102 investigation instrument gathering forensic characterisation; STEP 2 SAME-TEST-CONFIRMS-FIX — the SAME test source (§11.4.115 polarity RED_MODE=1→0) confirms the defect ABSENT on a clean target (§11.4.108/§11.4.139), validated-first-after-redeploy (§11.4.130), no separate happy-path substitute; STEP 3 EXTEND-TO-ALL-CASES (mandatory per-fix) — fan out across the full case-space (valid+invalid+boundary+concurrent+chaos+topology per §11.4.85/§11.4.3) with enumerated provable coverage (§11.4.118), each user-visible case registered into the §11.4.135 suite. All steps anti-bluff with captured physical evidence (§11.4.123). Propagation gate CM-COVENANT-114-146-PROPAGATION (literal 11.4.146) + gate CM-REPRODUCE-FIRST-THEN-EXTEND. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.146.

§11.4.147 — Crashed-agent respawn-until-complete + no-work-loss registry (cascaded from constitution submodule §11.4.147)

§11.4.147 — Every dispatched agent/subagent MUST be tracked through its full lifecycle so a crash NEVER loses/forgets/corrupts its work (a crashed agent is NOT a completed agent; “it probably finished before dying” is a forbidden §11.4.6 guess). ALL hold: (a) durable append-only agent REGISTRY (stable id, task + §11.4.58 file-scope, output/evidence paths, status from {dispatched|in-flight|crashed|respawned|complete}, reusing the §11.4.116 sync substrate) = single source of truth for “is this work owed?”; (b) mechanical crash-detection + respawn-until-complete (any abnormal termination flips to crashed + keeps the unit OPEN, respawned with same id+scope until an agent reaches complete, transient causes use already-defined backoff, non-transient terminal errors investigated §11.4.102 first); (c) PARTIAL-STATE preserve → §11.4.84-quiescence-check → resume-or-clean-restart (§9.2 backup), per-agent git worktree isolation; (d) “crash ≠ done” — the endless-loop done-condition (§11.4.87/§11.4.94/§11.4.126) MUST NOT read satisfied while any entry is non-complete. Propagation gate CM-COVENANT-114-147-PROPAGATION (literal 11.4.147) + gate CM-CRASHED-AGENT-RESPAWN-TRACKED. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.147.

§11.4.148 — Workable-item integrity + comprehensive description + bidirectional tracker sync + BLOCKED unblock-choices (cascaded from constitution submodule §11.4.148)

§11.4.148 — Binds + strengthens the workable-item discipline (§11.4.15/.16/.54/.21/.91/.93/.95/.106) into one integrity contract spanning DB [?] docs [?] external tracker: (D1) no item without a valid status + valid type + stable id on ALL three surfaces (id is the cross-surface binding key); (D2) comprehensive structured description per item (WHAT + HOW-it-manifests + HOW-to-reproduce + ACCEPTANCE-CRITERIA, never a stub); (D3) every BLOCKED/operator-blocked item enumerates the closed list of unblock CHOICES ([A]...·[B]...·[C]... per §11.4.66); (D4) regular never-missed bidirectional DB[?]docs[?]tracker sync (git-tracked SQLite SSoT §11.4.93/.95, §11.4.86 fingerprint-gated, §11.4.106 docs_chain-bound); (D5) generic idempotent credential-redacted external-tracker push (statuses/types/assignee §11.4.104/sub-tasks, match-by-stable-key, sink-side created/updated/failed proof §11.4.69), machinery project-agnostic §11.4.28. Propagation gate CM-COVENANT-114-148-PROPAGATION (literal 11.4.148) + gates CM-ITEM-INTEGRITY-STATUS-TYPE-ID/CM-ITEM-COMPREHENSIVE-DESCRIPTION/CM-BLOCKED-UNBLOCK-CHOICES/CM-TRACKER-SYNC-IDEMPOTENT. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.148.

§11.4.149 — Per-workable-item testing-diary mandate (cascaded from constitution submodule §11.4.149)

§11.4.149 — Every workable item MUST carry an append-only TESTING DIARY (test EXECUTIONS, distinct from §11.4.93 lifecycle history + §11.4.55 reopens): (a) a `test_diary` table additive to the §11.4.93/.95 SQLite SSoT — one row per run (ISO-8601-UTC `date_time`, `tested_by` ∈ {User|Operator|AI-agent|HelixQA}, result §11.4.45, in-depth observations, action/status-change, §11.4.69 `evidence_path+feature_class`) with a SCHEMA CONSTRAINT making a PASS row without a non-empty evidence path impossible; (b) an in-depth diary doc + derived at-a-glance summary view feeding §11.4.132 risk-ordering; (c) four-format per-item Diary/Diary_Summary exports (§11.4.65) §11.4.86-fingerprinted + §11.4.106 `docs_chain`-bound; (d) external-tracker SUB-TASK model (each run a child sub-task, idempotent §11.4.148-D5 push); (e) minimal-LLM deterministic tooling (zero LLM in the data path), 100% test-covered §11.4.27. Propagation gate CM-COVENANT-114-149-PROPAGATION (literal 11.4.149) + gates CM-TEST-DIARY-SYNC/CM-DIARY-PASS-REQUIRES-EVIDENCE. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.149.

§11.4.150 — Mandatory deep multi-angle web research per change/issue (cascaded from constitution submodule §11.4.150)

§11.4.150 — For EVERY fix/improvement/change/closure — however big/small/simple/complex — the agent MUST, in addition to systematic-debugging (§11.4.102), the fix, independent review (§11.4.125/§11.4.142), and comprehensive tests (§11.4.4(b)/§11.4.40), perform a DOCUMENTED deep multi-angle web research pass (articles, official+vendor docs, API refs, standards, issue trackers, maintainer guidance, open-source code) to BOTH (i) discover the best solution AND (ii) confirm there is NOT a more serious underlying problem. ALL hold: (A) no closure-as-fixed/structural without a documented deep-research pass (makes §11.4.8 unconditional); (B) ≥2 genuinely-distinct angles (no single-source confirmation-bias miss); (C) confirm-no-bigger-problem (enumerated §11.4.118); (D) latest-source + cited (§11.4.99 URLs + access date in artefact + commit footer); (E) reopen-breaking is the purpose (§11.4.34/.55); (F) ALWAYS in parallel with the main stream (§11.4.70/.103/.89, never serialising); (G) apply ASAP to every workable item across the §11.4.93/.95 SSoT. Propagation gate CM-COVENANT-114-150-PROPAGATION (literal 11.4.150) + gate CM-DEEP-RESEARCH-PER-ISSUE. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.150.

§11.4.151 — Project-prefixed release-tag/version-naming mandate (cascaded from constitution submodule §11.4.151)

§11.4.151 — Every release tag AND version name on the main repo AND every owned submodule (§11.4.28) MUST be prefixed with the project's release prefix, form <PREFIX>-<version> (e.g. helix_translate-1.0.0-dev-0.0.1), so a release is greppable across every repository (`git tag -l '<PREFIX>-*'`). Prefix resolution (closed-set, deterministic §11.4.6): (1) HELIX_RELEASE_PREFIX from `.env` (`git-ignored` §11.4.30, documented in `.env.example` §11.4.77) else (2) lowercased snake_case project-root dir name (§11.4.29). The prefix MUST be IDENTICAL across the main repo and all owned submodules within one release; version codes increment monotonically. The tag is created only after the §11.4.40 GREEN retest and reaches every upstream via §11.4.113 merge-onto-latest-main (never a force-push), fanned out per §2.1. Propagation gate CM-COVENANT-114-151-PROPAGATION (literal 11.4.151) + gate CM-RELEASE-PREFIX-NAMING. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.151.

§11.4.152 — Crashlytics-recorded-data continuous monitoring + systematic-debug + regression-test-coverage (cascaded from constitution submodule §11.4.152)

§11.4.152 — Every project with Firebase Crashlytics enabled/wired MUST treat the Crashlytics console as a first-class captured-evidence channel from real end-user devices and continuously process every datum (an open Crashlytics issue with a green suite is a §11.4 field-telemetry PASS-bluff). Strengthens+completes §11.4.47 (which surfaces findings) with the fix lifecycle. ALL hold: (1) continuous monitoring of ALL four surfaces — fatal crashes, ANRs, performance traces, AND non-fatals (the silent class — every catch/fallback path is a real degraded UX); (2) systematic-debugging reproduce-before-fix (§11.4.102/§11.4.115, the recorded stacktrace IS the captured defect evidence); (3) a fix for every confirmed issue against its proven root cause (§11.4.9/.43/.108; “acknowledged in console” is not a fix); (4) a permanent §11.4.135 regression guard per closed issue (§11.4.115 polarity test) in the same commit + a closure log citing the console issue id/URL; (5) regular cadence (§11.4.47 five-trigger set); (6) no false results, no bluff. Propagation gate CM-COVENANT-114-152-PROPAGATION (literal 11.4.152) + gate CM-CRASHLYTICS-ISSUE-FULLY-COVERED. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.152.

§11.4.153 — Comprehensive per-feature Status + Status_Summary set with mandatory video confirmation (cascaded from constitution submodule §11.4.153)

§11.4.153 — Every project MUST maintain under docs/features/ a comprehensive feature Status set (Status.md + its §11.4.56 Status_Summary.md) enumerating EVERY component, EVERY client app/binary/surface (TUI/CLI/Web/desktop/mobile/API/gRPC/library/submodule/infra), and EVERY feature (incl. ported from incorporated CLI-agent/submodule catalogues §11.4.74) with NO feature left out, reconciled vs the codebase (§11.4.6/.118). Per-feature fields: Component/Feature/Category/Implementation/Wiring(§11.4.108)/Real-use/Tests-coverage(§11.4.4(b))/Validation(PASS/FAIL/SKIP/PENDING_FORENSICS/OPERATOR-BLOCKED §11.4.45)/Video-confirmation. Every user-visible “confirmed” claim is backed by a recorded real-use video (real prompts → real LLM/service responses → real results), NEVER a frozen frame (§11.4.107), NEVER a faked/bluff response (§11.4.2/.5); autonomous-infeasible ⇒ honest §11.4.3/.52 SKIP. Video-analysis remediation loop §11.4.102/.146/.134; always-in-sync §11.4.45/.106 docs_chain + §11.4.86 fingerprint; four-format export HTML+PDF+DOCX (adds DOCX to §11.4.65 for this class). Propagation gate CM-COVENANT-114-153-PROPAGATION (literal 11.4.153) + gates CM-FEATURE-STATUS-COMPLETE/CM-FEATURE-STATUS-VIDEO-CONFIRMED. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.153.

§11.4.154 — Window-scoped capture + fresh-corpus rotation for feature/QA recordings (cascaded from constitution submodule §11.4.154)

§11.4.154 — Refines §11.4.2/.5/.107/.153 recording discipline with two capture-hygiene invariants. (A) Window-scoped, NOT whole-screen — every feature/QA video captures ONLY the window/surface under test (GUI window / CLI-TUI pane / web tab-viewport / emulator frame), NEVER the whole desktop or unrelated windows (whole-desktop leaks operator-private content §11.4.10/.83, dilutes the §11.4.107 freeze oracle, breaks the §11.4.137 OCR/ROI oracle); target by stable identity (window id/title, device serial, browser context, tmux target) per §11.4.111; cannot capture below whole-screen ⇒ honest §11.4.3 SKIP. (B) Fresh-corpus rotation — when a new in-scope run begins, the agent’s OWN prior in-scope stale recordings at the raw path are removed FIRST (§11.4.107 not-stale + §11.4.86 freshness); honest boundary (§11.4.6/§9.2) — own SAME-scope recordings ONLY, never another project’s/operator-authored files (uncertain ⇒ surface, §11.4.122); committed docs/qa/<run-id>/ (§11.4.83) is durable, not rotated. Propagation gate CM-COVENANT-114-154-PROPAGATION (literal 11.4.154) + gate CM-WINDOW-SCOPED-FRESH-CORPUS-RECORDING. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.154.

§11.4.155 — Project-name-prefixed feature/QA recording filenames (cascaded from constitution submodule §11.4.155)

§11.4.155 — Every recorded video the project produces — every feature/QA real-use recording (§11.4.153), every window-scoped capture (§11.4.154), every always-on device recording (§11.4.128), every raw/curated artefact + the committed docs/qa/<run-id>/ trail (§11.4.83) — MUST have a filename STARTING WITH the PROJECT-NAME prefix, ALWAYS (an unprefixed recording makes a multi-project corpus un-greppable/un-attributable — the §11.4.151 failure on the recording-corpus axis). Prefix resolution is the closed-set IDENTICAL to §11.4.151: (1) HELIX_RELEASE_PREFIX from .env else (2) lowercased snake_case project-root dir name (§11.4.29); canonical form <PREFIX>---<feature-or-scope>---<run-id>.<ext>; MUST equal the §11.4.151-resolved release prefix (one project, one name). Honest boundary §11.4.6 — guarantees attribution+greppability, not content validity (§11.4.107/.137/.153), and does not relax §11.4.154 window-scope/rotation. Propagation gate CM-COVENANT-114-155-PROPAGATION (literal 11.4.155) + gate CM-RECORDING-PROJECT-NAME-PREFIX. Non-compliance is a release blocker.
Canonical authority: constitution submodule Constitution.md §11.4.155.

§11.4.156 — All CI/CD automation MUST be disabled (cascaded from constitution submodule §11.4.156)

§11.4.156 — Every governed repository (main repo, this constitution submodule, every owned+nested submodule we author and push) MUST ship with ALL server-side CI/CD automation DISABLED: no push to any owned upstream may trigger a GitHub Actions run, GitLab pipeline, or equivalent (Jenkins/CircleCI/Travis/Drone/Woodpecker/Bitbucket/Azure). Generalises + makes ABSOLUTE the §11.4.75 Layer-5 posture across all governed repos; enforcement migrates to the LOCAL §11.4.75 git-hook ritual + §11.4.40 pre-tag sweep. ALL hold: (A) zero active .github/workflows/*.yaml|yml / .gitlab-ci.yml / equivalent at the ROOT of any authored repo; (B) “disabled” = a push triggers ZERO runs (delete OR rename to a non-trigger name; a live-on:+if:false workflow is NOT compliant); (C) scope = repos we author+push (vendored/third-party nested configs below the root are INERT, out of scope, §11.4.29 vendor-exempt); (D) no new CI may be added; (E) pre-push verify + §11.4.109-class guard enforce mechanically. Honest boundary §11.4.6 — file-level disabling does not reach provider-side server settings (operator turns those off). Propagation gate CM-COVENANT-114-156-PROPAGATION (literal 11.4.156) + gate CM-NO-ACTIVE-CI. Non-compliance is a release blocker.
Canonical authority: constitution submodule Constitution.md §11.4.156.

§11.4.157 — GEMINI.md maintained in lockstep with CLAUDE.md / AGENTS.md / QWEN.md (cascaded from constitution submodule §11.4.157)

§11.4.157 — GEMINI.md is a FIRST-CLASS governance context carrier EQUAL to CLAUDE.md/AGENTS.md/QWEN.md, never optional/best-effort (forensic FACT 2026-06-15: GEMINI.md silently drifted to §11.4.141 while the others carried §11.4.155 — a §11.4 propagation-bluff, a Gemini-CLI agent reading a stale Constitution). ALL hold: (A) five-carrier lockstep — no governance change is complete until GEMINI.md carries it alongside the other three mirrors; GEMINI.md is added to the §11.4.26 propagation + cross-reference set explicitly; (B) no silent drift — GEMINI.md lagging the other mirrors' highest rule is a §11.4.157 violation (§11.4.65-class), back-fill required; (C) equal status — GEMINI.md restates the SAME literal 11.4.N anchors the propagation gates require (§11.4.35), the fleet count INCLUDES GEMINI.md; (D) consumer projects' own CLAUDE/AGENTS/QWEN/GEMINI bind too (§11.4.35). Propagation gate CM-COVENANT-114-157-PROPAGATION (literal 11.4.157, GEMINI.md INCLUDED) + gate CM-GEMINI-MD-LOCKSTEP. Non-compliance is a release blocker. **Canonical authority:** constitution submodule Constitution.md §11.4.157.

§11.4.158 — Intensive all-feature/flow/edge-case video-recording + read-the-screen content-verification mandate (User mandate, 2026-06-16). Every project MUST be covered by intensive automated testing that exercises + RECORDS every feature/flow/use-case/edge-case (valid/invalid/boundary/concurrent/failure-injection §11.4.85), each recording showing the feature GENUINELY WORKING with REAL results (real prompts→real responses→real outputs §11.4.153) and NO false/simulated/stale/frozen result (§11.4.2/.5/.107) — a recording showing an error/bluff/non-working feature is a FINDING (→§11.4.153(4)/§11.4.4 fix→retest→re-record), never a confirmation. The testing System MUST ACTUALLY READ every shown log line / message / UI label / dialog / toast / status text and VERIFY it is a genuine working result (OCR/ROI §11.4.117/.137 + confidence floor for pixel surfaces; direct capture for terminal/log; §11.4.107(10) self-validated golden-good/golden-bad analyzer) — “a video was produced” is NOT evidence, “the System read the screen + confirmed a genuine result” is. HelixQA (§11.4.27) MUST drive this exercise→record→read→score pass, PASS only on a read-confirmed genuine result + captured artefact path (§11.4.69). Default recording save path = \$HOME/Downloads (host user's home Downloads, resolved at runtime never hardcoded) unless a project declares an override per §11.4.35; §11.4.155 project-prefix + §11.4.154 window-scope/rotation + §11.4.128 git-ignored raw corpus + §11.4.83 curated docs/qa evidence apply. Composes §11.4.2/.5/.25/.27/.52/.69/.83/.85/.107/.108/.117/.118/.128/.137/.138/.153/.154/.155/§1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-158-PROPAGATION (literal 11.4.158) + recommended gates CM-INTENSIVE-RECORDING-COVERAGE + CM-RECORDING-CONTENT-READ-VERIFIED + paired §1.1 mutation. **Canonical authority:** constitution submodule Constitution.md §11.4.158. Non-compliance is a release blocker. No escape hatch — no --skip-recording-coverage, --video-without-content-read, --happy-path-recording-suffices, --recording-path-anywhere, --unread-recording-OK, --skip-helixqa-read flag. **Project instantiation (§11.4.35):** helix_code overrides the default save path to /Volumes/T7/Downloads/Recordings (recordings are NOT moved to \$HOME/Downloads).

§11.4.142 — Universal code-review mandate — every change reviewed, always, no exception (User mandate, 2026-06-09). Verbatim operator mandate: “ALL changes we do MUST pass through the code review step!!! ALWAYS!!!” EVERY change made to ANY repository this Constitution governs — without exception — MUST pass through an INDEPENDENT code-review step BEFORE it is accepted, committed, or built. NO change class is exempt: source code, fixes, tests, gates, meta-test mutations, documentation, doc-tooling, build/CI scripts, configuration, governance files (Constitution / CLAUDE / AGENTS / QWEN), conductor main-stream edits, sub-agent output, refactors, single-line edits — if a diff exists, it gets an independent review. This is the ABSOLUTE form of §11.4.125 (code-review agent gate after a batch, before pre-build sweep + main build): §11.4.142 strips every implicit scoping §11.4.125’s “after all fixes/changes/implementations are done” phrasing could leave open — no “just a doc edit”, no “just a one-liner”, no “the author already self-reviewed (§11.4.92)”, no “trivial change” carve-out.

Independence is load-bearing — the reviewer MUST be structurally separated from the author (a dedicated code-review agent, subagent-driven by default per §11.4.70 / §11.4.20, or a distinct human), NEVER the author re-reading their own work; §11.4.92’s multi-pass self-evaluation is the AUTHOR-side discipline and PRECEDES (never satisfies) §11.4.142. **The review is itself anti-bluff** (§11.4 / §11.4.1) — a rubber-stamp “looks good” is not a review; it MUST genuinely analyse correctness, safety (no host §12 / data §9 / target-hardware §11.4.133 regression), will-it-really-work (no solve-A-create-B), end-user behaviour (§11.4 / §107), and test-genuineness (§1.1), its conclusions captured evidence per §11.4.5 / §11.4.69, and **it iterates to a clean GO per §11.4.134** — any finding (BLOCKING / nit / warning) re-arms the loop, acceptance only on ZERO new findings + ZERO warnings with rock-solid physical evidence. Honest boundary (§11.4.6): a passing review does NOT replace §11.4.108 four-layer runtime-signature verification nor the §11.4.40 full-suite retest — it is one of MULTIPLE STRONG LAYERS, and the FIRST one every change crosses. Composes §11.4.1 / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.20 / §11.4.40 / §11.4.69 / §11.4.70 / §11.4.92 / §11.4.108 / §11.4.110 / §11.4.125 / §11.4.134 / §107 / §1.1. Classification: universal (§11.4.17) — the consuming project supplies its reviewer-dispatch mechanism + change-acceptance seam (commit wrapper / merge queue / PR gate) per §11.4.35. Propagation gate CM-COVENANT-114-142-PROPAGATION (literal 11.4.142) + recommended gate CM-EVERY-CHANGE-REVIEWED (every accepted change carries a fresh independent-review-completed marker for its diff, produced by a reviewer distinct from the author, before acceptance/commit/build) + paired §1.1 mutation (strip the literal → propagation gate FAILs; accept a diff with no independent-review marker → CM-EVERY-CHANGE-REVIEWED FAILs). **Canonical authority:** constitution submodule Constitution.md §11.4.142. Non-compliance is a release blocker. No escape hatch — no --skip-review, --no-review, --trivial-change-exempt, --doc-edit-exempt, --self-review-suffices, --review-after-commit flag.

§11.4.143 — Real-user-journey mandate for video-streaming-app full-automation tests (User mandate, 2026-06-10). Verbatim operator mandate: “All video streaming apps ... require to choose some title and to press proper UI button to start or resume playing! Proper UI interaction to play exact show with proper content and subtitles is MANDATORY! Without it we just eventually play on 2nd display sample, and that’s it mostly! THIS MUST BE ADDED as MANDATORY RULE regarding testing of any video streaming app in general with full automation tests! Universal in root constitution + ATMOSphere project extensions.” Any full-automation test that asserts a video player / streaming application plays content MUST drive the REAL end-user journey through the app’s OWN UI — launch →

BROWSE the actual catalog → choose a SPECIFIC title → press the real Play/Resume button → confirm THAT chosen content is genuinely playing, with its correct subtitles, on the intended routing target. A test that bypasses the journey with a sample / demo / built-in-loop clip, a deep-link / `am start -a VIEW` / intent shortcut, a synthetic or pre-staged stream, or any path that does NOT exercise the app's own browse-select-play UI is a §11.4 PASS-bluff at the user-journey layer: it validates ROUTING (that *something* reaches the display) while leaving the user-visible behaviour the operator cares about — “the show I picked actually plays” — unproven (the operator's “we just eventually play on 2nd display sample, and that's it mostly” gap). The mandate (ALL hold): (1) **Real journey, not a shortcut** — launch → catalog browse → specific-title selection → real Play/Resume press through the app's own UI (§11.4.48 UI-driven), NEVER a deep-link / intent / sample / loop-clip shortcut; (2) **Chosen-content confirmation** — the PASS proves the SPECIFIC selected title is playing (not merely that pixels move on the target) via the §11.4.107 liveness battery on the §11.4.136 real-content path + the §11.4.137 subtitle content-correctness oracle for the chosen title's captions; (3) **Non-introspectable UIs use the pixel oracle** — when the app's accessibility hierarchy is blank/unreliable (TV-Compose / leanback / canvas / GL), DRIVE input + ASSERT content via the §11.4.117 CV/OCR pixel oracle, never a hierarchy-only tool; (4) **Login via the credential single-source** (§11.4.10), never hardcoded, never logged; (5) **Honest SKIP, never a faked PASS** — where the autonomous journey is genuinely infeasible (hard human-only login / CAPTCHA, geo-block per §11.4.3, secure-surface pixel-blanking per §11.4.112) the test is `operator_attended SKIP-with-reason` per §11.4.52 + §11.4.3 with a tracked migration item — NEVER a metadata-only / sample-played / routing-only PASS. Honest boundary (§11.4.6): “the routing fired so the title is playing” is a guess — only the chosen-content liveness + subtitle oracle on the real journey proves it. Composes §11.4.48 / §11.4.107 / §11.4.117 / §11.4.136 / §11.4.137 / §11.4.52 / §11.4.3 / §107 / §1.1. Classification: universal (§11.4.17) — the consuming project supplies its concrete app roster, login-credential source, routing target, and UI-driving / pixel-oracle harness per §11.4.35. Propagation gate CM-COVENANT-114-143-PROPAGATION (literal 11.4.143) + recommended gate CM-VIDEO-REAL-JOURNEY-TEST (every video-streaming-app playback test drives the real browse-select-play journey + confirms the chosen content + subtitles, or SKIPS-with-reason) + paired §1.1 mutation (replace a real-journey test's browse-select-play path with a deep-link / sample shortcut → gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule Constitution.md §11.4.143. Non-compliance is a release blocker. No escape hatch — no `--sample-playback-ok`, `--skip-real-journey`, `--deep-link-suffices`, `--routing-only-pass`, `--no-title-selection` flag.

§11.4.144 — Tracked/recorded-device availability-following mandate (User mandate, 2026-06-10). Direct operator mandate: every device the project is tracking / following / recording (test / debug / manual-testing device, across every reachable transport — USB / wireless ADB / SSH / serial / network introspection API) MUST be availability-FOLLOWED — its connection state continuously monitored, any drop handled, never silently abandoned and never presented as a continuous recording. The §11.4.128 always-on recorder KNOWS a tracked device is absent (its per-device loop guards on the reachable state) but, lacking a following discipline, merely spins idle — no data captured, no offline event logged, no resume, no escalation — so the recording corpus presents a continuous timeline with a silent hole, a §11.4 PASS-bluff at the recording-integrity layer (it claims continuous capture while no data exists for the gap). On a tracked device leaving its reachable state the system MUST, automatically: (a) **DETECT** the drop and **log an**

honest offline event into the recording corpus (§11.4.6 — a silent gap presented as continuous capture is a fabricated-continuity bluff; §11.4.128 — a silent recording gap defeats always-on recording); (b) **WAIT** for the device to return using the project's ALREADY-DEFINED reconnection timings — never invented numbers (§11.4.6), the SAME grace / reconnect / poll budgets the project's recovery path already uses; (c) **RE-ATTACH** — resume recording / tracking the moment the device returns and log an honest online / resume event; (d) **ESCALATE** to the project's sanctioned device-recovery path (per §11.4.69 feature class `device_recovery`) if the device does not return within the defined timeout — through the sanctioned, authorization-gated recovery entry point **ONLY**, never bypassing its gate and never performing a destructive recovery (e.g. a power-cycle) autonomously without that authorization (§11.4.21 / §11.4.101 — high-blast-radius recovery is gated; while blocked the system keeps following the device, logging the blocked-escalation honestly). A tracked-device drop that produces a silent corpus hole, a never-resumed recording, or a never-escalated permanent absence is the bluff this anchor forbids. Composes §11.4.128 (always-on recording — §11.4.144 closes its drop-handling gap) / §11.4.69 (`device_recovery` sink-side positive evidence) / §11.4.6 (honest offline / online events, reused-not-invented timings) / §11.4.14 (watchdog children reaped on stop) / §11.4.21 + §11.4.101 (gated, non-autonomous escalation). Classification: universal (§11.4.17) — the consuming project supplies its concrete tracking transport, device set, already-defined reconnection timings, and sanctioned recovery entry point per §11.4.35. Propagation gate **CM-COVENANT-114-144-PROPAGATION** (literal 11.4.144) + recommended gate **CM-DEVICE-AVAILABILITY-FOLLOWED** + paired §1.1 meta-test mutation (strip the watchdog wiring / let an absence go unlogged → gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule Constitution.md §11.4.144. Non-compliance is a release blocker. No escape hatch — no `--skip-availability-following`, `--silent-recording-gap-OK`, `--no-reconnect-wait`, `--invent-reconnect-timing`, `--skip-recovery-escalation`, `--autonomous-power-cycle-OK` flag.

§11.4.145 — Independent multi-angle impact-research per change (User mandate, 2026-06-10). For EVERY fix / change / new feature, **INDEPENDENT** impact-research agents (subagent-driven §11.4.70/§11.4.20, structurally separate from the author — §11.4.92 self-eval PRECEDES, never satisfies — adversarial “refute-the-change” stance to defeat the documented LLM confirmation-bias failure mode) **MUST** research the change **AND** its connected/dependent features across a **CLOSED SET OF EIGHT ANGLES** — (1) correctness/logic, (2) regression (what existing working feature could break — via call-graph impact enumerating every direct + transitive caller and contract-dependent feature, each mapped to its test), (3) latent/dangerous-code — the recents class (runtime-only failure, interface/ABI/contract mismatch, concurrency/data-race, resource lifecycle), (4) security (taint/injection/secret/unsafe-API), (5) performance (latency/memory/thermal under load vs baseline), (6) host/data/target-hardware safety per §12/§9/§11.4.133, (7) cross-feature interaction (shared state/timing/hardware contention), (8) business-logic conformance (matches the spec **AND** the connected features' contracts, not merely compiles). Forensic anchor (FACT, project-internal): the recents / 3-button-nav path shipped a latent AIDL-interface-contract mismatch that **PASSED** code review yet failed at **RUNTIME ONLY** (ANR on a recents-reaping gesture) because no review angle interrogated the interface contract / concurrency-lifecycle / silently-broken connected feature — the §11.4.108 **SOURCE→ARTIFACT→RUNTIME→USER-VISIBLE** gap caught only after the fact; §11.4.145 shifts that discovery **LEFT**. Each angle **MUST** name the **TOOL(S)** used + obtain the required **DEPTH** (the diff, the full changed unit, its declared contract, the spec section, every connected feature —

NEVER the diff lines alone) + emit a captured-evidence conclusion per §11.4.5/§11.4.69 (“looks fine” forbidden, §11.4.1); a genuinely-N/A angle is recorded NOT_APPLICABLE: <reason> per §11.4.6, never silently skipped. Output = a per-change impact-research REPORT (one section per angle: tool + evidence path + verdict) + a single GO/NO-GO that BLOCKS acceptance/commit/build on ANY unmitigated risk in ANY angle; the change is fixed/mitigated + affected angles re-researched, iterating to a CLEAN GO (zero unmitigated risk + zero warning) per §11.4.134 before the §11.4.125 review gate. Honest boundary (§11.4.6): a GO proves cross-angle internal-consistency + bounded blast-radius — it does NOT replace §11.4.108 runtime-signature verification nor §11.4.40 full-suite retest; it is one of MULTIPLE STRONG LAYERS, the FIRST research pass every change crosses. Composes/strengthens §11.4.92/.125/.108/.110/.134/.78/.79/.107/.85/.133/§12/§9/.99/.6/§1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-145-PROPAGATION (literal 11.4.145) + recommended gate CM-IMPACT-RESEARCH-PER-CHANGE + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; accept a change with a report missing an angle or an unmitigated NO-GO → CM-IMPACT-RESEARCH-PER-CHANGE FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule Constitution.md §11.4.145. Non-compliance is a release blocker. No escape hatch — no --skip-impact-research, --single-angle-suffices, --author-researches-own-change, --diff-skim-OK, --no-go-overridable, --trivial-change-no-research flag.

§11.4.146 — Reproduce-first test + same-test-confirms-fix + mandatory extend-to-all-cases workflow (User mandate, 2026-06-10). Every reported problem MUST be handled by a NAMED three-step test workflow — §11.4.146 does NOT re-author its component disciplines, it NAMES + BINDS them into the operator’s exact per-defect sequence and adds ONLY the two emphases the components leave implicit. **(STEP 1 — REPRODUCE-FIRST + INVESTIGATE)** before any fix, author the §11.4.43 RED test as a §11.4.115 RED-baseline-on-the-broken-artifact (reproduce the defect on the CURRENT pre-fix artifact, capture defect-present physical evidence per §11.4.5/§11.4.69/§11.4.107); NEW EMPHASIS (D1) that same RED test is ALSO a deliberate investigation instrument per §11.4.102(A) — it MUST gather ADDITIONAL forensic data characterising the defect (triggers, boundaries, input/topology scope, adjacent failure modes) that feeds BOTH the fix design AND the STEP-3 extend scope; a RED test used only as a binary present/absent gate with no characterisation satisfies §11.4.115 but NOT §11.4.146 STEP 1. **(STEP 2 — SAME-TEST-CONFIRMS-FIX)** after the fix the SAME test source (the §11.4.115 polarity switch flipped RED_MODE=1→0) confirms the defect ABSENT — RED-on-broken then GREEN-on-fixed, both captured, on a clean target per §11.4.108/§11.4.139, validated-first-after-redeploy per §11.4.130, deterministic per §11.4.50; no separate happy-path test substitutes (the §11.4.43/§11.4.115 PASS-bluff). **(STEP 3 — EXTEND-TO-ALL-CASES, mandatory per-fix)** NEW EMPHASIS (D2) immediately after STEP 2 the test MUST be FANNED OUT across the full case-space of the SAME functionality — all flows, valid + invalid + boundary (§11.4.85 empty/max/off-by-one) + concurrent (§11.4.85 contention) + failure-injection (§11.4.85 chaos) + topology variants (§11.4.3) — confirming no issue of any kind, with PROVABLE enumerated coverage per §11.4.118 (a listed case-set with per-case outcome, never “no other issues found”); a REQUIRED step, NOT deferred to a release-cycle discovery sweep and NOT reduced to a single guard; each user-visible case carries rock-solid physical evidence per §11.4.123 + is registered into the §11.4.135 standing regression-guard suite; a newly-discovered fan-out defect triggers §11.4.4 test-interrupt + re-enters STEP 1. (ANTI-BLUFF, all steps) every PASS is rock-solid CAPTURED physical evidence per §11.4.123

(§11.4.5/§11.4.69/§11.4.107) — metadata-only / config-only / absence-of-error / grep-without-runtime PASS forbidden (§11.4/§11.4.1); unclear validation method ⇒ deep-research-before-declaring-untestable per §11.4.123/§11.4.8/§11.4.99; an operator-found defect the green suite missed triggers §11.4.138. Honest boundary (§11.4.6): STEP 3 reduces the unknown-unknown surface but does not prove zero remaining defects (§11.4.118) — un-exercised cases stated as honest gaps, never silently implied clean. Composes §11.4.43/.115/.102/.130/.108/.139/.50/.85/.118/.135/.3/.123/.5/.69/.107/.138/.4/§107/§1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-146-PROPAGATION (literal 11.4.146) + recommended gate CM-REPRODUCE-FIRST-THEN-EXTEND (every closed defect's fix carries a §11.4.115 reproduce-first polarity test with captured defect-characterisation [STEP 1] + its RED_MODE=0 GREEN confirmation [STEP 2] + an enumerated per-functionality extend case-set registered into the §11.4.135 suite [STEP 3]; a closure with the reproduce→confirm pair but NO enumerated extend case-set FAILs) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; close a defect with only the reproduce→confirm pair and no extend-case-set → CM-REPRODUCE-FIRST-THEN-EXTEND FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule Constitution.md §11.4.146. Non-compliance is a release blocker. No escape hatch — no --skip-reproduce-first, --fix-without-red, --skip-extend-to-all-cases, --reproduce-confirm-suffices, --defer-extend-to-release-sweep, --single-guard-suffices, --no-defect-characterisation flag.

§11.4.147 — Crashed-agent respawn-until-complete + no-work-loss registry mandate (User mandate, 2026-06-10). Verbatim operator intent: “any agent that crashed because of something MUST BE respawned and finish its work at some point! We MUST NOT lose any work, forget about or have it corrupted!” Forensic case study (FACT): dispatched subagents died on TRANSIENT causes (API Error: Server is temporarily limiting requests rate-limit killed 5 at once, API Error: socket connection closed unexpectedly killed 2, one left PARTIAL edits — two source files written, the dependent SystemUI sliders + doc + test NOT done), each re-dispatched by pure conductor vigilance with NO mechanical guarantee — the moment conductor context is lost / compacted / the conductor itself crashes, an in-flight-but-dead work unit is silently dropped (lost / forgotten / corrupted). Every dispatched agent/subagent MUST be tracked through its full lifecycle so a crash NEVER loses, forgets, or corrupts its work; a crashed agent is NOT a completed agent (§11.4.6 — “it probably finished before dying” is a forbidden guess; abnormal termination is positive evidence the unit is OPEN). The mandate (ALL hold): **(a) durable agent REGISTRY** — every dispatched agent has an append-only machine-readable registry entry (stable id, task + §11.4.58 file-scope, declared output path(s) + §11.4.5/§11.4.69 evidence dir, dispatch timestamp, status from the CLOSED SET {dispatched | in-flight | crashed | respawned | complete}), reusing the §11.4.116 sync substrate (JSONL event stream + atomic status snapshot; “an entry with no dispatch-event cannot show complete”); the registry is the SINGLE SOURCE OF TRUTH for “is this work owed?” and an unregistered dispatch is itself a violation; **(b) mechanical CRASH-DETECTION + RESPAWN-until-complete** — any abnormal termination (transient rate-limit/socket-close, any non-completion exit, or a terminal error after the runtime's own retries are exhausted) flips the entry to crashed + keeps the unit OPEN, and the unit is respawned (fresh agent claims the same id + scope + preserved state) and re-respawned until an agent reaches complete; respawn is the safe/reversible/bounded decision taken autonomously per §11.4.101 (never blocks the loop for a human), transient causes use the project's ALREADY-DEFINED

backoff budgets (never invented, §11.4.6), a deterministically-reproducible non-transient terminal error is investigated per §11.4.102 before further respawn (never a blind retry loop); **(c) PARTIAL-STATE preserve → §11.4.84-check → resume-or-clean-restart** — the crashed agent’s uncommitted edits + output doc are PRESERVED (never silently discarded → lost, never blindly committed → corruption), the respawn runs the §11.4.84 quiescence check on the preserved tree (every modified file accounted-for vs scope, no mutation// always pass/_mutated_* residue, no half-written/torn artifact), then EITHER RESUMES idempotently when it passes OR CLEAN-RESTARTS from a known-good base (§9.2 pre-op backup, reversible §11.4.101) when inconsistent — nothing lost, nothing corrupted; per-agent git worktree isolation (§11.4.58 L4 / §11.4.84) keeps the partial tree from contaminating other streams; **(d) “crash ≠ done” COMPLETION criterion** — a unit is DONE only when an agent reaches complete with its required captured evidence/output landed (§11.4.5/§11.4.69 + the §11.4.116 verdict-carries-evidence-path rule); the endless-loop done-condition (§11.4.87/§11.4.94/§11.4.97/§11.4.126) MUST NOT read satisfied while any entry is dispatched/in-flight/crashed/respawned-not-yet-complete, the zero-idle survey (§11.4.94) treats every non-complete entry as an OPEN item to reclaim, and a registry showing complete without landed evidence is a §11.4 PASS-bluff at the agent-lifecycle layer. Honest boundary (§11.4.6): the registry + respawn guarantee work is not lost/corrupted, NOT that it is correct — the respawned output still crosses §11.4.108 + §11.4.125/§11.4.142 review + §11.4.40 retest; §11.4.147 is the durability layer beneath those, the agent-side analogue of §11.4.144 (same detect→wait/backoff→re-attach/respawn→escalate shape). Composes §11.4.6/.58/.84/.87/.94/.97/.101/.116/.102/.108/.125/.142/.40/.128/.144/§9.2/§1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-147-PROPAGATION (literal 11.4.147) + recommended gate CM-CRASHED-AGENT-RESPAWN-TRACKED + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; mark a crashed entry as the loop’s done-condition complete without landed evidence, OR drop a dispatched agent without a registry entry → CM-CRASHED-AGENT-RESPAWN-TRACKED FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule Constitution.md §11.4.147. Non-compliance is a release blocker. No escape hatch — no --skip-agent-registry, --crash-equals-done, --no-respawn, --discard-partial-state, --blind-commit-partial, --forget-dead-agent, --loop-done-with-crashed-entries flag.

§11.4.148 — Workable-item integrity (status+type+id) + comprehensive structured description + bidirectional external-tracker sync + BLOCKED unblock-choices mandate (User mandate, 2026-06-10). BINDS + STRENGTHENS the workable-item discipline (§11.4.15 status / §11.4.16 type / §11.4.54 id / §11.4.21 Operator-blocked / §11.4.91 description-clarity / §11.4.93 + §11.4.95 SQLite SSoT / §11.4.106 docs_chain) into ONE integrity contract spanning DB [?] docs [?] external tracker, adding the operator’s three emphases: **(D1) no item without a valid status + valid type + stable id — on ALL three surfaces** (an item missing any of the three in the DB, the rendered docs, OR the external tracker FAILs the validator — release-blocker; the id is the cross-surface binding key); **(D2) comprehensive structured description per item** — WHAT it is (§11.4.91 ≥6-word/≥40-char clear meaning) + HOW it manifests + HOW to reproduce (§11.4.115/.146) + ACCEPTANCE CRITERIA (§11.4.123/.69 captured-evidence verdict), in the §11.4.93 DB description column AND the docs AND the tracker, never a stub/§11.4.91-anti-pattern fragment; **(D3) BLOCKED items carry WHY + enumerated unblock CHOICES** — tightens §11.4.21: every Operator-blocked item (incl. Blocked/BLOCKED documented alias normalised to canonical Operator-

blocked, never a silent fork §11.4.6) MUST enumerate the closed list of decisions/actions that would unblock it ([A]... [B]... [C]..., mirroring §11.4.66's 2–4-option shape); a BLOCKED item with no enumerated choices FAILs; **(D4) regular never-missed bidirectional DB[?]docs[?]tracker sync** — the §11.4.93/.95 git-tracked SQLite DB is the SSoT, docs + tracker DERIVED; full sync (db-to-md / md-to-db byte-identical round-trip §11.4.93 + tracker push) runs regularly + on every change, §11.4.86 drift-proof fingerprint (sha256 of the sorted item keyset, NOT mtime) gates freshness, §11.4.106 docs_chain-bound so drift is mechanically caught not vigilance-dependent; **(D5) generic external-tracker push** carries statuses (collapsed onto the tracker's native set per §11.4.33/.112 when fixed, precise value preserved in a header, never lost), types, assignee (§11.4.104 participant handle, UNSET defaults from a project env var, never hardcoded/logged §11.4.10), and sub-tasks, IDEMPOTENT (dry-run-then-real, match-by-stable-key [<ID>] prefix / id custom-field, present⇒UPDATE/absent⇒CREATE, rate-limited, credential-redacted, sink-side created=N updated=M failed=0 proof §11.4.69), the MACHINERY project-agnostic §11.4.28 (consumer registers its tracker/list/field-map at runtime). Anti-bluff §11.4: every sync + validator pass carries captured evidence §11.4.5/.69; honest boundary §11.4.6 — guarantees well-formed items + agreeing surfaces, NOT that the underlying work is correct (still crosses §11.4.108/.40/.123). Composes §11.4.15/.16/.21/.33/.34/.54/.66/.86/.91/.93/.95/.104/.106/.112/.123/.10/.28/.69/.6/ §1.1. Classification: universal (§11.4.17) — consumer supplies its DB path, id prefix, tracker service + list/board id + field map + default-assignee env var, docs_chain context per §11.4.35. Propagation gate CM-COVENANT-114-148-PROPAGATION (literal 11.4.148) + recommended gates CM-ITEM-INTEGRITY-STATUS-TYPE-ID / CM-ITEM-COMPREHENSIVE-DESCRIPTION / CM-BLOCKED-UNBLOCK-CHOICES / CM-TRACKER-SYNC-IDEMPOTENT + paired §1.1 mutation (gate-code = separate work item). **Canonical authority:** constitution submodule Constitution.md §11.4.148. Non-compliance is a release blocker. No escape hatch — no --item-without-status, --item-without-type, --item-without-id, --stub-description-OK, --blocked-without-choices, --skip-tracker-sync, --one-way-sync-OK, --non-idempotent-tracker-push, --hardcode-assignee flag.

§11.4.149 — Per-workable-item testing-diary mandate (User mandate, 2026-06-10). Every workable item MUST carry an append-only TESTING DIARY — a chronological record of every test run against it — distinct from §11.4.93 item_history (lifecycle STATE transitions) + §11.4.55 Reopens (reopen cycles): the diary records TEST EXECUTIONS (which may or may not change status). The mandate (ALL hold): **(a)** a test_diary table additive to the §11.4.93/.95 SQLite SSoT, one row per run soft-keyed to the item id, capturing ISO-8601-UTC date_time + tested_by (closed set User|Operator|AI-agent|HelixQA) + result (§11.4.45 PASS|FAIL|SKIP + detail) + in-depth observations (long markdown, facts or explicit UNCONFIRMED:/PENDING_FORENSICS: §11.4.6, the background a future fix needs) + action_taken+status_changed+from/to (did this run change status + WHY) + §11.4.69 evidence_path+feature_class, with a SCHEMA CONSTRAINT making a PASS row WITHOUT a non-empty evidence path impossible (a PASS-bluff rejected by the schema itself); **(b)** BOTH an in-depth diary doc + a derived at-a-glance summary VIEW (total/pass/fail/skip + last-verdict + last-run + status-changes + distinct testers/feature-classes, DERIVED never duplicated §11.4.93) feeding §11.4.132 risk-ordering; **(c)** four-format per-item Diary/Diary_Summary exports §11.4.65 + §11.4.86-drift-proof-fingerprinted (sha256 of the sorted diary keyset) + §11.4.106 docs_chain-bound so a diary row not exported/pushed FAILs a gate (never-missed); **(d)** external-tracker SUB-TASK

model — the item task’s description stays the item (§11.4.148 D2, NOT polluted with diary text), each run a CHILD sub-task (type task) with a {TODO|In-progress|Completed} lifecycle (collapsed per §11.4.33/.112), observations + action + an Evidence: line (path not raw artefact §11.4.10/.13) + a diary-entry idempotency key, reusing the §11.4.148 D5 idempotent rate-limited credential-redacted sink-side-proven push, mapper project-agnostic §11.4.28; (e) MINIMAL-LLM deterministic bash/Go tooling (zero LLM in the data path — the observations prose is authored by whoever ran the test; tooling only stores/renders/pushes/validates), 100% test-covered §11.4.27 (unit + integration-against-the-real-tracker with an honest §11.4.3 SKIP-when-token-absent never a faked PASS + export + HelixQA Challenge + paired §1.1 mutation) so a diary PASS-bluff is mechanically impossible. Honest boundary §11.4.6 — the diary guarantees a complete auditable per-item test-history, NOT that any single run’s verdict is correct (rests on its own §11.4.69/.107/.123 evidence). Composes §11.4.6/.27/.45/.50/.55/.65/.69/.86/.93/.95/.106/.107/.123/.132/.148/.10/.13/.28/.33/.112/§1.1. Classification: universal (§11.4.17) — consumer supplies its DB path, diary doc layout, export formats, tracker sub-task field map, docs_chain context per §11.4.35. Propagation gate CM-COVENANT-114-149-PROPAGATION (literal 11.4.149) + recommended gates CM-TEST-DIARY-SYNC / CM-DIARY-PASS-REQUIRES-EVIDENCE + paired §1.1 mutation (gate-code = separate work item). **Canonical authority:** constitution submodule Constitution.md §11.4.149. Non-compliance is a release blocker. No escape hatch — no --skip-testing-diary, --diary-without-evidence, --no-diary-summary, --diary-without-tracker-subtask, --llm-in-diary-data-path, --diary-export-optional flag.

§11.4.150 — Mandatory deep multi-angle web research per change/issue, before declaring fixed or structural (User mandate, 2026-06-11). Verbatim operator mandate: “For every single issue we fix or improvement we make — besides tight systematic-debugging, fixing, code review by independent agents, comprehensive tests — we MUST ALWAYS do deep web research from various angles! No matter how big/small/simple/complex, dig deep the internet for articles, technical documentation, APIs, open-source code — EVERYTHING that can help make the best possible solution OR confirm we don’t have a more serious problem we’re unaware of! We MUST do everything possible to STOP the constant issue-reopening and finally start closing items as fixed+working! Do this ALWAYS in parallel with the main work stream!” For EVERY fix / improvement / change / closure — no matter how big, small, simple, or complex — the agent MUST, IN ADDITION to tight systematic-debugging (§11.4.102), the fix, independent-agent code review (§11.4.125 / §11.4.142), and comprehensive multi-layer tests (§11.4.4(b) / §11.4.40), perform a DOCUMENTED **deep multi-angle web research pass** digging the internet from VARIOUS angles — articles, official + vendor technical documentation, API references, standards, issue trackers, maintainer guidance, reusable open-source code — to BOTH (i) discover the best possible solution AND (ii) confirm there is NOT a more serious underlying problem the team is unaware of. Forensic case study (FACT, 2026-06-11): a subtitle-on-secondary-display goal verdicted Won't-fix: structurally-impossible (§11.4.112 secure-surface pixel-blanking) was OVERTURNED by a deep multi-angle research pass that surfaced the real OCR-of-the-PRIMARY-display path — a “we can’t” became a shipped capability; the canonical anti-pattern of a structural verdict reached for want of research. The mandate (ALL hold): **(A) No closure-as-fixed/structural WITHOUT a documented deep-research pass** — no item may be marked Fixed/Implemented/Completed (§11.4.33) OR classified structurally-impossible won't-fix (§11.4.112) until a deep multi-angle pass is documented; §11.4.150 makes

§11.4.8's pass UNCONDITIONAL (every item, however trivial, at the closure AND structural-verdict gates). **(B) Multiple angles, not a single lookup** — ≥ 2 genuinely-distinct angles (official-docs / standards / known-bug-trackers / alternative-approach / failure-mode / security / performance / platform-constraint / open-source-precedent) so a single-source confirmation-bias miss (§11.4.145) cannot pass; a one-link drive-by is NOT deep multi-angle. **(C) Confirm-no-bigger-problem, not just find-a-fix** — explicitly seek evidence the fix does not mask a deeper defect AND the closure/verdict is genuinely safe; “we found nothing worse” requires the enumerated search (§11.4.118). **(D) Latest-source + cited** — LATEST authoritative versions per §11.4.99 (never training-data/memory), each cited by URL + access date in the research artefact AND the closure commit footer (Deep-research <date>: <urls> OR the literal NO external solution found – original work per §11.4.8). **(E) Reopen-breaking is the PURPOSE** — STOP the constant Fixed[?]Reopened churn (§11.4.34 / §11.4.55); a reopen whose root cause a deep pass would have surfaced is a §11.4.150 miss; composes §11.4.7 (demotion-evidence) + §11.4.112 (structural verdict now REQUIRES the cited-authorities pass). **(F) ALWAYS in parallel with the main stream** — background subagent-driven (§11.4.70 / §11.4.20 / §11.4.103 / §11.4.89) concurrent with main fix/build/test work, NEVER serialising it or stalling the loop (§11.4.94 / §11.4.97 / §11.4.101 / §11.4.126). **(G) Apply ASAP to every workable item** — retroactively + going forward across the §11.4.93 / §11.4.95 SSoT; items closed without a documented pass are re-audited in the §11.4.40 / §11.4.42 release-gate sweep. Honest boundary (§11.4.6): the pass reduces the unknown-unknown surface + breaks the most common reopen causes — it does NOT prove zero remaining defects (§11.4.118) and does NOT replace §11.4.108 runtime-signature verification, §11.4.125 / §11.4.142 review, or §11.4.40 retest; it is the research layer every fix/closure/structural-verdict additionally crosses, and “we probably don't have a bigger problem” without the enumerated multi-angle search is a guess (§11.4.6), never a finding. Composes §11.4.8 / §11.4.99 / §11.4.123 / §11.4.118 / §11.4.145 / §11.4.125 / §11.4.142 / §11.4.7 / §11.4.112 / §11.4.34 / §11.4.55 / §11.4.70 / §11.4.20 / §11.4.89 / §11.4.103 / §11.4.40 / §11.4.42 / §11.4.93 / §11.4.95 / §11.4.6 / §1.1. Classification: universal (§11.4.17) — the consuming project supplies its concrete research corpora, angle set, item tracker, and closure-commit-footer convention per §11.4.35. Propagation gate CM-COVENANT-114-150-PROPAGATION (literal 11.4.150) + recommended gate CM-DEEP-RESEARCH-PER-ISSUE (every closed/structural-verdicted item carries a documented multi-angle deep-research artefact + cited-source closure footer, run in parallel, before the closure/structural verdict is accepted) + paired §1.1 mutation (strip the literal → propagation gate FAILs; close an item or reach a structural verdict with no documented multi-angle research artefact / cited footer → CM-DEEP-RESEARCH-PER-ISSUE FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule Constitution.md §11.4.150. Non-compliance is a release blocker. No escape hatch — no --skip-deep-research, --single-source-suffices, --trivial-change-no-research, --close-without-research, --structural-without-research, --serialise-research, --research-from-memory-OK flag.

§11.4.151 — Project-prefixed release-tag/version-naming mandate (User mandate, 2026-06-12). Verbatim operator mandate: “Every release tag and version name we create — on the main repository and on every Submodule we own — MUST be prefixed with the project's release prefix, e.g. myproject-1.0.0-dev-0.0.1. The prefix MUST come from HELIX_RELEASE_PREFIX in our .env if it is set, otherwise from the lowercased project root directory name. The SAME prefix MUST be used across the main repo and all owned Submodules in one release so a release is greppable across every repository.” Every release tag AND every version

name created on the main repository AND on every owned-by-us submodule (§11.4.28) MUST be prefixed with the project's release prefix, form <PREFIX>-<version> (canonical example: myproject-1.0.0-dev-0.0.1), so a release is identifiable + greppable across every repository it spans (one `git tag -l '<PREFIX>-*'` enumerates the whole release surface). **Prefix resolution order (closed-set, deterministic — §11.4.6 no-guessing):** (1) `HELIX_RELEASE_PREFIX` from the project's `.env` — authoritative when set; `.env` is git-ignored per §11.4.30 and the variable is documented in the tracked `.env.example` (a §11.4.77 re-obtain mechanism, never committed); (2) fallback = the lowercased snake_case form of the project root directory name (no spaces) per §11.4.29 — used whenever the `env` var is unset/empty, so a prefix is ALWAYS resolvable from the checkout with zero operator input. **The prefix MUST be IDENTICAL across the main repo and all owned submodules within a single release** — a release that tags the main repo <PREFIX>-1.2.0 while tagging an owned submodule with a different/unprefixed value is a §11.4.151 violation (the cross-repo grep no longer enumerates the release). Version codes increment monotonically within the prefix (<PREFIX>-...-0.0.1 → <PREFIX>-...-0.0.2 → ...), never reset, never skipped. Honest boundary (§11.4.6): the prefix guarantees a release is identifiable + uniform across every repository, NOT that its contents are correct — the tag is still created only after the §11.4.40 full-suite retest GREEN and reaches every upstream via the §11.4.113 merge-onto-latest-main path (NEVER a force-push), fanned out per §2.1. Composes §2.1 / §11.4.29 / §11.4.30 / §11.4.40 / §11.4.113 / §11.4.126 (the release-scope terminal condition is a published, prefixed tag). Classification: universal (§11.4.17) — the consuming project supplies its concrete prefix value + the `HELIX_RELEASE_PREFIX` env var per §11.4.35. Propagation gate CM-COVENANT-114-151-PROPAGATION (literal 11.4.151) + recommended gate CM-RELEASE-PREFIX-NAMING (every release tag/version on the main repo + every owned submodule carries the resolved <PREFIX>- prefix, identical across the release) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; create an unprefixed or differing-prefix release tag → CM-RELEASE-PREFIX-NAMING FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule Constitution.md §11.4.151. Non-compliance is a release blocker. No escape hatch — no `--no-release-prefix`, `--unprefixed-tag`, `--prefix-optional`, `--differing-submodule-prefix` flag.

§11.4.152 — Crashlytics-recorded-data continuous monitoring + systematic-debug + regression-test-coverage mandate (User mandate, 2026-06-13).

Verbatim operator intent: “For every project that has Firebase Crashlytics enabled / wired, we MUST continuously monitor ALL of the Crashlytics-recorded data — crashes, ANRs, performance traces, and non-fatals — systematically debug each, fix and improve, and cover everything with validation and verification tests. This MUST be checked regularly, with no false results and no bluff of any kind!” Every project that has Firebase Crashlytics enabled/wired (SDK linked into a shipping artifact, crash+non-fatal+ANR reporting active) MUST treat the Crashlytics console as a first-class captured-evidence channel from real end-user devices and continuously process every datum it records — an open Crashlytics issue with a green test suite is a §11.4 PASS-bluff at the field-telemetry layer (a real user hitting a broken feature while the suite reports green). STRENGTHENS+COMPLETES §11.4.47: §11.4.47 owns the periodic REVIEW + dedup + Issue-creation pass that SURFACES Crashlytics/Analytics/Performance findings; §11.4.152 owns what happens to each surfaced item AFTER — the systematic-debug → fix/improve → regression-test-coverage lifecycle that drives it to a proven regression-immune closure (§11.4.47 finds it; §11.4.152 fixes it + proves it stays fixed; both mandatory, neither substitutes). The mandate (ALL hold): (1) **continuous monitoring of ALL**

four surfaces — fatal crashes, ANRs, performance traces/regressions, AND non-fatals (skipping any one is a PASS-bluff; non-fatals are the silent class — every catch/fallback/recovered-error path is a real degraded UX and MUST be triaged, not ignored because the app did not crash); (2) **systematic-debugging of each (reproduce-before-fix)** per §11.4.102 Iron Law + §11.4.115 RED-baseline-on-the-broken-artifact (the recorded stacktrace/ANR-trace/non-fatal-context IS the §11.4.5/.69 captured defect-present evidence) BEFORE the fix — a fix without a prior reproducing test is a §11.4.43/.123 violation; unreproducible ⇒ deep-research-before-declaring-untestable §11.4.123/.150; (3) **a fix/improvement for every confirmed issue** against its proven root cause (§11.4.9/.43/.108), “acknowledged in console” is not a fix, muting a recurring issue without a tracked rationale is a §11.4.6/.90 violation; (4) **validation+verification regression-test coverage per closed issue** — in the SAME commit as the fix register a permanent §11.4.135 regression guard (§11.4.115 polarity test: RED_MODE=1 captures the recorded defect on the pre-fix artifact, RED_MODE=0 standing GREEN guard asserting ABSENT) exercising the same user-reachable path the stacktrace identifies, with rock-solid captured evidence §11.4.5/.69/.107/.123 + a closure log citing the console issue id/URL + root-cause + fix commit + the validation/verification test paths; a Crashlytics issue marked resolved WITHOUT a falsifiable regression test is FORBIDDEN (the silent-recurrence vector, §11.4.138 operator-escape class applied to field telemetry); (5) **regular cadence** reusing the §11.4.47 five-trigger set (pre-build/pre-flash/pre-distribute/pre-tag blocking; daily + post-deployment-burn-in non-blocking) — a one-time sweep never repeated is a violation; (6) **no false results, no bluff** (§11.4.1/.6 — “no new issues” requires the enumerated monitored-surfaces+window §11.4.118; “fixed” requires the RED→GREEN flip §11.4.115/.130; a guard whose §1.1 mutation does not FAIL it is a bluff gate). Honest boundary §11.4.6: processing every recorded issue breaks the silent-recurrence vector — it does NOT prove zero remaining field defects nor replace §11.4.108/.40; an issue is “closed” only when its guard is GREEN on a clean artifact, never when the console mark is flipped. Classification: universal (§11.4.17) — the consuming project supplies its Crashlytics handle, console-access credential (§11.4.10, never logged), severity table, and per-issue closure-log path per §11.4.35; the reference project-level instantiation is the Lava client’s §6.O (Crashlytics-Resolved Issue Coverage — per-issue validation test + Challenge test + .lava-ci-evidence/crashlytics-resolved/<date>-<slug>.md closure log) + §6.AC (Comprehensive Non-Fatal Telemetry — every catch/fallback records a non-fatal with triage context). Composes §11.4.1/.5/.6/.9/.34/.40/.43/.47/.69/.90/.102/.107/.108/.115/.118/.123/.130/.135/.138/.150/§1.1. Propagation gate CM-COVENANT-114-152-PROPAGATION (literal 11.4.152) + recommended gate CM-CRASHLYTICS-ISSUE-FULLY-COVERED (every closed Crashlytics issue carries a systematic-debug root-cause record + a registered §11.4.135 regression guard + a closure-log entry citing the console issue id/URL + the validation/verification test paths; a console-resolved issue with no falsifiable regression guard FAILs) + paired §1.1 meta-test mutation (gate-code = separate work item). **Canonical authority:** constitution submodule Constitution.md §11.4.152. Non-compliance is a release blocker. No escape hatch — no --skip-crashlytics-monitoring, --monitor-crashes-only, --skip-non-fatals, --resolve-without-regression-test, --console-mark-is-fixed, --monitor-once, --mute-without-rationale flag.

§11.4.153 — Comprehensive per-feature Status + Status_Summary document set with mandatory video-recording confirmation (User mandate, 2026-06-15). Every project MUST maintain, under docs/features/, a comprehensive feature Status document set (Status.md + its §11.4.56 Status_Summary.md companion)

enumerating EVERY system component, EVERY client app/binary/surface (TUI / CLI / Web / desktop / mobile / API / gRPC / library / submodule / infrastructure), and EVERY feature — including features ported from any incorporated CLI-agent / submodule catalogue (§11.4.74) — with NO single feature left out. (1) **Total categorized coverage** — per-feature table organized Component→Category, reconciled against the actual codebase (a code-present feature missing from the table, or a row with no code, is a §11.4.6/§11.4.118 bluff). (2) **Per-feature fields** — Component / Feature / Category / Implementation / Wiring (genuinely end-user-reachable, not merely compiled, §11.4.108) / Real-use / Tests-coverage (four-layer §11.4.4(b)) / Validation (PASS / FAIL / SKIP / PENDING_FORENSICS / OPERATOR-BLOCKED per §11.4.45) / **Video-recording confirmation** (path to the real-use video or honest gap marker). (3) **Mandatory per-feature real-use video** — every user-visible “confirmed” claim backed by a recorded real-use video of a genuine end-user scenario (real prompts → real LLM/service responses → real results), NEVER a frozen/stale frame (§11.4.107), NEVER faked/mock/demo-loop/bluff response or LLM error passed off as success (§11.4.2/§11.4.5), stored at the project-declared recording path (§11.4.35); a confirmed row with no real video or a bluff video = §11.4 PASS-bluff; autonomous-infeasible ⇒ honest §11.4.3/§11.4.52 SKIP + tracked migration item, NEVER a faked confirmation. (4) **Video-analysis remediation loop** — every video analysed; any defect it surfaces triggers §11.4.102 systematic-debug → fix → §11.4.146 retest → re-record → clean GO per §11.4.134 before “confirmed” (a video exposing a broken feature is a §11.4.4 test-interrupt). (5) **Always-in-sync** — §11.4.45-class roster/corpus-backed, §11.4.106 docs_chain-bound + §11.4.86 drift-proof fingerprint (sha256 of sorted feature-key roster AND sorted video-artefact roster, NOT mtime), re-syncs out-of-the-box; stale = violation. (6) **Four-format export** — HTML + PDF + **DOCX** (this doc class ADDS DOCX to the §11.4.65 HTML+PDF set; other classes unchanged), in sync per §11.4.60. (7) Follows §11.4.44/.45/.56/.57/.59/.60. (8) **MP4 format REQUIRED**. All video confirmations MUST be in .mp4 format (H.264). Window-specific capture ONLY (§11.4.159(A)). Vision validation REQUIRED (§11.4.159(D)). .cast files are supplementary only. Honest boundary §11.4.6 — guarantees a complete video-confirmed always-synced ledger, NOT per-feature correctness (rests on each row’s §11.4.69/.107/.123 evidence) and NOT a §11.4.40 retest substitute. Classification: universal (§11.4.17). Composes §11.4.2/.5/.44/.45/.52/.56/.57/.59/.60/.65/.86/.102/.106/.107/.108/.118/.123/.134/.146 /§1.1. Propagation gate CM-COVENANT-114-153-PROPAGATION (literal 11.4.153) + recommended gates CM-FEATURE-STATUS-COMPLETE + CM-FEATURE-STATUS-VIDEO-CONFIRMED + paired §1.1 mutation.

Canonical authority: constitution submodule Constitution.md §11.4.153. Non-compliance is a release blocker. No escape hatch — no --skip-feature-status, --feature-without-video, --frozen-video-OK, --bluff-response-video-OK, --skip-video-analysis, --feature-ledger-incomplete-OK, --no-docx-export, --allow-feature-ledger-drift flag.

§11.4.154 — Window-scoped capture + fresh-corpus rotation for feature/QA recordings (User mandate, 2026-06-15). Verbatim: “recording you perform does record the window containing apps and services, not the whole desktop or monitor screen!” + “All old recording files MUST BE removed when new one starts!” Refines §11.4.2/.5/.107/.153 recording discipline with two capture-hygiene invariants. (A) **Window-scoped, NOT whole-screen** — every feature/QA video MUST capture ONLY the window/surface of the app/service under test (GUI window / CLI-TUI terminal pane / web tab-viewport / device-emulator-simulator frame), NEVER the whole desktop/monitor or unrelated windows; whole-desktop capture leaks operator-private content (§11.4.10/.83), dilutes the §11.4.107 liveness/

freeze oracle, and breaks the §11.4.137 OCR/ROI content oracle. Target the window/region by stable identity (window id/title, device serial, browser context, tmux target) per §11.4.111 — never a fixed full-screen device index. Platform genuinely cannot capture below whole-screen ⇒ honest §11.4.3 SKIP + tracked migration item, never a whole-screen pass-off. **(B) Fresh-corpus rotation** — when a new recording run for a scope begins, the agent's OWN prior in-scope stale recordings at the raw recording path MUST be removed FIRST so the live corpus reflects the current run (§11.4.107 not-stale + §11.4.86 roster-freshness). Honest boundary (§11.4.6 + §9.2): “remove old” = the agent's own prior recordings for the SAME scope/project ONLY — NEVER another project's/scope's/operator-authored files; uncertain ⇒ surface, don't delete (§11.4.122); committed docs/qa/<run-id>/ evidence (§11.4.83) is the durable record, NOT rotated. Classification: universal (§11.4.17). Composes §11.4.2/.5/.10/.83/.86/.107/.111/.122/.128/.137/.153/§9.2/.6. Propagation gate CM-COVENANT-114-154-PROPAGATION (literal 11.4.154) + recommended gate CM-WINDOW-SCOPED-FRESH-CORPUS-RECORDING + paired §1.1 mutation.

Canonical authority: constitution submodule Constitution.md §11.4.154. Non-compliance is a release blocker. No escape hatch — no --whole-screen-capture-OK, --skip-window-scope, --keep-stale-recordings, --no-corpus-rotation, --full-desktop-recording flag. **(C) MP4 auto-conversion REQUIRED.** Any .cast file produced MUST be auto-converted to .mp4 via agg + ffmpeg immediately after capture. The .mp4 is the primary evidence; .cast is supplementary only.

§11.4.155 — Project-name-prefixed feature/QA recording filenames (User mandate, 2026-06-15). Verbatim: “All recorded videos MUST START with prefix: the PROJECT NAME (ALWAYS USE THE PROJECT NAME). Project name MUST be obtained according to the constitution's own project-name resolution.” Every recorded video the project produces — every feature/QA real-use recording (§11.4.153), every window-scoped capture (§11.4.154), every always-on device recording (§11.4.128), every raw or curated artefact at the project-declared recording path (§11.4.35) + the committed docs/qa/<run-id>/ trail (§11.4.83) — MUST have a filename that STARTS WITH the PROJECT-NAME prefix, ALWAYS; an unprefixed recording is a §11.4.155 violation (a multi-project/scope corpus on one host per §11.4.128/.103 becomes un-greppable + un-attributable — the §11.4.151 identify-and-grep failure on the recording-corpus axis). **Prefix resolution (closed-set, deterministic — §11.4.6, IDENTICAL to §11.4.151):** (1) HELIX_RELEASE_PREFIX from .env (authoritative, git-ignored §11.4.30, documented in tracked .env.example §11.4.77) else (2) lowercased snake_case project-root dir name §11.4.29 — ALWAYS resolvable, zero operator input. SAME prefix for EVERY recording in a checkout so ls '<PREFIX>---'* enumerates the corpus; canonical form <PREFIX>---<feature-or-scope>---<run-id>.<ext> (--- delimits the prefix unambiguously); MUST equal the §11.4.151-resolved release-tag prefix (divergence is itself a §11.4.155 violation — one project, one name). Honest boundary (§11.4.6): the prefix guarantees attribution + greppability, NOT content validity (still §11.4.107/.137/.153) and does NOT relax §11.4.154's window-scope/rotation (rotation removes the agent's OWN <PREFIX>---* only; foreign/operator files surfaced not deleted §11.4.122/§9.2). Classification: universal (§11.4.17). Composes §11.4.151/.128/.153/.154/.111/.83/.6/.29/.30/.35/.77/.86/§1.1. Propagation gate CM-COVENANT-114-155-PROPAGATION (literal 11.4.155) + recommended gate CM-RECORDING-PROJECT-NAME-PREFIX + paired §1.1 mutation.

Canonical authority: constitution submodule Constitution.md §11.4.155. Non-compliance is a release blocker. No escape hatch — no `--no-recording-prefix`, `--recording-without-project-name`, `--unprefixed-recording`, `--prefix-optional-for-recording`, `--differing-recording-prefix` flag.

§11.4.156 — All CI/CD automation (GitHub Actions / GitLab pipelines / equivalents) MUST be disabled (User mandate, 2026-06-15). Verbatim operator mandate: “Any GitHub actions or GitLab pipelines MUST BE disabled! Add this critical mandatory rule / mandatory constraint into the root constitution Submodule, commit and fetch all its changes to all upstreams and make sure we respect and follow this rule (we do apply it) ASAP!!!” Every repository this Constitution governs — main repo, this constitution submodule, every owned + nested submodule we author and push — MUST ship with ALL server-side CI/CD automation DISABLED: no push to any owned upstream may trigger a GitHub Actions run, GitLab pipeline, or equivalent (Jenkins/CircleCI/Travis/Drone/Woodpecker/Bitbucket/Azure, any on: `push/schedule/workflow_dispatch`). GENERALISES + makes ABSOLUTE the §11.4.75 Layer-5 posture (remote CI DISABLED, workflow preserved at a `...disabled-local-only non-.yaml` name a provider ignores) across ALL governed repos; enforcement migrates to the LOCAL §11.4.75 git-hook ritual + §11.4.40 pre-tag sweep, never a remote runner. ALL hold: **(A)** zero active `.github/workflows/*.yaml|yml / .gitlab-ci.yml / .gitlab/** /` equivalent at the ROOT of any governed repo/submodule (the only place a provider executes); **(B)** “disabled” = a push triggers ZERO runs — delete OR rename to a non-trigger name (§11.4.75 `.disabled/.disabled-local-only`); a live-on: `+if:false` workflow still queues runs, NOT compliant; **(C)** scope = repos we author+push — vendored/third-party nested configs below the root (AOSP `external/**`, `prebuilts/**`, vendored submodules) are INERT (a provider never runs a non-root config), OUT of scope, MUST NOT be mass-edited (§11.4.29 vendor-exempt); test = “does a push to OUR upstream trigger a run?” `yes⇒disable`, `inert⇒document+leave` (§11.4.6 verify-not-assume); **(D)** no new CI may be added (release blocker); **(E)** pre-push verify `git ls-files | grep -E '^\.github/workflows/.*\.ya?ml$|^\.gitlab-ci\.yaml$'` empty for authored repos, §11.4.109-class PreToolUse guard + gate enforce mechanically. Honest boundary (§11.4.6): file-level disabling stops FILE-triggered runs, NOT provider-side server settings (org-default required workflows, branch-protection required checks, provider scheduled exports) — the operator turns those off; the agent documents what it cannot reach, never claims unachieved completeness. Composes §11.4.75/.29/.6/.40/.42/.109/.113/§2.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-156-PROPAGATION (literal 11.4.156) + recommended gate CM-NO-ACTIVE-CI + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; add a root `.github/workflows/x.yaml` to an authored repo → CM-NO-ACTIVE-CI FAILs). **Canonical authority:** constitution submodule Constitution.md §11.4.156. Non-compliance is a release blocker. No escape hatch — no `--allow-ci`, `--enable-workflow`, `--keep-pipeline`, `--remote-ci-OK`, `--ci-exempt` flag.

§11.4.157 — GEMINI.md maintained in lockstep with CLAUDE.md / AGENTS.md / QWEN.md (User mandate, 2026-06-15). Verbatim operator mandate: “Make sure with CLAUDE.md, AGENTS.md, QWEN.md we maintain GEMINI.md too! Add this mandatory fact / rule to root constitution Submodule we are inheriting / extending - CONSTITUTION.md, CLAUDE.md, QWEN.md, AGENTS.md, GEMINI.md and other related relevant files!” Forensic FACT (2026-06-15): when §11.4.156/§11.4.157 were authored, Constitution/CLAUDE/AGENTS/QWEN carried the family through §11.4.155 but GEMINI.md had silently drifted to §11.4.141 — 14 mandates (§11.4.142–155) never propagated — a §11.4

propagation-bluff (a Gemini-CLI agent reads a stale Constitution). GEMINI.md is a FIRST-CLASS governance context carrier EQUAL to CLAUDE.md/AGENTS.md/QWEN.md, never optional/best-effort. ALL hold: **(A)** five-carrier lockstep — no governance change is complete until GEMINI.md carries it alongside the other three mirrors; GEMINI.md is added to the §11.4.26 propagation + cross-reference set explicitly; **(B)** no silent drift — GEMINI.md lagging the other mirrors' highest rule is a §11.4.157 violation (§11.4.65-class), back-fill required; **(C)** equal status — GEMINI.md restates the SAME literal 11.4.N anchors the propagation gates require (§11.4.35), fleet count INCLUDES GEMINI.md; **(D)** consumer projects' own CLAUDE/AGENTS/QWEN/GEMINI bind too (§11.4.35). Honest boundary (§11.4.6): the §11.4.142–155 GEMINI.md back-fill is a tracked release-blocking remediation; claiming GEMINI.md “in sync” while the back-fill is incomplete is itself a §11.4.157 violation. Composes §11.4.26/.35/.17/.44/.65/.140/.156/§1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-157-PROPAGATION (literal 11.4.157, GEMINI.md INCLUDED) + recommended gate CM-GEMINI-MD-LOCKSTEP + paired §1.1 meta-test mutation. **Canonical authority:** constitution submodule Constitution.md §11.4.157. Non-compliance is a release blocker. No escape hatch — no --skip-gemini-md, --gemini-optional, --gemini-lag-OK, --four-carrier-suffices flag.

§11.4.159 — Mandatory window-specific video recording + vision validation mandate (User mandate, 2026-06-20). Every feature test, validation, verification, challenge, and QA session that produces video evidence MUST comply with ALL of the following: **(A) Window-specific recording ONLY** — every video MUST record ONLY the target application window (Terminal pane, TUI app, browser tab, emulator frame), NEVER the whole desktop/monitor screen; use macOS screencapture -l<window_id>, Linux xdotool+ffmpeg, or equivalent; whole-desktop capture leaks operator-private content (§11.4.10), dilutes the §11.4.107 liveness oracle, and breaks §11.4.137 OCR/ROI; platform genuinely cannot capture below whole-screen => honest §11.4.3 SKIP + tracked migration item. **(B) MP4 format REQUIRED** — .mp4 (H.264, movflags +faststart, pix_fmt yuv420p); .cast files are supplementary only; auto-convert via agg+ffmpeg per §11.4.154(C). **(C) Project-name prefix REQUIRED** — filename MUST start with project name in snake_case from HELIX_RELEASE_PREFIX in .env (§11.4.151) or lowercased project root dir name (§11.4.29); format <project_name>-<feature>-<timestamp>.mp4; unprefixed = violation. **(D) Mandatory vision validation** — after EVERY recording, the agent MUST read the terminal output / video content and verify: (i) feature ACTUALLY WORKS, (ii) LLM responses are REAL, (iii) all tests show PASS, (iv) no “TODO implement” / “simulate” / “for now” patterns, (v) output demonstrates end-user working feature; MUST produce a verdict (PASS/FAIL) with evidence path (§11.4.69). **(E) Terminal window cleanup** — after each recording, dismiss/close ONLY the Terminal window used for that recording (window-specific close via osascript/xdotool); MUST NOT close windows belonging to other processes. **(F) Real results ONLY** — every recording MUST show REAL working features; errors/empty output/simulated responses trigger fix→retest→re-record. **(G) Re-runnable evidence** — the command shown MUST be re-runnable to produce the same results. **(H) Fresh-corpus rotation** — remove agent's own prior in-scope stale recordings FIRST (§11.4.154). **(I) Content verification MANDATORY — not duration-based** — the value of a recording is NOT its duration but its CONTENT; a recording MUST demonstrate the ACTUAL feature being used with REAL results; before accepting ANY recording, the agent MUST verify: expected output patterns ARE present (e.g., test PASS lines, API response data, feature-specific output), the feature ACTUALLY WORKS as demonstrated (not just “something ran”), LLM responses are REAL content (not

simulated, not placeholder, not empty), every claim of “working” is backed by visible evidence; a 5-second recording showing a feature working correctly is MORE valuable than a 60-second recording of empty terminal; duration is NOT a proxy for quality. **(J) Expected-content specification REQUIRED** — before recording, the agent MUST specify what content SHOULD appear (expected patterns, expected test results, expected API responses); after recording, the agent MUST verify these patterns ARE present; if expected content is MISSING, the recording is REJECTED regardless of duration. **(K) Content-verification recording workflow** — mandated workflow: (1) SPECIFY expected content patterns, (2) RECORD the feature execution, (3) EXTRACT all text from the recording, (4) VERIFY expected patterns are present, (5) CHECK for simulated/placeholder content, (6) ACCEPT only if ALL patterns found AND zero bluffs detected, (7) REJECT and re-record if ANY pattern missing or bluff detected. **(L) Root cause analysis REQUIRED for rejected recordings** — when a recording is rejected (missing expected content, bluff detected, or empty capture), the agent MUST investigate WHY before re-recording per §11.4.102; determine the root cause (timing issue, wrong command, tool failure, etc.) and fix it; simply re-recording without understanding WHY the first attempt failed is a §11.4.159 violation. **(M) Real-time monitoring RECOMMENDED** — for complex features, use real-time monitoring that analyzes output DURING recording (not after); this catches issues immediately and allows corrective action before the recording completes. Classification: universal (§11.4.17). Composes §11.4.2/.3/.5/.10/.29/.69/.83/.107/.111/.128/.137/.151/.153/.154/.155/.158/§1.1. Propagation gate CM-COVENANT-114-159-PROPAGATION (literal 11.4.159) + recommended gate CM-WINDOW-VIDEO-VALIDATED + paired §1.1 mutation. **Canonical authority:** constitution submodule Constitution.md §11.4.159. Non-compliance is a release blocker. No escape hatch — no --whole-screen-ok, --cast-only, --skip-vision-validation, --no-cleanup, --simulated-recording-ok, --unprefixed-recording flag.

§11.4.160 — Vision-verified recording + HelixQA bridge mandate (User mandate, 2026-06-21). Compact summary: every video recording for feature/QA evidence MUST be processed through a vision/OCR pipeline that reads on-screen content and confirms expected results BEFORE acceptance; the recording system MUST provide a bridge feeding captured frames to HelixQA’s test infrastructure (or equivalent) for automated read-the-screen verification against SPECIFY-phase expected patterns (§11.4.159(J)); the bridge MUST capture frames at ≤5s intervals, run OCR/vision analysis with a self-validated golden-good/golden-bad analyzer (§11.4.107(10)), compare extracted text against specified patterns, produce a per-frame PASS/FAIL verdict with an evidence path to the frame, and surface failures immediately so the recording can be re-done per §11.4.159(L). The capture interval, OCR confidence floor, and pattern-matching thresholds are project-configured per §11.4.35, calibrated on the project’s own fixtures (§11.4.6). Honest boundary (§11.4.6): vision verification confirms the feature produced expected on-screen output — it does NOT replace §11.4.5 captured-evidence quality analysis (audio/video metrics) nor §11.4.108 runtime-signature verification; FLAG_SECURE/DRM-protected/sink-blanked surfaces document the gap per §11.4.112 and use the §11.4.117 proxy oracle instead. Classification: universal (§11.4.17). Composes §11.4.5/.27/.69/.107(10)/.112/.117/.153/.158/.159(J)/§1.1. Propagation gate CM-COVENANT-114-160-PROPAGATION + recommended gate CM-VISION-VERIFIED-RECORDING-BRIDGE + paired §1.1 mutation.

§11.4.161 — Rootless container runtime mandate (User mandate, 2026-06-21). Compact summary: every project MUST use Podman in rootless mode (or equivalent rootless container runtime) for ALL containerized workloads — Docker

in rootful mode, sudo, or any escalation to root for container management is STRICTLY FORBIDDEN unless the target platform has no rootless option AND that constraint is documented per §11.4.112; the vasic-digital/containers Submodule (§11.4.76) MUST be used as the sole container orchestration layer — no ad-hoc docker/podman commands outside the Submodule's pkg/boot/pkg/compose/pkg/health primitives; if a missing capability forces a raw command, the containers Submodule MUST be extended upstream per §11.4.74 rather than worked around; all container-related integration tests MUST boot infra on-demand via the Submodule (the on-demand-infra invariant). Honest boundary (§11.4.6): rootless execution eliminates the container-to-root privilege-escalation vector — it does NOT replace §11.4.10 credentials-handling (credentials in containers still require leak audits) nor §12.3 container hygiene (memory limits/OOM policies/restart backoff still apply). Classification: universal (§11.4.17). Composes §11.4.76/.74/.10/.112/§12.3/.6. Propagation gate CM-COVENANT-114-161-PROPAGATION + recommended gate CM-ROOTLESS-CONTAINER-RUNTIME + paired §1.1 mutation.

§11.4.162 — OpenDesign UI design system mandate (User mandate, 2026-06-21). Compact summary: every project producing user-facing interfaces (web, desktop, mobile, TUI) MUST use OpenDesign (<https://github.com/nexu-io/open-design>) as the mandatory UI design-and-refinement system — NOT ad-hoc CSS or one-off design tools; install as a project dependency and use its design tokens/themes system for: (a) all color palette definitions supporting BOTH light and dark themes from project brand assets (§11.4.35), (b) typography scale and spacing, (c) component-level design tokens; if a desired UI pattern is not supported, extend OpenDesign upstream per §11.4.74 (extend-don't-reimplement); every UI component MUST ship light + dark theme variants; elements MUST NOT overlap, fonts MUST NOT collide, labels MUST NOT overlay labels — any layout regression is a §11.4.162 violation; all UI changes MUST be covered by the project's standard test types including visual regression tests (before/after screenshots with per-pixel or perceptual-diff PASS/FAIL). Honest boundary (§11.4.6): OpenDesign governs design tokens and themes — it does NOT replace functional testing (§11.4.27), WCAG accessibility assertions (§11.4.107), nor the §11.4.48/§11.4.49 UI-driven and dual-approach testing methodology. Classification: universal (§11.4.17). Composes §11.4.74/.25/.27/.4(b)/.107/.48/.49/.35/.69/§1.1. Propagation gate CM-COVENANT-114-162-PROPAGATION + recommended gate CM-OPENDESIGN-UI-SYSTEM + paired §1.1 mutation.

§11.4.163 — Universal Media Validation & Verification Mandate (User mandate, 2026-06-21). Compact summary: every recorded artifact (video MP4, audio WAV, screenshots PNG, asciinema cast, text output) MUST pass through a MEDIA VALIDATION pipeline before acceptance — OCR for video/screenshots with confidence floor (§11.4.117/.107(12)), transcription for audio, text parsing for asciicast; compare extracted content against SPECIFY-phase expected patterns (§11.4.159(J)); self-validated analyzer with golden-good/golden-bad fixture pair (§11.4.107(10)) — golden-bad MUST FAIL or the pipeline is a bluff gate; produce structured verdict (PASS/FAIL + evidence path + matched/unmatched patterns + pinpoint data on FAIL — which frame/line/timestamp, expected vs actual); triggerable as post-recording AND real-time step (§11.4.159(M)); paired §1.1 mutation ensures golden-bad fixture produces FAIL. Honest boundary: confirms artifact content matches intended patterns — does NOT replace §11.4.5 quality analysis, §11.4.107 liveness, nor §11.4.108 runtime-signature. Classification: universal (§11.4.17). Composes

§11.4.107(10)/.112/.117/.159(J)/.159(K)/.159(M)/.83/.5/.69/.102/.134/§1.1.
Propagation gate CM-COVENANT-114-163-PROPAGATION + recommended gate CM-MEDIA-VALIDATION-PIPELINE + paired §1.1 mutation.

§11.4.164 — Universal Constitution Auto-Propagation & Hook System (User mandate, 2026-06-21). Compact summary: every fetch+pull of the constitution submodule MUST trigger constitution/scripts/post_update_hook.sh (inherited by reference §11.4.28, NEVER copied) that: (a) detects which files changed (Constitution.md/CLAUDE/AGENTS/QWEN/GEMINI + scripts/ hooks/ skills/ mcp/ plugins/) via `git diff --name-only`; (b) registers new/modified skills with the agent's skill system via consumer's scripts/register_skills.sh; (c) registers new/modified MCP servers via consumer's scripts/register_mcp.sh; (d) installs new/modified hooks into `.git/hooks/`; (e) makes scripts executable + validates syntax (`sh -n/bash -n` per §11.4.67); (f) emits a summary to stdout + `.constitution/state/last_update.log`; (g) on failure logs exact file:line and continues (§11.4.6); (h) tested by §1.1 mutation (add skill → hook detects it). Consumer MUST invoke hook after every constitutional pull; failure to invoke is non-compliance. Honest boundary: installs components — does NOT guarantee consumer runtime acceptance nor replace §11.4.32 validation sweep. Classification: universal (§11.4.17). Composes §11.4.28/.32/.35/.67/.77/.102(B)/.122/.31/§1.1. Propagation gate CM-COVENANT-114-164-PROPAGATION + recommended gate CM-CONSTITUTION-AUTO-PROPAGATION + paired §1.1 mutation.

§11.4.165 — Universal Independent Verification Agent Mandate (User mandate, 2026-06-21). Compact summary: every code change OR recorded media artifact MUST pass an INDEPENDENT verifier structurally separate from the author (§11.4.70/.20, NEVER author self-review — §11.4.92 PRECEDES, never satisfies), producing structured findings with evidence paths and iterating to GO per §11.4.134. (a) CODE: §11.4.142 review + build+test diff scope + run paired §1.1 mutations + verify §11.4.108 runtime-signature registry; (b) MEDIA: run §11.4.163 pipeline + confirm genuine content (no mock/stub/placeholder §11.4.159(I)) + confirm format (MP4 H.264/WAV/PNG/cast+mp4 §11.4.159(B)) + cross-check evidence path; (c) DOCS: validate HTML+PDF+DOCX exports current §11.4.65 + revision header incremented §11.4.44 + §11.4.86 fingerprint match; (d) CONFIG: validate YAML/JSON/TOML syntax + schema cross-reference + §11.4.10 leak check; (e–f) structured findings, iterate to zero-finding GO; (g) verifier self-validated by §1.1 mutation (knowingly broken change MUST be caught). Honest boundary: confirms source/artifact integrity — does NOT replace §11.4.108 runtime-signature nor §11.4.40 full-suite retest. Classification: universal (§11.4.17). Composes
§11.4.142/.163/.44/.65/.86/.108/.125/.134/.10/.40/.92/.70/.20/.102/.159(I)/.159(B)/
§1.1. Propagation gate CM-COVENANT-114-165-PROPAGATION + recommended gate CM-INDEPENDENT-VERIFICATION-AGENT + paired §1.1 mutation.